

**НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
“КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО”**

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

**До захисту допущено:
В.о. завідувача кафедри
Михайло НОВОТАРСЬКИЙ**

(підпис)

“__” _____ 2025 р.

Дипломний проєкт

**на здобуття ступеня бакалавра
за освітньо-професійною програмою “Інженерія програмного
забезпечення комп’ютерних систем”
спеціальності 121 “Інженерія програмного забезпечення”**

на тему: Навчальна платформа для розміщення та проходження курсів

Виконав : студент 4 курсу, групи ІМ-11
(шифр групи)

Бащак Ярина Володимирівна

(прізвище, ім’я, по батькові)

(підпис)

Керівник асистент Гайдай А. Р.

(посада, науковий ступінь, вчене звання, прізвище та ініціали)

(підпис)

Консультант (нормоконтроль) асистент Нікольський С. С.

(назва розділу)

(посада, вчене звання, науковий ступінь, прізвище та ініціали)

(підпис)

Рецензент доцент кафедри ІСТ, к.т.н. Шимкович В. М.

(посада, науковий ступінь, вчене звання, прізвище та ініціали)

(підпис)

Засвідчую, що у цьому дипломному
проєкті немає запозичень з праць інших
авторів без відповідних посилань.

Студент _____
(підпис)

Київ – 2025 р.

**НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
“КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО”**

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

Рівень вищої освіти – перший (бакалавр)

Освітньо-професійна програма

“Інженерія програмного забезпечення комп’ютерних систем”

спеціальності 121 “Інженерія програмного забезпечення”

ЗАТВЕРДЖУЮ

В.о. завідувача кафедри

Михайло НОВОТАРСЬКИЙ

(підпис)

“ ____ ” _____ 2025 р.

ЗАВДАННЯ

на бакалаврський дипломний проєкт студента

Бащак Ярини Володимирівни

1. Тема проєкту Навчальна платформа для розміщення та проходження курсів
керівник проєкту Гайдай Анатолій Русланович, асистент,
(прізвище, ім’я, по батькові, науковий ступінь, вчене звання)
затверджені наказом по університету від 23 травня 2025 року №1705-с
2. Термін здачі студентом закінченого проєкту 2 червня 2025 р.
3. Вихідні дані до проєкту технічна документація, теоретичні дані.
4. Зміст розрахунково-пояснювальної записки (перелік питань, які розробляються)
Розділ 1. Огляд платформ для онлайн-курсів.
Розділ 2. Інструменти і технології для розробки платформи.
Розділ 3. Розробка ключових функцій платформи.

Розділ 4. Тестування і оцінка результатів.

5. Перелік графічного матеріалу (з точним позначенням обов'язкових креслень) структурна схема системи, функціональна схема (діаграма класів), алгоритм дій програмного забезпечення.

6. Консультанта проєкту, з вказівкою розділів проєкту, які до них вносяться

Розділ	Консультант	Підпис, дата	
		Завдання видав	Завдання прийняв
Нормоконтроль	Нікольський С.С.		

7. Дата видачі завдання «3» січня 2025 р.

Календарний план

№ п/п	Найменування етапів дипломного проєкту	Терміни виконання етапів проєкту	Примітки
1.	<i>Затвердження теми проєкту</i>	<i>14.10.2024-29.11.2024</i>	
2.	<i>Вивчення та аналіз завдання</i>	<i>02.12.2024-05.01.2025</i>	
3.	<i>Розробка архітектури та загальної структури системи</i>	<i>13.01.2025-14.02.2025</i>	
4.	<i>Розробка структур окремих підсистем</i>	<i>03.02.2025-21.03.2025</i>	
5.	<i>Програмна реалізація системи</i>	<i>31.03.2025-11.05.2025</i>	
6.	<i>Оформлення пояснювальної записки</i>	<i>14.04.2025-31.06.2025</i>	
7.	<i>Захист програмного продукту</i>	<i>02.06.2025</i>	
8.	<i>Передзахист</i>	<i>06.06.2025</i>	
9.	<i>Захист</i>	<i>17.06.2025</i>	

Студент-дипломник _____ Ярина БАЩАК

(підпис)

Керівник проєкту _____ Анатолій ГАЙДАЙ

(підпис)

АНОТАЦІЯ

У дипломній роботі досліджено переваги використання навчальних платформ в умовах зростання популярності дистанційної освіти. Проаналізовано наявні рішення провідних гравців на ринку, зокрема визначено їхню цільову аудиторію та можливості розміщення курсів. На основі цього аналізу визначено потреби користувачів і ключові компоненти навчальної платформи, зокрема необхідність інтеграції з хмарним сервісом для збереження контенту.

Розроблено архітектуру та реалізовано навчальну платформу для розміщення й проходження курсів. Платформа підтримує авторизацію, створення акаунтів і збереження прогресу користувачів. Для викладачів передбачено можливість створення нових курсів, додавання відеоуроків, тестів і встановлення вартості. Серверну частину реалізовано з використанням фреймворку ASP.NET Core, клієнтську – за допомогою React з використанням мови TypeScript, як СУБД обрано PostgreSQL. Для зберігання відеоконтенту інтегровано хмарний сервіс Cloudinary, який надає безплатний доступ до сховища в межах визначених лімітів. Також реалізовано механізм здійснення тестових оплат за доступ до курсів через платіжний сервіс Stripe.

Ключові слова: навчальна платформа, онлайн-курс, відеоурок, інтеграція, .NET, React, API, хмарне сховище, платіж.

ANNOTATION

This thesis explores the advantages of using educational platforms in the context of the growing popularity of distance learning. Existing solutions offered by leading market players are analyzed, with particular attention paid to their target audiences and the ability to publish courses. Based on this analysis, user needs and the key components of a modern learning platform are identified, including the necessity of integrating with a cloud service for content storage.

The architecture of the system was designed and a learning platform for creating and taking courses was implemented. The platform supports user authentication, account creation, and progress tracking. For instructors, the system enables the creation of new courses, addition of video lessons, quizzes, and setting course prices. The server-side was developed using the ASP.NET Core framework, while the client-side was implemented in React with TypeScript. PostgreSQL was chosen as the database management system. To store video content, the Cloudinary cloud service was integrated, which provides free storage within specified limits. A mechanism for handling test payments for course access via the Stripe payment service was also implemented.

Keywords: learning platform, online course, video lesson, integration, .NET, React, API, cloud storage, payment.

справки	Формат	Значення	Найменування	Кіл. листів	№ екземпля	Додаток
			Документація загальна			
			Знову розроблена			
	A4	ІАЛЦ.467200.002 ТЗ	Навчальна платформа для	4		
			розміщення та проходження			
			курсів			
			Технічне завдання			
	A4	ІАЛЦ.467200.003 ПЗ	Навчальна платформа для	70		
			розміщення та проходження			
			курсів			
			Пояснювальна записка			
	A4	ІАЛЦ.467200.004 Д1	Навчальна платформа для	1		
			розміщення та проходження			
			курсів			
			Структурна схема			
			системи			
	A4	ІАЛЦ.4672008.005 Д2	Навчальна платформа для	1		
			розміщення та проходження			
			курсів			
			Функціональна схема			
			(діаграма класів)			
	A4	ІАЛЦ.467200.006 Д3	Алгоритм дій програмного	1		
			забезпечення			
			Принципова схема			
			(блок-схема алгоритму)			
	A4	ІАЛЦ.467200.007 Д4	Навчальна платформа для	44		
			розміщення та проходження			
			курсів			
			Текст програмного коду			

					ІАЛЦ.467200.001 ОА							
Зм	Лист	№ докум.	Підп	Дата								
Розроб		Бащак Я.В.			Навчальна платформа для розміщення та проходження курсів Опис альбому				Літ.	Аркуш	Аркушів	
Перев		Гайдай А.Р.									1	1
					КПІ ім. Ігоря Сікорського ФІОТ ІМ-11							

ТЕХНІЧНЕ ЗАВДАННЯ ДО ДИПЛОМНОГО ПРОЄКТУ

на тему: «Навчальна платформа для розміщення та проходження курсів»

Київ – 2025

ЗМІСТ

НАЙМЕНУВАННЯ ТА ОБЛАСТЬ ЗАСТОСУВАННЯ	2
ПІДСТАВИ ДЛЯ РОЗРОБКИ	2
МЕТА ТА ПРИЗНАЧЕННЯ РОЗРОБКИ.....	2
ДЖЕРЕЛА РОЗРОБКИ.....	3
ТЕХНІЧНІ ВИМОГИ.....	3
Вимоги до розробленого продукту	3
Вимоги до програмного забезпечення	3
Вимоги до апаратної частини	3
ЕТАПИ РОЗРОБКИ	4

					ІАЛЦ.467200.002 ТЗ				
		№ докум.	Підпис	Дата					
Розробив	Бащак Я. В.				Навчальна платформа для розміщення та проходження курсів Технічне завдання		Літ.	Аркуш	Аркушів
Перевірив	Гайдай А. Р.							1	4
Реценз.	Шимкович В. М.						КПІ ім. Ігоря Сікорського, ФІОТ, ІМ-11		
Н. Контр.	Нікольський С. С.								
Затвердив									

1 НАЙМЕНУВАННЯ ТА ОБЛАСТЬ ЗАСТОСУВАННЯ

Це технічне завдання поширюється на розробку навчальної платформи для розміщення та проходження курсів, а також на подальшу підтримку та вдосконалення розробленої системи.

Областю застосування цієї системи є організація та автоматизація онлайн-навчання, зокрема для незалежних викладачів, експертів чи репетиторів, які бажають самостійно публікувати освітні матеріали і надавати доступ до нього на платній основі.

2 ПРИЧИНИ ДЛЯ РОЗРОБКИ

Підставою для розробки є завдання для виконання роботи кваліфікаційно-освітнього рівня «бакалавр інженерії програмного забезпечення», який був затверджений факультетом “Інформатики та обчислювальної техніки” кафедрою обчислювальної техніки Національного технічного Університету України «Київський Політехнічний інститут ім. Ігоря Сікорського».

3 МЕТА ТА ПРИЗНАЧЕННЯ РОЗРОБКИ

Метою та призначенням цього проєкту є розробка навчальної платформи для розміщення та проходження курсів, що вирішуватиме завдання:

- забезпечення масштабованої й безпечної архітектури системи із застосуванням підтримуваних технологій;
- реалізації механізму платного доступу до навчальних матеріалів;
- інтеграції з хмарним сховищем;
- надання викладачам можливості самостійно створювати та публікувати онлайн-курси;
- організації структурованого навчального процесу для студентів.

					ІАЛЦ.467200.002 ТЗ	Арк.
						2
Зм.	Арк.	№ докум.	Підпис	Дата		

4 ДЖЕРЕЛА РОЗРОБКИ

Джерелом розробки дипломного проекту є науково-технічна література в області розробки програмного забезпечення, офіційні документації, публікації та статті в мережі Інтернет на дану тему.

5 ТЕХНІЧНІ ВИМОГИ

5.1. Вимоги до розробленого продукту

Розроблена система має виконувати такі вимоги:

- Технічна коректність і актуальність інформації, що використовується в системі.
- Наявність простого й інтуїтивно зрозумілого інтерфейсу.
- Можливість створення користувачами особистих акаунтів на платформі.
- Надання викладачам функціональності для створення та публікації курсів, управління їхнім змістом і встановлення вартості.
- Надання користувачам можливості обирати курси, отримувати доступ до навчальних матеріалів і проходити тестування для перевірки рівня засвоєння тем, підготовлених викладачем.

5.2. Вимоги до програмного забезпечення

- ОС Windows, Mac чи Linux.
- Visual Studio 2017 IDE версії або вище.

5.3. Вимоги до апаратної частини

- ЦП не менше ніж Intel® Core (TM) i3-2100T.
- ROM не менше ніж 64 ГБ.
- RAM не менше ніж 4 ГБ.

					ІАЛЦ.467200.002 ТЗ	Арк.
						3
Зм.	Арк.	№ докум.	Підпис	Дата		

6 ЕТАПИ РОЗРОБКИ

Назва етапів виконання	Термін виконання
Затвердження теми роботи	14.10.2024-29.11.2024
Вивчення та аналіз завдання	02.12.2024-05.01.2025
Розробка архітектури та загальної структури системи	13.01.2025-14.02.2025
Розробка структур окремих частин системи	03.02.2025-21.03.2025
Програмна реалізація системи	31.03.2025-11.05.2025
Виправлення помилок	04.04.2025-18.05.2025
Оформлення пояснювальної записки	14.04.2025-31.06.2025

**ПОЯСНЮВАЛЬНА ЗАПИСКА
ДО ДИПЛОМНОГО ПРОЄКТУ**

на тему: «Навчальна платформа для розміщення та проходження курсів»

Київ – 2025

ЗМІСТ

ПЕРЕЛІК СКОРОЧЕНЬ	4
ВСТУП	5
РОЗДІЛ 1 ОГЛЯД ПЛАТФОРМ ДЛЯ ОНЛАЙН-КУРСІВ	6
1.1 Онлайн-освіта: причини популярності та нові можливості	6
1.2 Переваги використання онлайн-платформ для навчання.....	7
1.3 Аналіз існуючих навчальних платформ.....	9
1.4 Потреби користувачів та ключові компоненти успішної навчальної платформи	13
1.5 Проблеми зберігання великих обсягів відеоконтенту: потреба в хмарних рішеннях	15
ВИСНОВКИ ДО РОЗДІЛУ 1	16
РОЗДІЛ 2 ІНСТРУМЕНТИ ТА ТЕХНОЛОГІЇ ДЛЯ РОЗРОБКИ ПЛАТФОРМИ	17
2.1 Архітектура навчальної платформи	17
2.1.1 Загальна клієнт-серверна архітектура	18
2.1.2 Основні програмні компоненти.....	19
2.2 Вибір технологій та інструментів розробки	19
2.2.1 Фреймворки та бібліотеки для фронтенду	20
2.2.2 Стек технологій для бекенду	21
2.2.3 Вибір бази даних	23
2.2.4 Хмарне сховище для збереження відео	23
2.2.5 Платіжна система для обробки оплат	25
2.3 Механізм авторизації користувачів	25
ВИСНОВКИ ДО РОЗДІЛУ 2	27

					ІАЛЦ.467200.003 ПЗ			
		№ докум.	Підпис	Дата				
Розробив	Бацак Я. В.				Навчальна платформа для розміщення та проходження курсів Пояснювальна записка		Літ.	Аркуш
Перевірив	Гайдай А. Р.							1
Реценз.	Шимкович В. М.						КПІ ім. Ігоря Сікорського, ФІОТ, ІМ-11	
Н. Контр.	Нікольський С. С.							
Затвердив								
							70	

РОЗДІЛ 3 РОЗРОБКА КЛЮЧОВИХ ФУНКЦІЙ ПЛАТФОРМИ.....	28
3.1 Побудова серверної частини платформи з використанням .NET та EF Core	28
3.2 Реалізація системи управління акаунтами та обмеження доступу ..	29
3.2.1 Права доступу через ролі та клейми	29
3.2.2 Реєстрація, автентифікація та генерація токенів на сервері.....	30
3.2.3 Збереження токена та авторизація на клієнтській частині	31
3.3 Побудова структури курсів: розділи, уроки, тести	33
3.4 Механізм збереження та відображення прогресу навчання	35
3.4.1 Структура таблиці збереження прогресу	35
3.4.2 Включення прогресу користувача у відповідь API.....	36
3.4.3 Візуалізація стану проходження уроків на клієнтській частині	37
3.4.4 Автоматичне оновлення прогресу	38
3.5 Інтеграція з хмарним сховищем Cloudinary	39
3.5.1 Підготовка середовища для інтеграції з Cloudinary	39
3.5.2 Додавання та видалення відео на сервері.....	40
3.6 Впровадження інтеграції платіжної системи Stripe.....	41
3.6.1 Налаштування середовища для інтеграції з Stripe	41
3.6.2 Структура таблиць бази даних для обробки оплат	42
3.6.3 Створення платежу та проведення оплати.....	44
3.6.4 Підтвердження платежу через Webhook Stripe.....	46
ВИСНОВКИ ДО РОЗДІЛУ 3	48
РОЗДІЛ 4 ТЕСТУВАННЯ І ОЦІНКА РЕЗУЛЬТАТІВ.....	49
4.1 Тестування авторизації та реєстрації	49
4.2 Тестування функціональності створення навчального курсу.....	52
4.2.1 Створення курсу, розділів, уроків.....	52
4.2.2 Завантаження відео, перевірка інтеграції з хмарним сховищем	54
4.2.3 Додавання тестів	55
4.2.4 Публікація і поширення курсу.....	56

					ІАЛЦ.467200.003 ПЗ	Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		2

4.3 Взаємодія користувача з навчальними матеріалами.....	58
4.3.1 Придбання курсу.....	58
4.3.2 Перегляд відеоуроків.....	60
4.3.3 Проходження тестування.....	61
4.3.4 Перегляд прогресу навчання.....	64
ВИСНОВКИ ДО РОЗДІЛУ 4	66
ВИСНОВКИ.....	67
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	68

					ІАЛЦ.467200.003 ПЗ	Арк.
						3
Зм.	Арк.	№ докум.	Підпис	Дата		

ПЕРЕЛІК СКОРОЧЕНЬ

API	(Application Programming Interface) Інтерфейс прикладного програмування
CRUD	(Create, Read, Update, Delete) Створення, читання, оновлення, видалення
DTO	(Data Transfer Object) Об'єкт передавання даних
HTTP	(HyperText Transfer Protocol) Протокол передавання гіпертексту
JWT	(JSON Web Token) Вебтокен у форматі JSON
ORM	(Object-Relational Mapping) Об'єктно-реляційне відображення
REST	(Representational State Transfer) Передавання репрезентативного стану
SPA	(Single Page Application) Односторінковий застосунок
SQL	(Structured Query Language) Мова структурованих запитів
URL	(Uniform Resource Locator) Уніфікований локатор ресурсу
БД	База даних
СУБД	Система управління базами даних

					ІАЛЦ.467200.003 ПЗ	Арк.
						4
Зм.	Арк.	№ докум.	Підпис	Дата		

ВСТУП

В умовах стрімкої цифровізації знань і навичок особливої популярності набуває онлайн-навчання в індивідуальному форматі. Все більше викладачів і репетиторів прагнуть ділитися досвідом та монетизувати свої знання через створення авторських курсів. Такий формат дає змогу залучити значно ширшу аудиторію учнів порівняно з традиційним офлайн-навчанням. Це створює попит на зручні та доступні навчальні платформи.

Проблематика полягає в тому, що більшість наявних освітніх платформ орієнтовані на великі навчальні установи, компанії чи організації та часто мають складні вимоги до публікації курсів і високу конкуренцію, що обмежує можливості для окремих авторів. Крім того, постає технічна потреба в ефективному зберіганні відеоконтенту, що є основним типом матеріалу для таких курсів. Це вимагає розробки рішення, яке б відповідало як функціональним, так і інженерним вимогам сучасного ринку онлайн-освіти.

Метою дипломного проєкту є розробка навчальної платформи для розміщення та проходження курсів, яка надає викладачам можливість створювати структуровані відеокурси, встановлювати їхню вартість і додавати тести для перевірки знань. Платформа має підтримувати авторизацію користувачів, збереження прогресу навчання та інтеграцію з хмарним сховищем для зберігання відео та платіжною системою для обробки оплат.

Практичною цінністю проєкту є створення доступного технічного рішення для незалежних авторів курсів, яке дає змогу масштабувати власну освітню діяльність, автоматизувати навчальний процес та монетизувати знання, створивши додатковий або основний дохід. Така система може бути основою для подальшого розвитку персоналізованих освітніх продуктів і розширення ринку індивідуального онлайн-навчання.

					ІАЛЦ.467200.003 ПЗ	Арк.
						5
Зм.	Арк.	№ докум.	Підпис	Дата		

РОЗДІЛ 1 ОГЛЯД ПЛАТФОРМ ДЛЯ ОНЛАЙН-КУРСІВ

1.1 Онлайн-освіта: причини популярності та нові можливості

За останні роки онлайн-освіта стала повноцінним форматом здобуття знань. Першим викликом стала пандемія, яка спричинила вимушений стрімкий перехід навчальних закладів до проведення занять у форматі онлайн-зустрічей [2]. Водночас популярності набули платформи, що надавали викладачам можливість розміщувати матеріали для учнів: записи уроків і лекцій, умови завдань, тести тощо. Онлайн-формати стали звичними як для учнів, так і для педагогів.

З початком повномасштабного вторгнення в Україну питання безпеки студентів і викладачів стало ще більш актуальним. У цей період онлайн-інструменти виявилися особливо корисними. Більшість учасників онлайн-навчання вже були знайомі з цим форматами, що дозволило швидко адаптуватися до нових умов. До того ж частина учнів була змушена виїхати за кордон, але завдяки освітнім платформам вони змогли продовжити навчання дистанційно, незалежно від місця перебування.

В процесі більшість учнів та викладачів помітили переваги цифрового формату: можливість навчатися у зручний час, не витратити ресурси на дорогу, а також зберігати доступ до матеріалів, щоб переглядати їх у будь-який момент. Як наслідок, сформувалося нове ставлення до онлайн-освіти – як до повноцінного, зрілого способу навчання, до якого тепер користувачі готові не лише технічно, а й психологічно.

Учні та студенти дедалі частіше звертаються до онлайн-курсів для підготовки до національного мультипредметного тесту, покращення

					ІАЛЦ.467200.003 ПЗ	Арк.
						6
Зм.	Арк.	№ докум.	Підпис	Дата		

успішності в школі, здобуття необхідних професійних навичок для майбутнього пошуку роботи тощо. Онлайн-формат дозволяє їм зосередитися на конкретних темах, які чітко розділені, і не прив'язуватись до місця та часу.

Своєю чергою, викладачі та репетитори отримали нові можливості для розвитку і в певних випадках навіть для заробітку. Через зростання інтересу до онлайн-навчання вони мають змогу створювати власні курси підготовки відповідно до запитів учнів. Онлайн-формат дозволяє їм масштабувати свою діяльність, залучати більшу кількість учнів незалежно від географічного розташування. Викладачі можуть записати уроки наперед і викладати у зручний час, що зменшує залежність від кількості проведених занять у реальному часі та, своєю чергою, дає змогу краще планувати навантаження та збільшувати дохід.

Таким чином, онлайн-освіта здобула свою популярність у відповідь на сучасні реалії. Учні та студенти отримали змогу навчатися, незалежно від місця перебування, а викладачі – створювати та поширювати власні навчальні матеріали. Саме для задоволення цих потреб розробляється навчальна платформа, що є об'єктом дипломного проєкту. Звісно, одному розробнику в стиснуті терміни неможливо реалізувати широкий вибір функціональності, які ми зазвичай очікуємо від комерційних платформ, але основні базові можливості будуть реалізовані.

1.2 Переваги використання онлайн-платформ для навчання

Однією з головних переваг онлайн-платформ є гнучкість. Навчання з інтернет-ресурсами дає можливість користувачеві самостійно вибирати зручний час для занять, проходження тестів, повторення. Для тих, для кого таке навчання є додатковим – учнів шкіл, студентів чи працевлаштованих людей, це особливо важливо. Онлайн-платформи дають змогу керувати навчанням для

					ІАЛЦ.467200.003 ПЗ	Арк.
						7
Зм.	Арк.	№ докум.	Підпис	Дата		

себе і під свої потреби, на відміну від традиційного навчання, де є чіткий розклад [3].

Також користувач може переглядати відеоуроки заново, зупиняти їх і повертатися до складних моментів, якщо не було зрозуміло з першого разу, і працювати у своєму темпі. Матеріали залишаються доступними, що зменшує стрес чи страх щось не запам'ятати, не зрозуміти вчасно. Завдяки цьому, учень не обмежений тривалістю одного уроку, як зазвичай буває в школі, і теми можуть бути засвоєні краще.

Онлайн-курси зазвичай гарно структуровані: поділені на секції по темах, урок має як теоретичну, так і практичну частини, а після кожного уроку передбачені тести. Це все підвищує зацікавленість і мотивацію користувача, бо він розуміє, чого очікувати далі. Виконуючи інтерактивні тести учень також може побачити свої слабкі місця і за потреби переглянути тему ще раз.

На платформах зазвичай зберігається прогрес учня, що є дуже зручною функціональністю. Студент може бачити, скільки матеріалу він вже опрацював, де зупинився, скільки тестів пройшов і які результати отримав, що також розвиває самоконтроль під час навчання. А викладач, своєю чергою, може бачити прогрес кожного учня, автоматично перевіряти роботи та збирати результати – залежить від рівня необхідного зворотного зв'язку на курсі.

Викладач теж може мати переваги, працюючи через платформу. По-перше, він може заздалегідь записати відеоуроки. Це дозволяє виконати роботу тоді, коли йому зручно, планувати наперед, не залежати від чіткого розкладу. По-друге, автоматизувати перевірку тестів. Викладачу не потрібно перевіряти кожну роботу, йому достатньо вказати правильні відповіді при створенні завдання. На додаток, деякі платформи дозволяють залишати коментарі з поясненнями до правильних і неправильних відповідей.

Економія часу – ще одна з важливих переваг як для студентів, так і для викладачів. Викладач може зосередитися на наповненні курсу, розділенні складніших тем на коротші уроки, не витрачаючи час на проведення одних і

					ІАЛЦ.467200.003 ПЗ	Арк.
						8
Зм.	Арк.	№ докум.	Підпис	Дата		

тих самих занять. Студент же може вивчати підготовлений матеріал, рухатись у своєму темпі, мати постійний доступ до уроків з дому.

Отже, навчальні платформи можуть бути зручним способом систематизувати свої знання, підтримувати мотивацію навчатися, використовувати час ефективніше тощо. Студенти мають можливість навчатися тоді, коли їм зручно, кілька разів повторювати матеріал, на практиці закріплювати знання та бачити загальну картину, де на шляху до завершення вони зараз перебувають. А для викладачів це – змога створювати чіткі, структуровані уроки в записі та не проводити одні й ті ж заняття щоразу, оптимізувавши свої зусилля.

1.3 Аналіз існуючих навчальних платформ

Платформи для онлайн-освіти – не новий ринок, але він все ще розвивається. Вони відрізняються за структурою курсів, ціною, політикою відбору авторів курсів та цільовою аудиторією. Перед тим, як розробляти нову платформу, варто проаналізувати основних гравців на ринку, їх відмінності, і визначити напрямок, в якому розробляти дипломний проєкт.

Платформа, яку точно варто розглянути – це Udemu. Вона пропонує величезну кількість курсів по різних тематиках [4]. Студенти можуть навчатися у власному темпі, залишати відгуки, бачити рейтинг кожного курсу, отримувати сертифікати.

Інструктором на Udemu може стати майже кожен після реєстрації та заповнення необхідних полів у профілі. Для авторів є можливість створити власний курс, керувати його наповненням та виставити ціну. На рис. 1.1 зображено інтерфейс користувача, який проходить курс на Udemu.

Курси на Udemu поділені на секції і кожен викладач сам визначає скільки відеоматеріалів буде містити курс, скільки тестувань входитиме, і чи будуть

					ІАЛЦ.467200.003 ПЗ	Арк.
						9
Зм.	Арк.	№ докум.	Підпис	Дата		

вони взагалі. Тести можна проходити безліч разів до досягнення бажаного результату. Користувач одразу бачить результати і пояснення до відповідей.

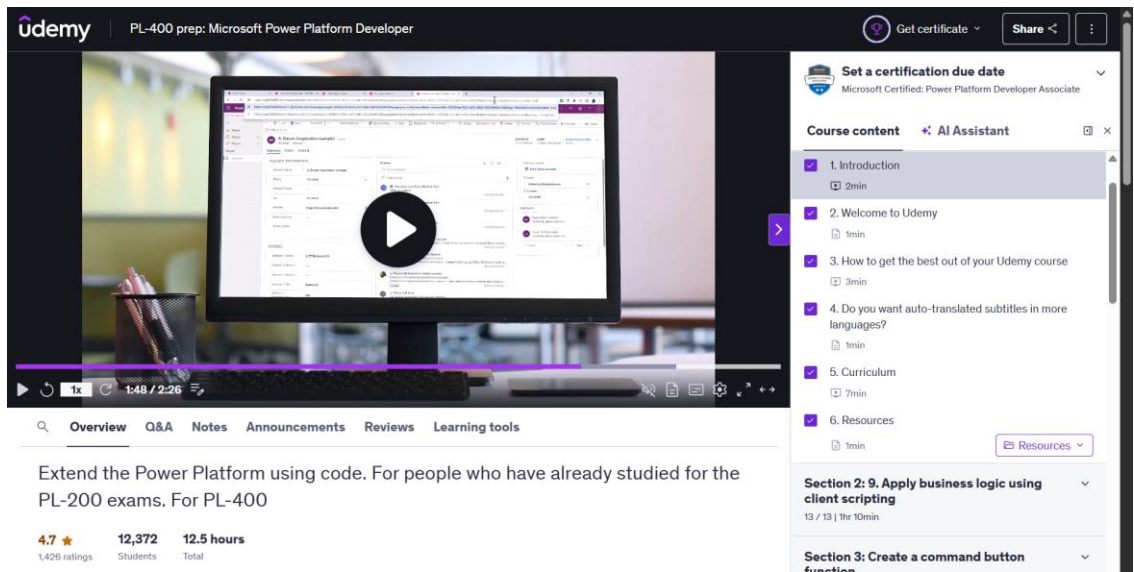


Рисунок 1.1 – Інтерфейс Udemy для студента при проходженні курсу

Але варто зазначити, що конкуренція на платформі надзвичайно висока. Це створює додаткові виклики для нових авторів, яким складно привернути увагу до власного курсу без активного просування і позитивних відгуків.

Coursera – ще один відомий гравець на ринку. Вона працює у співпраці з провідними університетами та компаніями, надає доступ до професійних курсів як на платній, так і на безоплатній основі. На рис. 1.2 зображено користувацький інтерфейс на Coursera при відкритті курсу.

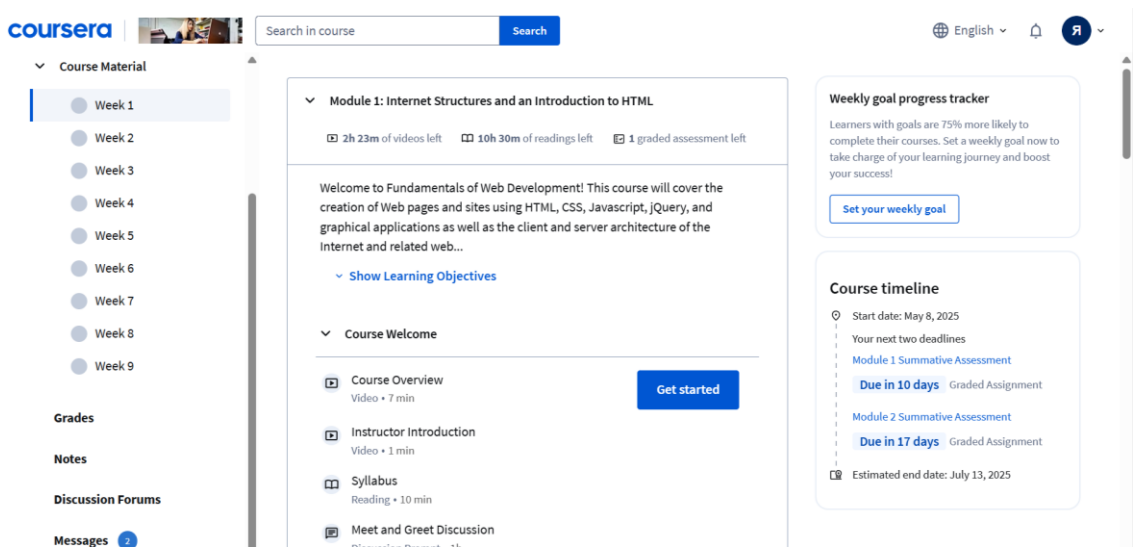


Рисунок 1.2 – Інтерфейс огляду курсу на Coursera

Ця платформа більше орієнтована на академічну освіту і це відображається в структурі її курсів і загальній політиці платформи. На Coursera навчання зазвичай має чіткі терміни: матеріали поділені на тижні, кожен з яких може мати проміжні і фінальні оцінювання. Тести мають прохідний бал і ліміт на кількість спроб. Всі результати тестів і завдань зібрані в таблицю на окремій сторінці, які потім визначають наскільки успішно завершено курс.

Також варто зазначити, що не будь-хто може створювати навчальні курси на Coursera. Ця платформа допускає створення курсів лише для тих, хто співпрацює з її партнерами, тобто університетами чи компаніями [5]. Це обмежує можливість публікування курсів на Coursera для окремих авторів.

Наступна платформа, яку варто оглянути – Moodle. Вона пропонує інструменти для організації дистанційного навчання: управління курсами, комунікації з учнями, обміну файлами, створення тестів, ведення і перегляду успішності. Через це вона часто використовується навчальними закладами, зокрема 2/3 закладів вищої освіти в Європі її обрали [6]. На рис. 1.3 представлено вигляд сторінки на Moodle з боку студента, який переглядає матеріали курсу і статус своєї роботи.

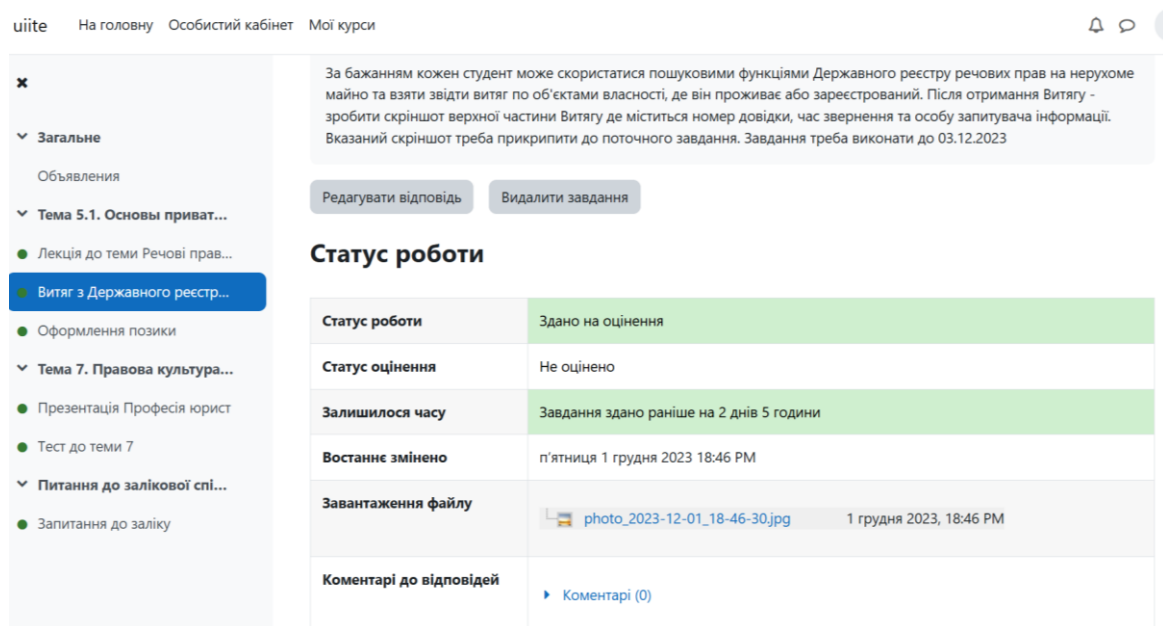


Рисунок 1.3 – Інтерфейс курсу на Moodle

Однак впровадження використання Moodle не таке просте, бо для Moodle треба забезпечити власний вебсервер або зовнішній хостинг [7]. Це вимагає як додаткових технічних знань і підтримки на постійній основі, так і звичайно фінансових витрат. Такий рівень складності робить Moodle нераціональним варіантом для індивідуального викладача чи репетитора, щоб швидко і легко створити онлайн-курс.

Порівняльна характеристика розглянутих платформ наведена у таблиці 1.1.

Таблиця 1.1 – Порівняльна характеристика популярних платформ онлайн-курсів

Критерій	Udemy	Coursera	Moodle
Цільова аудиторія	Будь-хто, хто бажає здобувати знання	Будь-хто, хто бажає здобувати знання	Студенти освітніх установ
Хто може створювати курси	Будь-хто після реєстрації	Лише партнери Coursera	Викладачі освітніх установ, адміністратори
Можливість монетизації	Так, можна виставити ціну на курс	Лише в межах партнерських програм	Ні
Інтерфейс	Зручний, проста навігація	Зручний, структурований, але складніший	Менш інтуїтивний, часто складний інтерфейс
Підходить для індивідуального викладача	Так	Ні	Ні

Як видно з таблиці, індивідуальному викладачу не так легко опублікувати власний курс на більшості популярних платформ. Наприклад,

Coursera співпрацює лише з офіційними партнерами – університетами чи організаціями, а Moodle взагалі орієнтований переважно на внутрішнє використання в освітніх установах і не підтримує монетизацію контенту. Навіть у випадку з Udemu, яка надає можливість публікації курсів будь-кому, існують свої обмеження, зокрема висока конкуренція.

Оптимальним варіантом із порівняних платформ є Udemu, адже вона підтримує публікацію та продаж курсів незалежними викладачами. Проте це рішення не є досконалим, бо платформа має власні правила модерації і перевірки курсів.

Це обґрунтовує потребу в розробці власної навчальної платформи для розміщення та проходження курсів, що дає викладачам змогу безпосередньо створювати, структурувати та продавати свої курси без обмежень, характерних для великих платформ.

1.4 Потреби користувачів та ключові компоненти успішної навчальної платформи

Успішна онлайн-платформа для навчання повинна враховувати потреби обох сторін: студентів та викладачів, і мати зручний, інтуїтивний інтерфейс. Основними компонентами такої платформи є ті, що забезпечують зручну взаємодію, інтерактивність і персоналізований досвід.

Кожен студент повинен мати можливість створити персональний акаунт. Там буде зберігатися вся його історія навчання, включаючи пройдені уроки, результати тестів і загальний прогрес. Викладачі, у свою чергу, повинні мати доступ до панелі керування, де вони можуть створювати курси, завантажувати навчальні матеріали та слідкувати за успішністю студентів.

Курси на платформі мають бути добре структуровані та зручні для навігації. Кожен курс може складатися з секцій, а секції – з уроків. Кожен урок містить відеоматеріали та тестові завдання для перевірки засвоєння знань.

					ІАЛЦ.467200.003 ПЗ	Арк.
						13
Зм.	Арк.	№ докум.	Підпис	Дата		

Відеоуроки повинні бути організовані так, щоб вони охоплювали теоретичний матеріал, а також демонстрували практичне застосування знань. Наприклад, одне відео може бути присвячене теоретичним аспектам теми, а друге – пояснювати, як розв'язувати типові і складніші задачі, що дозволяє студентам закріпити отриману інформацію на практиці. Такий підхід забезпечить глибший розбір теми і продуктивніше навчання.

Після кожного уроку можуть бути тести, вправи чи практичні завдання. Це дозволяє студентам перевіряти і закріплювати свої знання. Також має бути можливість переглядати відео кілька разів, повторно проходити тести для закріплення, вивчати матеріал у своєму темпі.

Студенти повинні мати можливість бачити, який матеріал вони вже пройшли і які результати тестувань отримали. Це реалізується за допомогою авторизації, щоб пов'язати акаунт студента з уроками і фіксувати результати. Це дозволяє не лише контролювати власний процес, а й мотивувати учня продовжувати навчання.

Також не менш важливою складовою успішної платформи є інтуїтивний інтерфейс. Необхідно забезпечити зручні кнопки навігації на кожній сторінці, зрозумілі назви вкладок. Має бути 2 головні сторінки: загальна з каталогом доступних курсів з можливістю придбати або продовжити навчання і для перегляду і створення нових курсів викладачами. При відкритті курсу користувач має бачити чітко розділені секції з уроками, а також відмітки – які уроки вже пройдено, які в процесі, а які ще не розпочаті. Викладач в свою чергу має бачити список своїх курсів з короткою інформацією про кожен, мати можливість створити, редагувати, приховати курс. В середині кожного курсу – зрозуміла структура для додавання нових секцій, уроків і тестів.

Таким чином, ключовими компонентами навчальної платформи є авторизація, чітка структура курсів, інтерактивні елементи для закріплення знань, збереження прогресу студентів та зрозумілий інтерфейс.

					ІАЛЦ.467200.003 ПЗ	Арк.
						14
Зм.	Арк.	№ докум.	Підпис	Дата		

1.5 Проблеми зберігання великих обсягів відеоконтенту: потреба в хмарних рішеннях

Відео займають набагато більше пам'яті, ніж звичайні записи в таблицях. Стандартні БД не призначені для збереження файлів безпосередньо в них. По-перше, вони не здатні швидко працювати з доступом до великих медіафайлів, а по-друге, якщо зберігати відео безпосередньо в таблицях, розмір БД буде дуже швидко рости, що вплине негативно на продуктивність усієї платформи.

Саме тому кожна навчальна онлайн-платформа повинна зберігати всі відео, які додають на неї користувачі, в окремому місці, поза межами БД. І найкращим рішенням для цього є хмарне сховище.

Хмарні сервіси – програми, які працюють на віддалених серверах. Користувач зазвичай не знає, де саме розташований сервер, на якому зберігаються його дані, і, по суті, йому це не потрібно. Найважливіше для нього – мати змогу зберігати дані безпечно і не турбуватися про фізичне обладнання, на якому вони зберігаються.

Одна з головних переваг використовувати хмарні рішення – це економічно вигідніше. Це дешевше, ніж встановлювати власне обладнання для хостингу. До того ж такі сервіси часто пропонують модель оплати «по факту використання», тобто користувач платить лише за реально використану кількість сховища або за здійснений трафік. Власне обладнання не дає такої гнучкості, бо має фіксований розмір – частина якого в деякі періоди може взагалі не використовуватись, або, навпаки, розмір буде недостатній.

Хмарні сервіси подбали про те, щоб їх було зручно інтегрувати в системи. Модель взаємодії проста: щоб завантажити відео на хмарний сервіс, платформа формує HTTP-запит з файлом і надсилає його до API (Application Programming Interface) сервісу. У відповідь API повертає дані про збережений файл. Серед них буде публічне URL, яке обов'язково необхідно зберегти в БД. Доступ до цього файлу відбувається по наданому посиланню.

					ІАЛЦ.467200.003 ПЗ	Арк.
						15
Зм.	Арк.	№ докум.	Підпис	Дата		

ВИСНОВКИ ДО РОЗДІЛУ 1

У першому розділі дипломної роботи досліджено тенденції розвитку онлайн-освіти, зумовлені глобальними викликами останніх років, зокрема пандемією та повномасштабним вторгненням. Проаналізовано переваги використання онлайн-формату як для студентів, так і для викладачів, а також виявлено зростаючий попит на асинхронне навчання.

Вивчено функціональні можливості популярних платформ, таких як Udemy та Coursera, і визначено їхні обмеження у контексті потреб та можливостей індивідуальних викладачів – зокрема, високу конкуренцію, сувору модерацию та вимоги до співпраці з навчальними закладами. На основі цього проаналізовано доцільність створення платформи для розміщення і проходження курсів, яка надає можливості для публікації відеоуроків, тестування та забезпечення доступу до курсів через систему оплат.

Визначено ключові потреби цільової аудиторії. Обґрунтовано необхідність використання хмарних сервісів для зберігання відеоконтенту, враховуючи обмеження звичайних баз даних у роботі з великими файлами.

Сформовано основні завдання для роботи в наступних розділах:

- обґрунтування архітектури системи;
- вибір технологічного стека;
- реалізація функціоналу реєстрації, створення курсів, додавання відеоуроків і тестів у серверній частині проєкту;
- інтеграція з хмарним сховищем і системою для тестових оплат;
- створення інтерфейсу, зручного для викладачів і студентів.

РОЗДІЛ 2 ІНСТРУМЕНТИ ТА ТЕХНОЛОГІЇ ДЛЯ РОЗРОБКИ ПЛАТФОРМИ

2.1 Архітектура навчальної платформи

Серед сучасних підходів до побудови вебсистем найбільш логічним і технічно доцільним для даного проєкту є клієнт-серверна архітектура. Вона дозволяє розподілити функції між двома сторонами: клієнтом, який ініціює запити, та сервером, що обробляє ці запити й повертає результат. Описана взаємодія зображена на рис. 2.1.

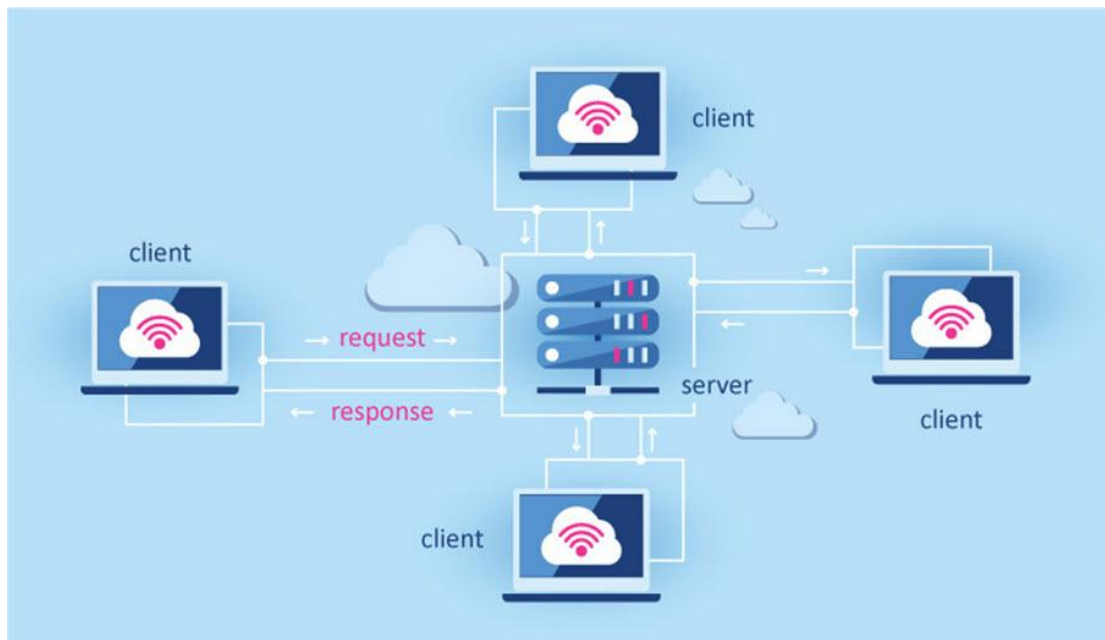


Рисунок 2.1 – Взаємодія між клієнтами і сервером [8]

На стороні клієнта зазвичай працює вебзастосунок, що надає користувачеві інтерфейс для взаємодії, формує запити (наприклад, переглянути всі курси, створити урок) та обробляє відповіді сервера. Сервер, у свою чергу, виконує більш ресурсомісткі операції, такі як обробку й збереження інформації, реалізацію бізнес-логіки, керування доступом, авторизацію, взаємодію з базою даних і сторонніми сервісами [8].

Таким чином, з огляду на вимоги до навчальної платформи – авторизація, управління курсами, відео, тестами та оплатами – обрана архітектура є зрозумілою, перевіреною та найбільш придатною для реалізації даного проєкту.

У наступних підрозділах розглянуто загальну схему взаємодії між клієнтом і сервером, а також внутрішню організацію серверної частини системи.

2.1.1 Загальна клієнт-серверна архітектура

На клієнтському боці працює вебінтерфейс. Він відповідає за відображення контенту, перемикання між сторінками, збір введених даних і формування HTTP-запитів до серверного API. Після отримання відповіді від сервера інтерфейс оновлюється або показує повідомлення користувачу – залежно від того, яка дія передбачена після запиту.

Серверна частина – це API, який приймає запити, перевіряє права доступу (авторизацію), виконує бізнес-логіку, звертається до бази та формує відповідь. Всі запити та відповіді передаються у форматі JSON, що є загальноприйнятим стандартом у веброботі.

Взаємодія між клієнтом і сервером відбувається через протокол HTTP, що забезпечує передачу даних у захищеному вигляді. Взаємодія організована згідно з принципами REST – архітектурного стилю, який передбачає використання стандартних методів (GET, POST, PUT, DELETE) для роботи з ресурсами.

Таким чином, клієнт зосереджений на презентації та взаємодії з користувачем, а сервер – на обробці логіки, керуванні даними та захисті доступу. Така структурна роздільність сприяє зрозумілій організації коду, повторному використанню компонентів та легшій адаптивності до змін.

					ІАЛЦ.467200.003 ПЗ	Арк.
						18
Зм.	Арк.	№ докум.	Підпис	Дата		

2.1.2 Основні програмні компоненти

Серверна частина платформи структурована відповідно до принципу розділення відповідальностей. Це передбачає поділ на окремі компоненти: контролери, сервіси, репозиторії. Такий підхід забезпечує зрозумілу архітектуру, спрощує підтримку коду, а також полегшує масштабованість проєкту.

У рамках цієї роботи основну роль відіграють контролери та репозиторії, оскільки більшість реалізованого функціоналу базується на стандартних CRUD-операціях (створення, читання, оновлення та видалення). Контролери приймають HTTP-запити від клієнта, викликають відповідні методи доступу до даних і формують відповіді. Репозиторії, в свою чергу, інкапсулюють взаємодію з базою даних, що дозволяє ізолювати логіку збереження даних.

Для складніших завдань, що виходять за межі стандартних CRUD-операцій, буде доцільно винести логіку в сервіси. Зокрема, в такі:

- сервіс для генерації та перевірки JWT-токенів для авторизації користувачів;
- сервіс, що забезпечує інтеграцію з хмарним сховищем для завантаження й доступу до відеоматеріалів;
- сервіс, що працює з тестовими платіжними операціями.

Такий поділ дозволяє розділити логіку на менші частини відповідно до відповідальності кожного модуля проєкту. Це покращує розуміння коду, особливо для інших розробників, які вже знайомі з такою архітектурою.

2.2 Вибір технологій та інструментів розробки

На цьому етапі необхідно правильно підібрати стек технологій. Тому що від обраних інструментів буде залежати швидкість впровадження нових функцій і зручність підтримки для розробників.

					ІАЛЦ.467200.003 ПЗ	Арк.
						19
Зм.	Арк.	№ докум.	Підпис	Дата		

У процесі вибору перевага надавалася сучасним, стабільним та активно підтримуваним технологіям. Додатково враховувався власний досвід роботи з відповідними мовами програмування і фреймворками, що дозволить значно скоротити час на розробку.

Розробка платформи ведеться за архітектурною моделлю SPA (Single Page Application), де клієнтська частина є окремим вебзастосунком, що взаємодіє з серверною частиною через REST API.

2.2.1 Фреймворки та бібліотеки для фронтенду

При виборі фреймворку для реалізації клієнтської частини навчальної платформи було розглянуто кілька сучасних рішень: React, Angular та Vue.js. Кожен із них широко використовується в індустрії та має активну спільноту.

Фреймворк *Angular* пропонує все необхідне для масштабних застосунків, проте він має доволі високу складність входження, значну кількість абстракцій і вимагає детального дотримання суворої структури [9].

Vue – легший у вивченні та гнучкий варіант, однак його екосистема досі вважається менш стабільною та менш зрілою порівняно з конкурентами, особливо у великих проєктах [10].

React – це гнучка бібліотека, яка побудована навколо ідеї простих, багаторазово використовуваних функціональних компонентів та ефективно працює шляхом використання віртуального DOM (Document Object Model) [9]. React не нав'язує жорсткої структури, а синтаксис JSX (JavaScript XML) робить створення шаблонів максимально наближеним до HTML (HyperText Markup Language), що полегшує розробку.

Зважаючи на гнучкість, зрозумілу архітектуру компонентів, велику кількість готових рішень, а також особистий досвід роботи, що є не менш важливим аргументом, для проєкту обрано бібліотеку React.

					ІАЛЦ.467200.003 ПЗ	Арк.
						20
Зм.	Арк.	№ докум.	Підпис	Дата		

Для забезпечення статичної типізації доцільним також виглядає використання TypeScript як надбудови над JavaScript, що підвищить стабільність і передбачуваність коду. Для реалізації маршрутизації між сторінками в межах SPA доцільно застосувати React Router, а для централізованого управління станом – React Redux.

Для побудови інтерфейсу можна також застосувати Material UI, що пропонує широкий вибір готових компонентів, що дозволяють швидше створити сучасний, приємний на вигляд дизайн [11].

Для роботи з API рекомендується використовувати бібліотеку Axios, яка надає зручний синтаксис для надсилання HTTP-запитів. Для відображення системних повідомлень (помилки, підтверджень тощо) варто застосувати React Toastify, а налаштовувати оформлення та стилізацію сторінки за допомогою власних CSS-правил або тем Material UI.

Таким чином, комбінація React з цими бібліотеками дає змогу побудувати інтерфейс, який цілком придатний для реалізації функціональності навчальної платформи даного дипломного проєкту.

2.2.2 Стек технологій для бекенду

Серед найпопулярніших варіантів для реалізації серверної частини – Node.js, Spring Boot (Java) та ASP.NET Core (.NET). Кожен з цих фреймворків добре зарекомендував себе у веброзробці, підтримує побудову REST API, має розвинену екосистему, бібліотеки для авторизації, обробки запитів і роботи з базами даних.

Node.js добре підходить для високонавантажених систем та дозволяє обробляти велику кількість одночасних підключень [12], але вимагає більше ручного налаштування, зокрема для забезпечення статичної типізації та складної бізнес-логіки.

					ІАЛЦ.467200.003 ПЗ	Арк.
						21
Зм.	Арк.	№ докум.	Підпис	Дата		

Spring Boot – це Java-фреймворк, який дозволяє розробникам створюваний складні, безпечні та водночас масштабовані системи. Однак він має складніший поріг входу та вимагає вищого рівня досвіду, потребує більше налаштувань [13] і може створити надлишкову складність для невеликого проєкту.

ASP.NET Core – універсальний фреймворк для розробки програмного забезпечення, що створений компанією Microsoft. Він надає широкий вибір додаткових бібліотек і фреймворків, які можуть значно зменшити витрачений час на розробку [12]. Ще однією перевагою ASP.NET є наявність вбудованих можливостей контролю доступу на основі ролей.

З огляду на ці чинники, а також враховуючи особистий досвід роботи з платформою .NET, було вирішено обрати ASP.NET Core як основний фреймворк для побудови бекенду. Такий вибір дозволить значно пришвидшити процес розробки, уникнути типових помилок і ефективно використати вже наявні знання.

Передбачається також використання таких компонентів:

- ASP.NET Core Web API – для побудови REST API з підтримкою маршрутизації, авторизації та обробки запитів;
- Entity Framework Core – як ORM (Object-Relational Mapping) для зручної роботи з реляційною базою даних без написання SQL-запитів;
- AutoMapper – для автоматичного мапінгу між DTO (Data Transfer Object) та моделями в БД, що дозволяє спростити перетворення даних між різними шарами програми.

Такий стек широко застосовується у комерційних проєктах. Його використання дозволить спростити і пришвидшити реалізацію окремих частин платформи завдяки готовим рішенням, а також дасть змогу створити структурований і масштабований сервер.

					ІАЛЦ.467200.003 ПЗ	Арк.
						22
Зм.	Арк.	№ докум.	Підпис	Дата		

2.2.3 Вибір бази даних

Для реалізації серверної частини платформи було розглянуто кілька популярних реляційних СУБД: Microsoft SQL Server, SQLite та PostgreSQL [14]. Кожна з них має свої переваги та обмеження, що впливають на доцільність використання в певних проєктах.

SQLite – це вбудована СУБД, яка не вимагає встановлення чи окремого серверного процесу. Вона ідеальна для легких локальних застосунків, проте не підходить для багатокористувацьких вебплатформ.

PostgreSQL – це система управління базами даних, особливістю, якої є підтримка роботи в контейнерах Docker, що робить її зручною для інженерних практик CI/CD (Continuous Integration / Continuous Deployment), автоматизованого розгортання та тестування. Також варто зазначити, що PostgreSQL є найпопулярнішою СУБД серед розробників за результатами Stack Overflow Developer Survey 2024, а саме 49% розробників її використовують [15].

Microsoft SQL Server теж є потужною СУБД і має широкий функціонал, але вирізняється глибокою інтеграцією в екосистему Microsoft. Вона підходить для великих корпоративних проєктів, але має високі ліцензійні витрати та складнішу підтримку.

У результаті порівняння було обрано PostgreSQL. Важливою перевагою є можливість запуску PostgreSQL у Docker-контейнері, що спрощує налаштування, тестування і значно полегшує подальший деплой платформи у хмару.

2.2.4 Хмарне сховище для збереження відео

Навчальна платформа для онлайн-курсів передбачає, що автори курсів повинні мати змогу завантажувати відео до уроків, а користувачі – їх

					ІАЛЦ.467200.003 ПЗ	Арк.
						23
Зм.	Арк.	№ докум.	Підпис	Дата		

переглядати. Вибране рішення має бути простим в підключенні, і головне – мати безплатну версію, з якою можна зберігати контент на етапі розробки без додаткових фінансових витрат.

Для реалізації цих вимог обрано хмарний сервіс Cloudinary. Це платформа для зберігання та транслювання медіафайлів, в тому числі відео, яка надає API і готові SDK для .NET [16], тож її буде легко інтегрувати в систему.

Cloudinary також надає безплатний тарифний план, який повністю покриває потреби для розробки. В його межах доступні такі функції: завантаження відео через API, сховище відеофайлів з підтримкою резервного копіювання, передача відео з урахуванням якості з’єднання користувача тощо.

На рис. 2.2 зображено сторінку Cloudinary з переліком планів і можливостей, що до них входять.

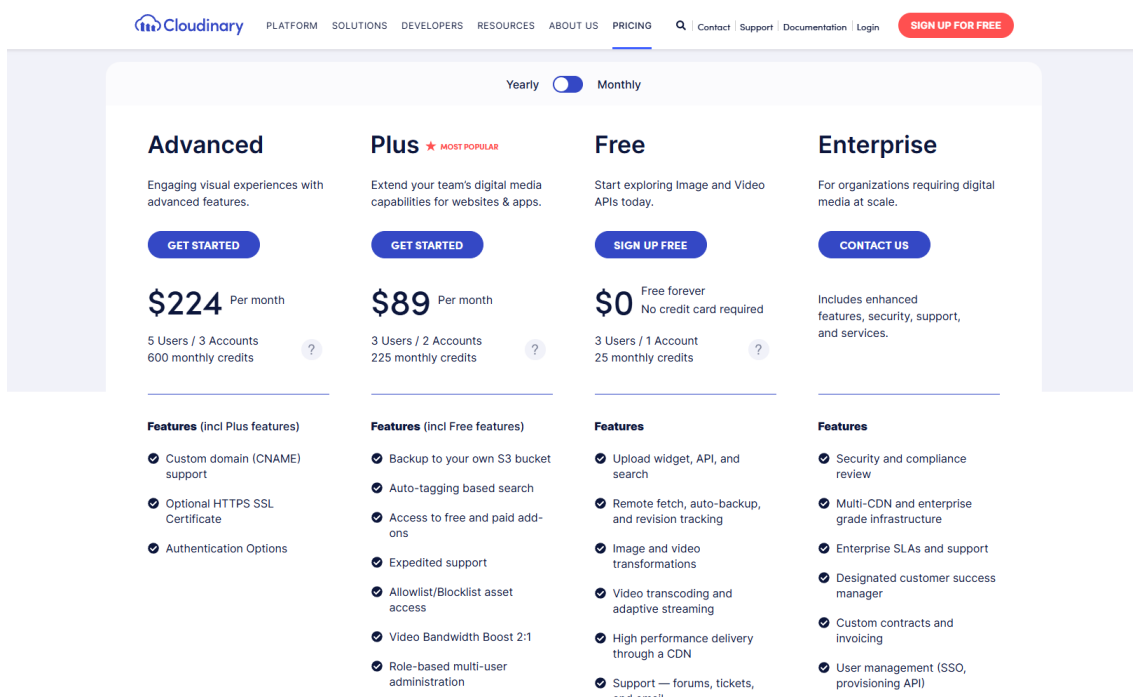


Рисунок 2.2 – Сторінка Cloudinary з переліком тарифів

Щомісячні ліміти у безплатному плані наступні:

- 25 000 трансформацій відео і зображень;
- 25 ГБ сховище для збереження файлів;

- 25 ГБ вхідного трафіку на перегляд відео.

Якщо раціонально використовувати ресурси, тоді цього обсягу буде цілком достатньо для створення навчальної платформи одним розробником.

2.2.5 Платіжна система для обробки оплат

Відповідно до вимог міжнародного стандарту PCI DSS (Payment Card Industry Data Security Standard) системі строго заборонено обробляти чи зберігати карткові дані користувачів без отримання відповідної сертифікації [18]. В більшості випадків, особливо для невеликих платформ, це економічно і технічно недоцільно. Саме тому для монетизації навчальної платформи необхідно інтегрувати платіжну систему, яка б надавала таке надійне і безпечне рішення оплати доступу до курсів.

При виборі рішення ключовими критеріями є: простота інтеграції з .NET, наявність тестового середовища, а також відсутність обов'язкових витрат на етапі розробки.

Усі ці вимоги задовольняє Stripe – платіжна платформа, що підтримує обробку транзакцій у понад 40 країнах, дозволяє приймати оплату банківськими картками, не вимагаючи складного банківського підключення [17]. Крім того, Stripe має зручну документацію, інтерфейс для легкого перегляду всіх транзакцій та офіційний .NET SDK, що спрощує інтеграцію з сервером платформи.

2.3 Механізм авторизації користувачів

У межах реалізації навчальної платформи використано авторизацію на основі токена JWT. Користувачі зможуть реєструватися або входити в систему через спеціальні методи API.

					ІАЛЦ.467200.003 ПЗ	Арк.
						25
Зм.	Арк.	№ докум.	Підпис	Дата		

Для зберігання облікових записів передбачається використання механізмів ASP.NET Core Identity, зокрема UserManager для управління користувачами та RoleManager для прив'язки ролей [19]. У процесі входу система перевірятиме правильність введених облікових даних, після чого виконується генерація JWT-токена, який містить основні claims: ім'я користувача та його ролі.

Для генерації токена планується реалізувати окремий сервіс – TokenService. Цей компонент на основі налаштувань із конфігураційного файлу формуватиме підписаний токен, який клієнт зможе використовувати для наступних запитів. Токен буде дійсний протягом обмеженого періоду (наприклад, 7 днів). Після завершення терміну дії користувачу потрібно буде повторно увійти в акаунт.

Також передбачається використання ролей, таких як Admin, Teacher і Member. Ролі будуть присвоюватись автоматично під час реєстрації залежно від того, чи користувач є викладачем чи студентом.

Після авторизації клієнтська частина додатка зберігає отриманий JWT-токен і додає його до кожного запиту в заголовок. Це дозволяє серверу швидко перевіряти запити без повторної взаємодії з базою даних. Доступ до захищених маршрутів реалізується через стандартний атрибут Authorize, що автоматично перевіряє дійсність і вміст токена.

Такий підхід дає змогу створити авторизацію, що легко підтримувати та яка не залежить від сесій.

ВИСНОВКИ ДО РОЗДІЛУ 2

У другому розділі дипломної роботи визначено архітектурний підхід до створення навчальної платформи. Обґрунтовано доцільність застосування клієнт-серверної архітектури, яка є надійним рішенням для веборієнтованих систем.

Проектування серверної частини виконано з урахуванням принципу розділення відповідальностей. Визначено, що обробка запитів має бути покладена на контролери, робота з базою даних – на репозиторії, а складну бізнес-логіку запропоновано винести у відповідні сервіси. Це дозволяє підвищити гнучкість системи та її придатність до масштабування.

На основі порівняльного аналізу обґрунтовано вибір технологій для фронтенду: React, TypeScript, Redux та Material UI. Такий стек забезпечує типізовану розробку, ефективне керування станом додатка та швидке створення зручного користувацького інтерфейсу. Для серверної частини обрано ASP.NET Core, EF Core, AutoMapper. Як систему керування базами даних обрано PostgreSQL через її стабільність та підтримку контейнеризації у середовищі Docker. Для інтеграцій із зовнішніми сервісами обрано Stripe для обробки оплат та Cloudinary для зберігання відео.

У результаті сформовано повний технологічний стек для реалізації навчальної платформи, який відповідає сучасним інженерним вимогам та забезпечує основу для ефективної реалізації функціоналу. У наступному розділі зосереджено увагу на реалізації основних компонентів системи відповідно до визначеної архітектури та обраних технологій.

					ІАЛЦ.467200.003 ПЗ	Арк.
						27
Зм.	Арк.	№ докум.	Підпис	Дата		

РОЗДІЛ 3 РОЗРОБКА КЛЮЧОВИХ ФУНКЦІЙ ПЛАТФОРМИ

У цьому розділі описано як розроблені основні частини навчальної платформи для онлайн-курсів.

3.1 Побудова серверної частини платформи з використанням .NET та EF Core

Серверну частину реалізовано на основі багаторівневої архітектури з розділенням на контролери, сервіси й репозиторії. Контролери приймають HTTP-запити, виконують базову валідацію вхідних даних і делегують подальшу обробку сервісам і репозиторіям. Щоб зменшити кількість абстрацій, сервіси створено не для кожної сутності, а використано лише там, де необхідно реалізувати складнішу бізнес-логіку.

Оскільки в проєкті використовується EF Core, застосунок має контекст бази даних CourseContext, який наслідує клас IdentityDbContext. Таблиці створюються на основі .NET-класів, а зміни в структурі БД вносяться за допомогою міграцій.

Щоб чітко визначити, які саме дані мають надходити від клієнта і повертатися у відповідь, у проєкті реалізовано підхід з використанням DTO. Це спеціальні класи, які містять тільки необхідні поля для кожної операції, і не дублюють повністю структуру таблиць бази даних.

Перетворення класів сутностей у DTO і навпаки – з DTO у відповідні таблиці БД – реалізовано за допомогою бібліотеки AutoMapper. Для цього створено клас AutoMapperProfile, що наслідує базовий клас Profile. Починаючи з версії .NET 6.0, конфігурацію перетворень необхідно оголошувати в

					ІАЛЦ.467200.003 ПЗ	Арк.
						28
Зм.	Арк.	№ докум.	Підпис	Дата		

конструкторі класу [20], тому саме там визначені налаштування для перетворення окремих сутностей у DTO та навпаки.

3.2 Реалізація системи управління акаунтами та обмеження доступу

Для збереження інформації про користувачів у БД створено клас User, який наслідує клас IdentityUser із бібліотеки ASP.NET Core Identity. Цей клас надає стандартний набір полів, необхідних для базового управління акаунтами.

3.2.1 Права доступу через ролі та клейми

У проєкті визначено 3 ролі:

- Member – це звичайний користувач, який проходить курси;
- Teacher – користувач, який при реєстрації вказав, що є викладачем. Він має всі права Member, а також доступ до функціоналу створення власних курсів і керування ними;
- Admin – адміністратор платформи, що має повний доступ до всіх курсів.

Однак обмеження доступу реалізовано не напряму на основі ролей, а через систему клеймів. Кожній ролі призначено відповідні клейми типу «Permission», які описують дозволені дії. Наприклад, роль Teacher має клейми BasicAccess і ManageCourses, що дають змогу створювати і редагувати власні курси. У роль Admin включено ті самі клейми, що й в Teacher, а також додатковий клейм AdminAccess.

Такий підхід дає змогу не залежати від визначеного переліку ролей. Замість того, щоб вказувати весь перелік ролей, що мають доступ до певного

					ІАЛЦ.467200.003 ПЗ	Арк.
						29
Зм.	Арк.	№ докум.	Підпис	Дата		

функціоналу (наприклад, редагування курсів), достатньо перевірити наявність відповідного клейма, такого як ManageCourses.

3.2.2 Реєстрація, автентифікація та генерація токенів на сервері

В API реалізовано POST-методи account/login та account/register, які приймають відповідно об'єкти LoginDto і RegisterDto:

- LoginDto містить поля Username і Password;
- RegisterDto містить ті самі поля, що й LoginDto, а також поле IsTeacher. Якщо при реєстрації користувач вказує, що є він викладачем, йому присвоюється роль Teacher, інакше – Member, та клейми, що передбачені для відповідної ролі.

На рис. 3.1 зображено реалізацію POST-методу для входу в акаунт.

```
[HttpPost("login")]
0 references
public async Task<ActionResult<UserDto>> Login(LoginDto loginDto)
{
    var user = await _userManager.FindByNameAsync(loginDto.Username);
    if (user == null) return Unauthorized();

    if (await _userManager.IsLockedOutAsync(user)) return AccountIsLocked();

    if (!await _userManager.CheckPasswordAsync(user, loginDto.Password))
    {
        await _userManager.AccessFailedAsync(user);
        return Unauthorized();
    }

    await _userManager.ResetAccessFailedCountAsync(user);
    var roles = await _userManager.GetRolesAsync(user);
    var userClaims = await _userManager.GetClaimsAsync(user);

    return new UserDto
    {
        Id = user.Id,
        Username = user.UserName,
        Token = await _tokenService.GenerateToken(user),
        Roles = roles.ToList(),
        Claims = userClaims.Select(c => $"{c.Type}: {c.Value}").ToList()
    };
}
```

Рисунок 3.1 – Реалізація методу API для входу в акаунт

					ІАЛЦ.467200.003 ПЗ	Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		30

Як зображено на рисунку 3.1, під час входу в акаунт API шукає користувача за наданим іменем за допомогою методу FindByNameAsync з UserManager і перевіряє пароль через CheckPasswordAsync. Якщо користувач автентифікований успішно, у відповідь повертається об'єкт UserDto, який містить JWT-токен для авторизації наступних запитів та іншу інформацію про користувача. В разі невдачі API повертає помилку з кодом 401 Unauthorized.

Генерацію токена винесено в окремий сервіс TokenService, в якому реалізовано метод GenerateToken. В процесі створення токена формується список клеймів з ім'ям, ролями і дозволами доступу користувача. Токен підписується за допомогою секретного ключа, що гарантує захист від підробки. Термін дії токена після видачі встановлюється на 7 днів.

Також реалізовано захист від перебору паролів. При кожному неправильному введенні пароля система інкрементує значення поля AccessFailedCount. Якщо кількість невдалих спроб ввести пароль досягає п'яти, акаунт блокується на 15 хвилин. Наступні спроби входу протягом цього часу будуть відхилені. Якщо ж користувач вводить пароль правильно до досягнення ліміту, лічильник AccessFailedCount скидається до нуля.

3.2.3 Збереження токена та авторизація на клієнтській частині

На клієнтській частині реалізовано повноцінну авторизацію користувача. Після успішної автентифікації сервер повертає JWT-токен разом з інформацією про користувача (унікальний ідентифікатор, ім'я, ролі та клейми). Ці дані записуються в локальне сховище браузера, щоб зберегти стан авторизації навіть після перезавантаження сторінки.

Збережений токен автоматично додається в заголовок Authorization кожного наступного запиту до API. Таким чином, система може ідентифікувати користувача й надавати відповіді відповідно до його прав доступу. На рис. 3.2 показано вміст локального сховища, де під ключем «user»

					ІАЛЦ.467200.003 ПЗ	Арк.
						31
Зм.	Арк.	№ докум.	Підпис	Дата		

в колонці «Value» видно збережену інформацію про користувача: його унікальний ідентифікатор, ім'я, ролі, клейми і JWT-токен.

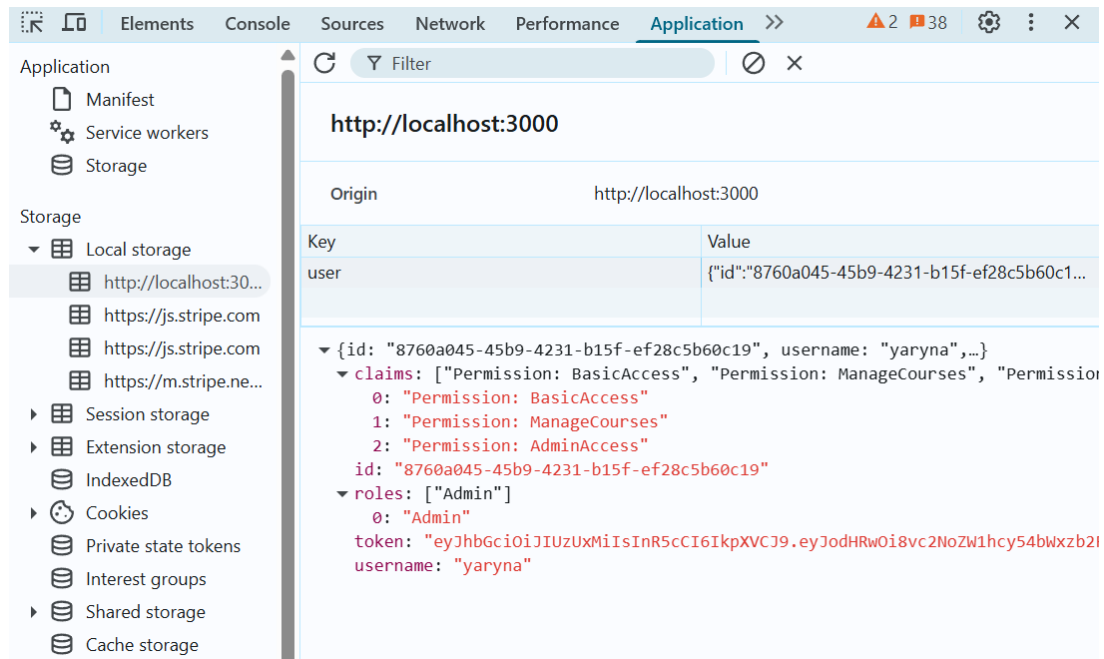


Рисунок 3.2 – Дані користувача в локальному сховищі

Інтерфейс для реєстрації і входу в акаунт реалізовано як окремі компоненти Register і Login. Обидві форми містять поля для введення імені користувача (username) та пароля (password). Для керування станом форми використано бібліотеку react-hook-form, яка дає змогу перевіряти введені дані. Поля перевіряються на заповнення перед надсиланням запиту, і якщо користувач залишає обов'язкові поля пустими, буде показане повідомлення з описом помилки.

Надсилання облікових даних відбувається через виклик асинхронної дії signInUser, яка реалізована за допомогою Redux Toolkit. Ця дія надсилає запит до API, отримує JWT-токен та іншу інформацію про користувача, після чого зберігає їх у локальному сховищі. Крім цього, під час авторизації виконується очищення попереднього стану (курси, тести), і користувача перенаправляють на сторінку курсів. На рис. 3.3 показано реалізацію Redux-дії signInUser, де видно, як відбувається звернення до API, обробка відповіді та збереження даних у локальне сховище.

```

export const signInUser = createAsyncThunk<User, FieldValues>(
  'account/signInUser',
  async (data, thunkAPI) => {
    try {
      const user = await agent.Account.login(data)
      localStorage.setItem('user', JSON.stringify(user))
      return user
    } catch (error: any) {
      return thunkAPI.rejectWithValue({ error: error.data })
    }
  }
)

```

Рисунок 3.3 – Реалізація дії signInUser за допомогою Redux Toolkit

Такий підхід дає змогу користувачу авторизуватись в системі і автоматично зберегти отриманий токен доступу для наступних запитів.

3.3 Побудова структури курсів: розділи, уроки, тести

Розроблено і реалізовано структуру курсів. Створено наступну ієрархію класів:

- Course – зберігає загальну інформацію про курс, таку як назва, опис, вартість, стан курсу, автор;
- Section – представляє розділи курсу, містить назву і опис, належить до одного курсу;
- Lesson – уроки, які містяться в певній секції. Урок може містити теоретичний і практичний відеоматеріали;
- Test – клас, що представляє питання, прив'язані до уроку;
- Option – зберігає варіант відповіді до тесту і чи є цей варіант правильним, належить до одного запитання.

На рис. 3.4 зображено згенеровану схему бази даних на основі .NET-класів.

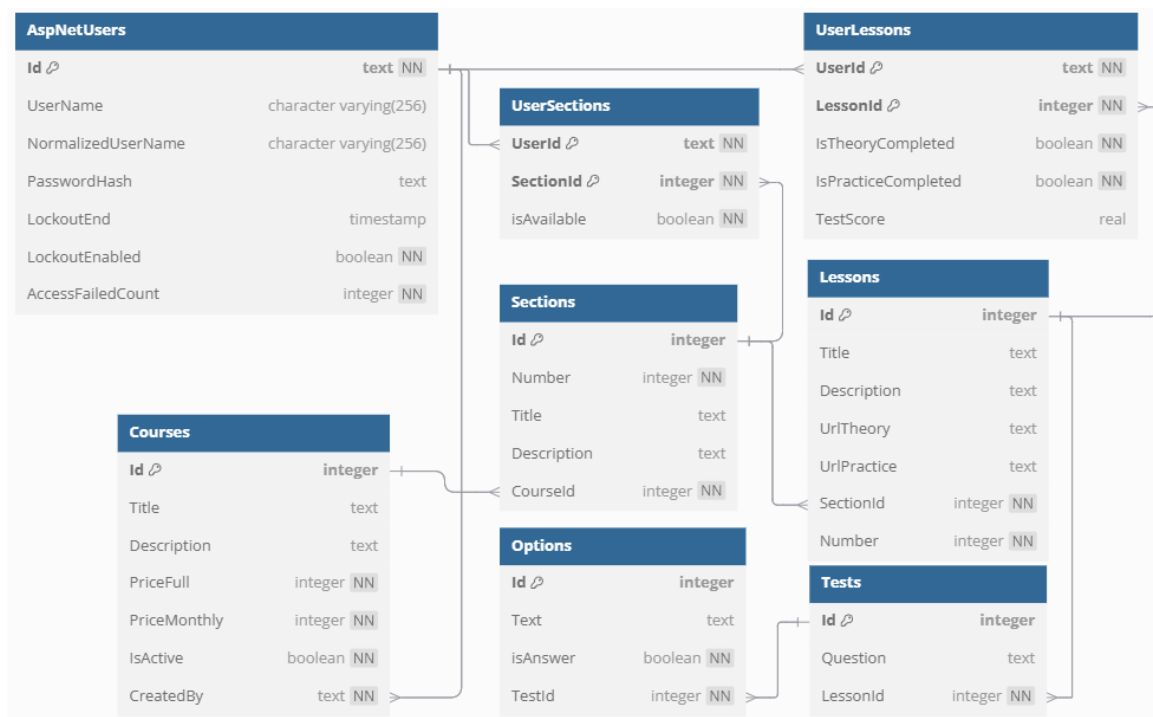


Рисунок 3.4 – Структура курсу в базі даних

Режим доступу до структури курсу реалізовано через контролери з відповідними обмеженнями. Для захисту методів API використано атрибут `Authorize`, де можна вказати, які користувачі з якими ролями та клеймами мають мати право виконувати запит. Якщо атрибут використано без параметрів, запит може виконати будь-який авторизований користувач. Обмеження доступу додано до тих маршрутів, які змінюють дані курсів: POST, PUT, DELETE.

Зокрема, доступ до керування курсами обмежено клеймом `ManageCourses`, який надається викладачам і адміністраторам. На рис. 3.5 зображено реалізацію POST-методу для створення курсу з відповідним обмеженням. Запити типу GET, які використовуються для перегляду курсу, працюють по-іншому, бо користувачі повинні мати змогу побачити базову інформацію про курс перед тим, як авторизуватись чи придбати доступ. Якщо користувач вже придбав курс – йому повертається повна інформація, включно з уроками та посиланнями на відеоматеріали; якщо ні – лише базові відомості, до яких не входить конфіденційний контент.

```

[Authorize(Policy = "ManageCourses")]
[HttpPost]
0 references
public async Task<ActionResult<GetCourseDto>> AddCourse(AddCourseDto newCourse)
{
    var username = User.Identity.Name ?? "";

    if (newCourse.PriceFull <= 0 || newCourse.PriceMonthly <= 0)
        return BadRequest("Full price and section price must be greater than zero.");

    var result = await _coursesRepository.AddCourse(newCourse, username);

    if (!result.IsSuccess)
    {
        return BadRequest(result.ErrorMessage);
    }

    return result.Data;
}

```

Рисунок 3.5 – Реалізація методу з обмеженням відповідно до клейму

Таким чином, побудовано структуру курсів, а захист даних від перегляду та редагування реалізовано на серверній частині на основі ролей, що є надійнішим підходом, ніж обмеження на клієнтському рівні.

3.4 Механізм збереження та відображення прогресу навчання

Для реалізації збереження прогресу вивчення уроків розроблено відповідну структуру даних. Також реалізовано необхідні механізми для автоматичного запису та отримання даних про прогрес користувача.

3.4.1 Структура таблиці збереження прогресу

Один користувач може проходити багато уроків, і один урок може бути пов'язаний з багатьма користувачами. Тому для збереження стану проходження уроку створено таблицю-зв'язок UserLesson.

Стан проходження уроку визначається за трьома параметрами: чи користувач переглянув відеотеорію, відеопрактику і чи пройшов тест. На основі цього визначено перелік колонок, які входять до таблиці UserLesson.

На рис. 3.6 зображено оголошення .NET-класу UserLesson.

```

12 references
public class UserLesson
{
    11 references
    public string UserId { get; set; }
    1 reference
    public User User { get; set; }
    5 references
    public int LessonId { get; set; }
    2 references
    public Lesson Lesson { get; set; }
    5 references
    public bool IsTheoryCompleted { get; set; } = false;
    5 references
    public bool IsPracticeCompleted { get; set; } = false;
    12 references
    public float? TestScore { get; set; }
}

```

Рисунок 3.6 – Клас UserLesson для збереження прогресу користувача

У таблиці UserLesson визначено такі поля:

- UserId – ідентифікатор користувача (типу string);
- LessonId – ідентифікатор уроку (типу int);
- IsTheoryCompleted та IsPracticeCompleted – бінарні поля, які відповідають за відмітку, чи користувач переглянув відеотеорію та відеопрактику відповідно;
- TestScore – необов’язкове поле типу float для збереження результату проходження тесту: якщо тест не пройдено – значення null, якщо пройдено – зберігається результат у відсотках (від 0 до 100).

3.4.2 Включення прогресу користувача у відповідь API

Під час отримання уроку (перетворення об’єкта Lesson у DTO GetLessonDto) налаштовано автоматичну перевірку: чи користувач уже

					ІАЛЦ.467200.003 ПЗ	Арк.
						36
Зм.	Арк.	№ докум.	Підпис	Дата		

розпочинав цей урок і чи існує відповідний запис у таблиці UserLesson. Це реалізовано за допомогою налаштування в профілі AutoMapper. Після перетворення основних полів з Lesson у клас GetLessonDto система додатково шукає ще запис у UserLesson для поточного користувача і автоматично додає інформацію про прогрес.

На рис. 3.7 зображено налаштування перетворення з Lesson до GetLessonDto.

```
public class AutoMapperProfile : Profile
{
    0 references
    public AutoMapperProfile()
    {
        CreateMap<Lesson, GetLessonDto>()
            .AfterMap((src, dest, context) =>
            {
                var userId = context.Items["UserId"] as string;
                var userLesson = src.UserLessons.FirstOrDefault(ul => ul.UserId == userId);
                if (userLesson != null)
                {
                    dest.IsTheoryCompleted = userLesson.IsTheoryCompleted;
                    dest.IsPracticeCompleted = userLesson.IsPracticeCompleted;
                    dest.TestScore = userLesson.TestScore;
                }
            });
    }
}
```

Рисунок 3.7 – Перетворення Lesson у GetLessonDto з додаванням інформації про прогрес користувача

Такий підхід дає змогу одразу додати у відповідь як загальну інформацію про урок, так і прогрес користувача. Це зменшує кількість необхідних запитів на сервер і спрощує обробку даних на клієнті.

3.4.3 Візуалізація стану проходження уроків на клієнтській частині

Після завантаження курсу клієнтська частина отримує повну інформацію про уроки, включно зі станом їх проходження. Цей стан візуально відображається в інтерфейсі. Визначено чотири можливі стани уроку:

- «недоступний» – користувач не придбав курс, але може бачити назви уроків, що до нього входять;

					ІАЛЦ.467200.003 ПЗ	Арк.
						37
Зм.	Арк.	№ докум.	Підпис	Дата		

- «нерозпочатий» – урок доступний, але користувач ще не переглядав відео і не проходив тести;
- «в процесі» – користувач виконав хоча б одну з частин уроку (теорію, практику чи тест);
- «виконаний» – користувач переглянув обидва відео і склав тест.

Для кожного стану підібрано відповідну іконку та колір, що візуально підсвічують прогрес. Іконки отримано з бібліотеки «@mui/icons-material», яку надає Material UI [21]. На рис. 3.8 зображено вигляд підібраних іконок.



Рисунок 3.8 – Іконки станів уроків із бібліотеки Material UI

Використано такі іконки:

- LockOutlined – іконка замка сірого кольору, що позначає недоступність;
- CalculateOutlined – нейтральна іконка темно-сірого кольору, що відображає нерозпочатий стан;
- AccessTime – іконка годинника оранжевого кольору для стану «в процесі»;
- DoneOutline – іконка галочки зеленого кольору, що позначає повне завершення уроку.

3.4.4 Автоматичне оновлення прогресу

Для оновлення стану проходження передбачено маршрут PUT userlesson/lessonid, до якого надсилається оновлена інформація, коли користувач перегляне одне з відеоматеріалів або виконає тести.

Крім цього, біля відеоматеріалів відображаються прапорці, які вказують, чи користувач переглянув відповідний фрагмент. Для зручності користувача реалізовано автоматичне оновлення статусу: якщо до кінця відео залишається менше десяти секунд, система автоматично надсилає запит на сервер, щоб позначити відео як переглянуте. Ця логіка реалізована у функції `handleTimeUpdate`, показаній на рис. 3.9.

```
const handleTimeUpdate = (event: React.SyntheticEvent<HTMLVideoElement>) => {
  const video = event.currentTarget;
  const timeRemaining = video.duration - video.currentTime;

  if (!isCompleted && timeRemaining <= 10) {
    const body = videoNumber === 0
      ? { isTheoryCompleted: 1, courseId }
      : { isPracticeCompleted: 1, courseId };
    dispatch(updateLessonAsync({ id: parseInt(lessonId!), body }));
  }
};
```

Рисунок 3.9 – Функція для оновлення статусу перегляду відео

Аналогічна логіка застосована і до проходження тестів: щойно користувач завершує тест, отриманий результат автоматично надсилається на сервер і зберігається у відповідному записі.

3.5 Інтеграція з хмарним сховищем Cloudinary

Для зберігання відеоконтенту на платформі використано хмарний сервіс Cloudinary. Ця інтеграція забезпечує зручне завантаження, зберігання та доступ до відеофайлів.

3.5.1 Підготовка середовища для інтеграції з Cloudinary

Створено обліковий запис на сайті `cloudinary.com` і згенеровано потрібні параметри: `CloudName`, `ApiKey`, `ApiSecret`, відповідно до офіційної

					ІАЛЦ.467200.003 ПЗ	Арк.
						39
Зм.	Арк.	№ докум.	Підпис	Дата		

документації [22]. Для безпечного зберігання цих даних використано механізм User Secrets, який дає змогу вказати конфіденційні дані на локальному комп'ютері і не додавати їх напряму у репозиторій. На рис. 3.10 показано виконання команд для додавання даних Cloudinary у секрети проєкту.

```
PS C:\Users\...\platform-for-courses\API> dotnet user-secrets set "Cloudinary:CloudName" "c..."
Successfully saved Cloudinary:CloudName to the secret store.
PS C:\Users\...\platform-for-courses\API> dotnet user-secrets set "Cloudinary:ApiSecret" "y..."
Successfully saved Cloudinary:ApiSecret to the secret store.
PS C:\Users\...\platform-for-courses\API> dotnet user-secrets set "Cloudinary:ApiKey" "4..."
Successfully saved Cloudinary:ApiKey to the secret store.
```

Рисунок 3.10 – Приклад виконання команд для збереження параметрів у системі секретів проєкту

Після запису секретів їх наявність перевірено командою «dotnet user-secrets list».

3.5.2 Додавання та видалення відео на сервері

Завантаження відео у хмарне сховище реалізовано на серверній частині. Маршрути для створення і оновлення уроку приймають відео як параметр типу IFormFile. Якщо файл додано, він автоматично завантажується до Cloudinary. В разі якщо відео замінюється чи урок видаляється, тоді передбачено і видалення файлу зі сховища, щоб раціонально використовувати ресурси.

Для збереження необхідної інформації в БД у таблицю Lesson додано додаткові поля: посилання на відео, за яким користувач буде переглядати відео, та ідентифікатор відео, потрібний для подальшого видалення зі сховища.

Логіку, безпосередньо пов'язану з інтеграцією з хмарним сховищем, реалізовано в окремому сервісі VideoService. Клас містить:

- приватне поле `_cloudinary` – екземпляр класу `Cloudinary` з бібліотеки `CloudinaryDotNet`;
- конструктор, який ініціалізує з'єднання на основі параметрів підключення;

- асинхронний метод AddVideoAsync, що відповідає за завантаження відео у сховище (рис. 3.11);
- асинхронний метод DeleteVideoAsync, який за ідентифікатором видаляє відео зі сховища.

```
public async Task<VideoUploadResult> AddVideoAsync(IFormFile file)
{
    var uploadResult = new VideoUploadResult();

    if (file.Length > 0)
    {
        using var stream = file.OpenReadStream();
        var uploadParams = new VideoUploadParams
        {
            File = new FileDescription(file.FileName, stream)
        };
        uploadResult = await _cloudinary.UploadAsync(uploadParams);
    }

    return uploadResult;
}
```

Рисунок 3.11 – Реалізація методу для завантаження відео до сховища

Після завершення завантаження сховище повертає всю інформацію про результат. З цих даних виділяються URL-посилання на відео і його унікальний ідентифікатор, які зберігаються у БД.

3.6 Впровадження інтеграції платіжної системи Stripe

Для обробки оплат для отримання доступів до курсів використано платіжну систему Stripe. Ця інтеграція забезпечує прийом платежів і надання доступу до відповідних матеріалів автоматично після успішної оплати.

3.6.1 Налаштування середовища для інтеграції з Stripe

Створено тестовий обліковий запис Stripe. На рис. 3.12 показано вкладку «Developers» («Для розробників») після реєстрації.

					ІАЛЦ.467200.003 ПЗ	Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		41

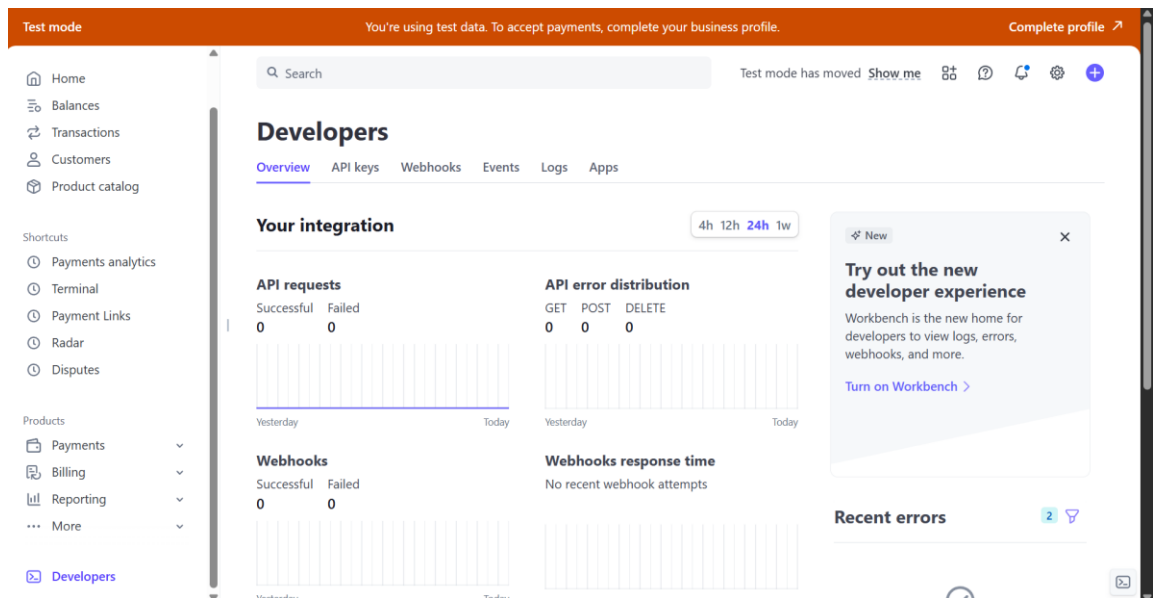


Рисунок 3.12 – Вигляд вкладки «Developers» в кабінеті Stripe

Для створеного акаунта згенеровано публічний і приватний ключі. Ці конфіденційні дані додано до User Secrets, а налаштування перевірено командою «dotnet user-secrets list» (рис. 3.13).

```
PS C:\Users\ [redacted] \platform-for-courses\API> dotnet user-secrets list
StripeSettings:WhSecret = whsec_d [redacted] 3
StripeSettings:SecretKey = sk_test_5 [redacted]
StripeSettings:PublishableKey = pk_test 5 [redacted]
```

Рисунок 3.13 – Перевірка збережених ключів для інтеграції з Stripe

Налаштування акаунту залишено в тестовому режимі, тобто всі оплати будуть також тестові.

3.6.2 Структура таблиць бази даних для обробки оплат

В рамках платформи передбачено можливість оплати не лише повного курсу, але й окремих його секцій. Це рішення прийнято для того, щоб користувач міг спробувати пройти першу секцію і перевірити, чи курс йому підходить або для того, щоб обрати лише ті частини курсу, які йому найцікавіші або найнеобхідніші. Крім того, такий підхід є корисним у випадках, коли користувач має обмежений бюджет.

					ІАЛЦ.467200.003 ПЗ	Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		42

Відповідно до цього, виникла потреба зберігати інформацію про те, до яких саме секцій користувач має доступ.

Для цього в базі даних створено таблицю-зв'язок UserSection, яка фіксує доступність кожної секції для конкретного користувача. Основним полем у цій таблиці є isAvailable – бінарне поле, яке визначає, чи секція доступна для користувача. За замовчуванням усі секції є недоступними (значення false), а після оплати секції або повного курсу відповідні записи оновлюються (значення true). Це дає змогу серверу швидко перевіряти права доступу до секцій і формувати відповідь клієнту, включаючи лише ті частини курсу, які користувач має право переглядати.

На рис. 3.14 зображено .NET-клас UserSection, який реалізує відповідну таблицю.

```

9 references
public class UserSection
{
    7 references
    public string UserId { get; set; }
    1 reference
    public User User { get; set; }
    5 references
    public int SectionId { get; set; }
    1 reference
    public Section Section { get; set; }
    5 references
    public bool isAvailable { get; set; } = false;
}

```

Рисунок 3.14 – Клас UserSection для фіксації доступу до секцій

У таблиці UserSection визначено такі поля:

- UserId – ідентифікатор користувача (типу string);
- SectionId – ідентифікатор секції (типу int);
- isAvailable – бінарне поле, що позначає наявність доступу користувача до секції.

Для зберігання самих оплат створено таблицю Payment (рис. 3.15). Вона зберігає інформацію про кожну спробу або факт оплати, незалежно від того, чи була транзакція успішною.

Ця таблиця підтримує як оплату за курс загалом, так і оплату за окрему секцію. Для цього використовується пара полів: PurchaseType (значення

					ІАЛЦ.467200.003 ПЗ	Арк.
						43
Зм.	Арк.	№ докум.	Підпис	Дата		

«Course» або «Section») та PurchaseId (відповідний ідентифікатор курсу або секції).

```
public class Payment
{
    7 references
    public int Id { get; set; }
    8 references
    public string UserId { get; set; }
    1 reference
    public User User { get; set; }
    3 references
    public decimal Amount { get; set; }
    2 references
    public DateTime PaymentDate { get; set; } = DateTime.UtcNow;
    4 references
    public PaymentStatus PaymentStatus { get; set; } = PaymentStatus.Pending;
    4 references
    public string PaymentIntentId { get; set; }
    3 references
    public string ClientSecret { get; set; }
    6 references
    public string PurchaseType { get; set; }
    6 references
    public int PurchaseId { get; set; }
}
```

Рисунок 3.15 – Клас Payment для збереження інформації про оплату

У таблиці Payment визначено такі поля:

- UserId – ідентифікатор користувача, який здійснює оплату;
- Amount – сума оплати (типу decimal);
- PaymentDate – дата і час здійснення оплати (типу DateTime);
- PaymentStatus – статус оплати (Pending, PaymentReceived тощо);
- PaymentIntentId – ідентифікатор платіжного наміру в Stripe;
- ClientSecret – токен доступу до платіжного наміру, який необхідний користувачу для проведення оплати;
- PurchaseType – тип покупки: «Course» або «Section»;
- PurchaseId – ідентифікатор курсу або секції, до якого стосується оплата.

3.6.3 Створення платежу та проведення оплати

Система, що розробляється, не має права зберігати або обробляти дані карток користувачів, зокрема номер картки, термін дії чи CVV-код, бо не має

					ІАЛЦ.467200.003 ПЗ	Арк.
						44
Зм.	Арк.	№ докум.	Підпис	Дата		

відповідної сертифікації. Такі дані мають потрапляти безпосередньо до платіжної системи, оминаючи сервер. Для цього використовується клієнтська бібліотека Stripe.js, яка забезпечує безпечне передання даних картки.

На рис. 3.16 зображено діаграму послідовності, яка показує повний процес взаємодії між користувачем, клієнтським інтерфейсом, сервером і платіжною системою під час створення платежу та проведення оплати.

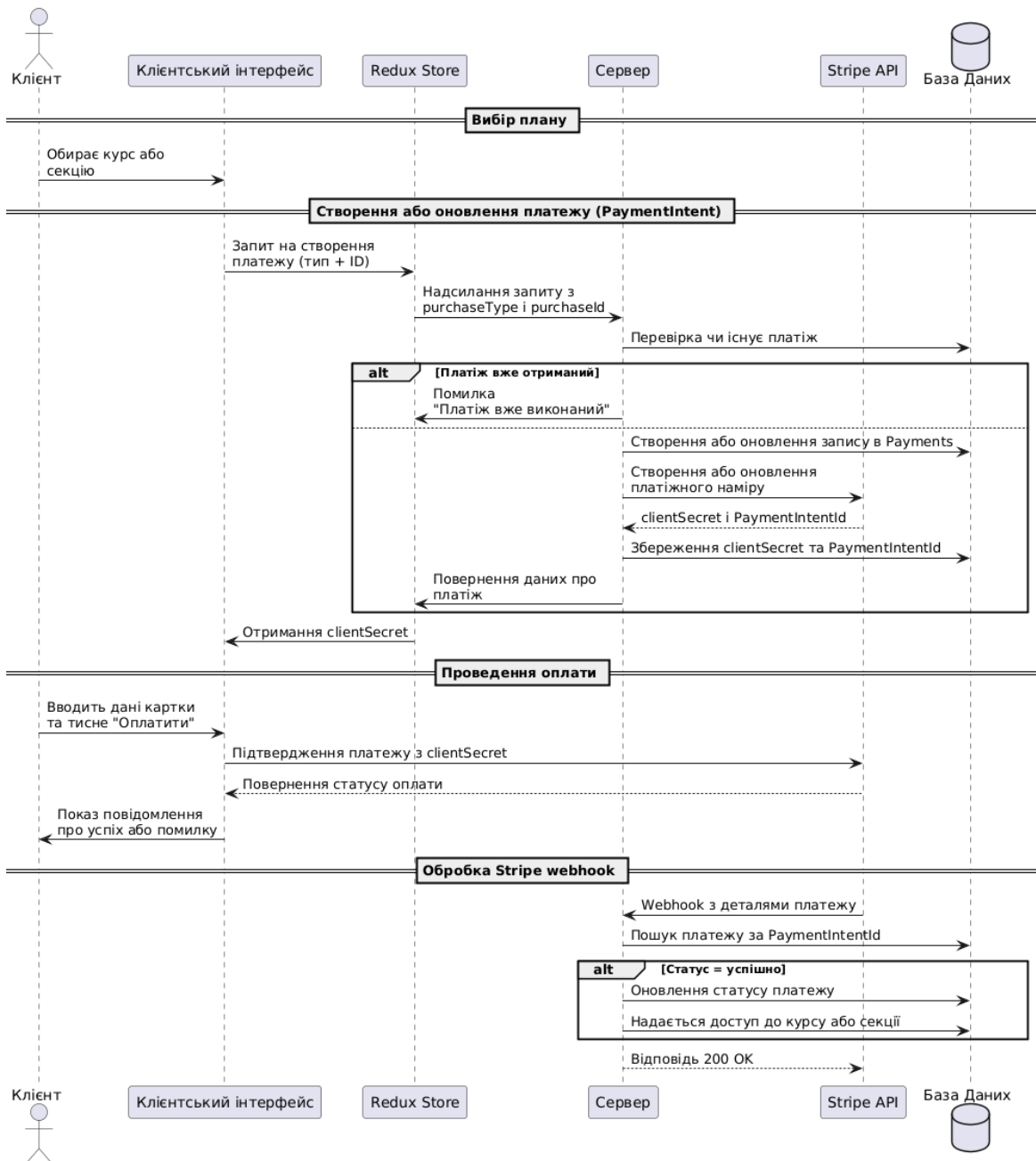


Рисунок 3.16 – Діаграма послідовності для процесу створення платежу та підтвердження оплати

Процес починається з того, що користувач обирає, що саме він хоче придбати – повний курс чи тільки окрему секцію. На основі цього клієнтська частина формує запит на створення платежу, в який додає інформацію про `purchaseType` і `purchaseId`. Далі запит через `Redux Store` надсилається на сервер. Сервер перевіряє, чи вже існує відповідний запис у таблиці `Payment`. Якщо такий запис знайдено і має статус «успішно оплачено», користувачу повертається повідомлення про те, що повторна оплата не потрібна. В іншому випадку – створюється новий запис або оновлюється наявний.

Далі сервер взаємодіє зі `Stripe API` для створення або оновлення `PaymentIntent` (платіжного наміру). `Stripe` у відповідь надсилає унікальні значення `clientSecret` та `PaymentIntentId`, які зберігаються у базі даних. Саме `clientSecret` передається назад клієнту, оскільки він потрібен для підтвердження оплати з боку користувача.

На наступному етапі користувач вводить дані своєї картки у спеціальних полях від `Stripe`. Ці дані не передаються на сервер, а надсилаються безпосередньо до платіжної системи разом із отриманим `clientSecret`. За наданим `clientSecret` платіжна система буде знати скільки і за що користувач платить. Після обробки `Stripe` повертає статус транзакції, який відображається користувачу на екрані: повідомлення про успіх або помилку.

3.6.4 Підтвердження платежу через `Webhook Stripe`

Оскільки сервер не обробляє платіжну транзакцію, а клієнтський інтерфейс взаємодіє напрямку зі `Stripe`, виникає необхідність знайти рішення як серверній частині дізнатись чи була оплата проведена і головне, чи успішно. Це необхідно для того, щоб записати в БД, що користувач тепер має доступ до матеріалу, який оплатив.

					ІАЛЦ.467200.003 ПЗ	Арк.
						46
Зм.	Арк.	№ докум.	Підпис	Дата		

Для цього Stripe підтримує механізм webhook – коли відбувається певна подія, сервіс надсилає запит за вказаною адресою. У випадку навчальної платформи це подія успішного платежу (charge.succeeded). Для цього розроблено окремий маршрут /api/payments/webhook. Коли на вказану адресу приходить запит про успішну оплату, сервер отримує дані про транзакцію і шукає відповідний платіж у таблиці Payment. Якщо платіж знайдено і він стосується курсу або секції, користувачу надається доступ до відповідного навчального матеріалу. Крім того, змінюється статус оплати у БД на «оплачено».

Для локального тестування webhook Stripe надає зручну команду, яка дозволяє підписатись на подію charge.succeeded і автоматично пересилати її на локальний сервер (рис. 3.17).

```
PS C:\Users\...\platform-for-courses\API> stripe login
Your pairing code is: nifty-warmer-outwit-divine
This pairing code verifies your authentication with Stripe.
Press Enter to open the browser or visit https://dashboard.stripe.com/stripecli/confirm_auth?t=gXeqEftycE8gPMrAopzZhwumKZLSHE1A (^C to quit)
> Done! The Stripe CLI is configured for MathCourse with account id acct_1PdqnLIg00EBkZA

Please note: this key will expire after 90 days, at which point you'll need to re-authenticate.
PS C:\Users\...\platform-for-courses\API> stripe listen -f http://localhost:5000/api/payments/webhook -e charge.succeeded
A newer version of the Stripe CLI is available, please update to: v1.27.0
> Ready! You are using Stripe API Version [2024-06-20]. Your webhook signing secret is whsec_d39ec5b4f72720ae57644ffb808eae4847f047ed2d717a86e
```

Рисунок 3.17 – Підписка на подію успішного платежу Stripe у режимі розробки

Після запуску цієї команди Stripe буде автоматично надсилати інформацію про всі успішні транзакції на зазначений маршрут, де сервер зможе їх обробити.

ВИСНОВКИ ДО РОЗДІЛУ 3

У третьому розділі дипломного проєкту виконано розробку ключового функціоналу для навчальної платформи. Зокрема, реалізовано такі компоненти:

- Описано архітектуру серверної частини платформи, включаючи контролери, сервіси, репозиторії.
- Реалізовано систему авторизації користувачів із захистом доступу до навчальних матеріалів.
- Побудовано і представлено схему структури курсів зі всіма компонентами, а також реалізовано механізм прив'язки навчального прогресу до конкретного користувача.
- Налаштовано інтеграцію з хмарним сховищем для зберігання відеоматеріалів, реалізовано функцію завантаження відео.
- Впроваджено інтеграцію з платіжною системою, наведено діаграму послідовності процесу створення платежу, підтвердження оплати та надання доступу до відповідних навчальних матеріалів.

					ІАЛЦ.467200.003 ПЗ	Арк.
						48
Зм.	Арк.	№ докум.	Підпис	Дата		

РОЗДІЛ 4 ТЕСТУВАННЯ І ОЦІНКА РЕЗУЛЬТАТІВ

У цьому розділі описано процес перевірки працездатності основних функцій системи, зокрема авторизації, розміщення навчального контенту та взаємодії користувача з курсами. Мета тестування – забезпечити стабільну роботу системи, виявити помилки та оцінити готовність до впровадження.

4.1 Тестування авторизації та реєстрації

Для реєстрації або входу в акаунт передбачено дві вкладки на верхній панелі керування: «Зареєструватися» та «Увійти», які розміщені у правому верхньому куті. Після натискання відкривається відповідна форма для введення імені користувача та пароля. Обидві форми містять посилання одна на одну: на формі реєстрації є напис «Вже маєте акаунт? Увійти», а на формі входу – «Ще не маєте акаунта? Зареєструватися».

При реєстрації реалізовано перевірку даних як на клієнтському, так і на серверному боці. У процесі введення користувач бачить підказки щодо правильного заповнення полів. Наприклад, при створенні пароля система вимагає, щоб він містив щонайменше одну велику та одну малу літеру й мав довжину від 6 до 12 символів (рис. 4.1).

Рисунок 4.1 – Перевірка пароля на формі реєстрації

					ІАЛЦ.467200.003 ПЗ	Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		49

Після натискання кнопки реєстрації, сервер додатково перевіряє унікальність імені користувача. Якщо акаунт із таким іменем уже існує, користувач бачить відповідне повідомлення про помилку. У разі успішної реєстрації з'являється зелене сповіщення, після чого користувача перенаправляє на форму входу (рис. 4.2).

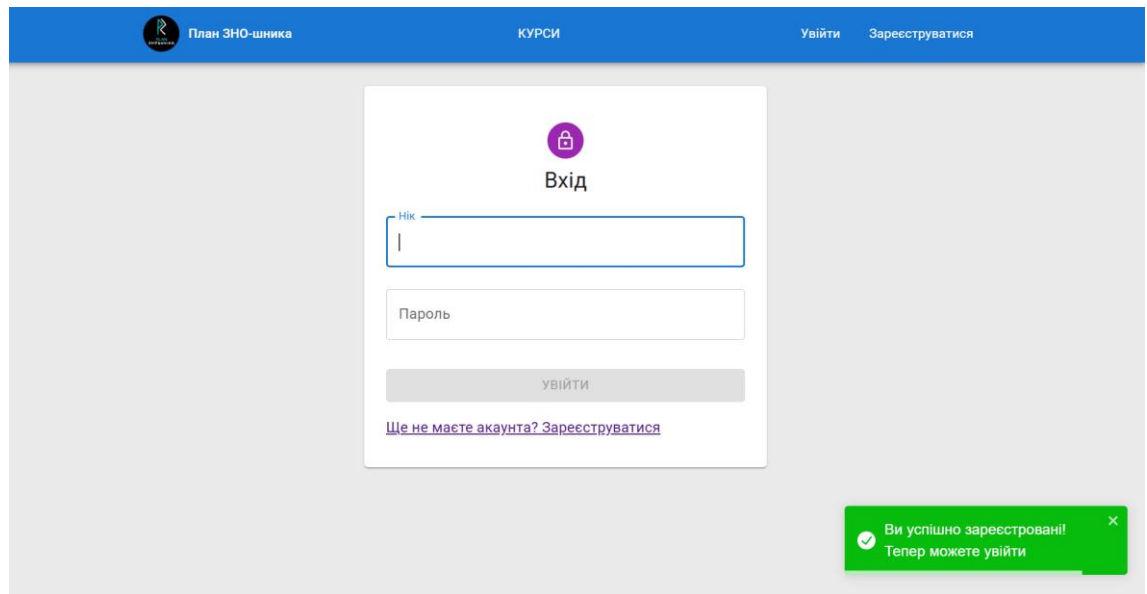


Рисунок 4.2 – Сповіщення про успішну реєстрацію

Після успішного входу користувач автоматично перенаправляється на головну сторінку – каталог курсів (рис. 4.3).

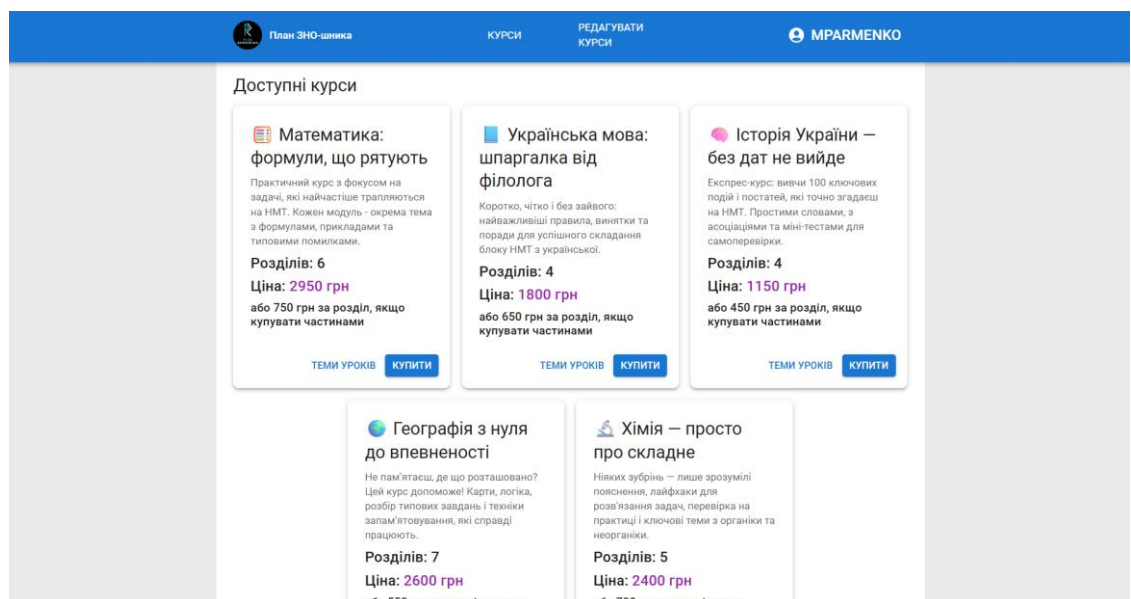


Рисунок 4.3 – Сторінка після успішного входу в акаунт

					ІАЛЦ.467200.003 ПЗ	Арк.
						50
Зм.	Арк.	№ докум.	Підпис	Дата		

Також після входу замість вкладок «Зареєструватися» і «Увійти» відображається ім'я користувача, що підтверджує його авторизацію в системі. Якщо при реєстрації було обрано роль викладача, користувач додатково бачить вкладку «Редагувати курси».

Обов'язковим є також тестування негативних сценаріїв, зокрема введення неправильного пароля. Якщо сервер повертає помилку з кодом 401 Unauthorized, з'являється червоне сповіщення з текстом помилки. У разі п'яти невдалих спроб входу акаунт блокується на 15 хвилин. При шостій спробі користувач бачить аналогічне червоне сповіщення, але вже з повідомленням про тимчасове блокування акаунта (рис. 4.4).

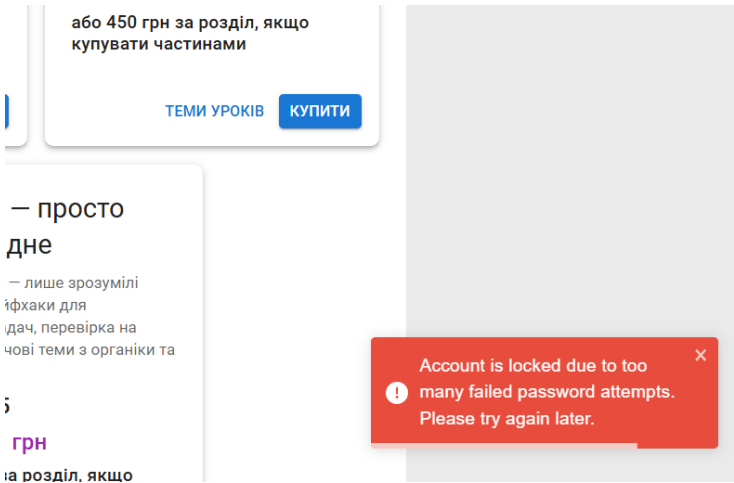


Рисунок 4.4 – Сповіщення про тимчасове блокування акаунта

Інформацію про заблоковані акаунти можна переглядати в БД. На рис. 4.5 показано результат відповідного запиту.

Results: untitled:untitled-1 X

2 rows returned

	UserName character varying	LockoutEnd timestamp with time zone	AccessFailedCount integer
1	ben	2025-05-29 22:01:46.903659+00	0
2	ben2	null	2

Рисунок 4.5 – Перелік заблокованих акаунтів або акаунтів із невдалими спробами входу

Щоб вивести всі такі акаунти, потрібно виконати запит до таблиці користувачів (AspNetUsers), де поле часу завершення блокування (LockoutEnd) не є порожнім, або значення в полі кількості невдалих спроб (AccessFailedCount) більше нуля.

4.2 Тестування функціональності створення навчального курсу

4.2.1 Створення курсу, розділів, уроків

Доступ до створення нового курсу має лише зареєстрований користувач із роллю викладача. Після входу в акаунт та переходу на вкладку «Редагувати курси» відкривається інтерфейс для керування курсами. У правому верхньому куті знаходиться кнопка «Створити», після натискання якої відкривається форма створення нового курсу.

У формі потрібно вказати такі обов'язкові поля (рис. 4.6):

- назва курсу;
- короткий опис;
- повна ціна;
- ціна розділу.

При спробі зберегти курс без заповнення всіх обов'язкових полів, відповідні поля підсвічуються.

Далі користувач може додати до курсу розділи. Для цього передбачена кнопка «Додати розділ», яка завжди відображається в кінці списку після останнього розділу. Кожен розділ має назву та необов'язковий опис. Після створення розділу стає доступною опція додавання уроків до нього.

					ІАЛЦ.467200.003 ПЗ	Арк.
						52
Зм.	Арк.	№ докум.	Підпис	Дата		

Рисунок 4.6 – Форма для заповнення даних про курс

Додавання уроку відбувається через окрему форму (рис. 4.7), в якій потрібно вказати назву і опис уроку, додати відео. Після збереження урок відображається в списку уроків даного розділу.

Рисунок 4.7 – Форма для заповнення даних про урок

На рис. 4.8 зображено приклад заповненої структури курсу з кількома розділами та уроками.

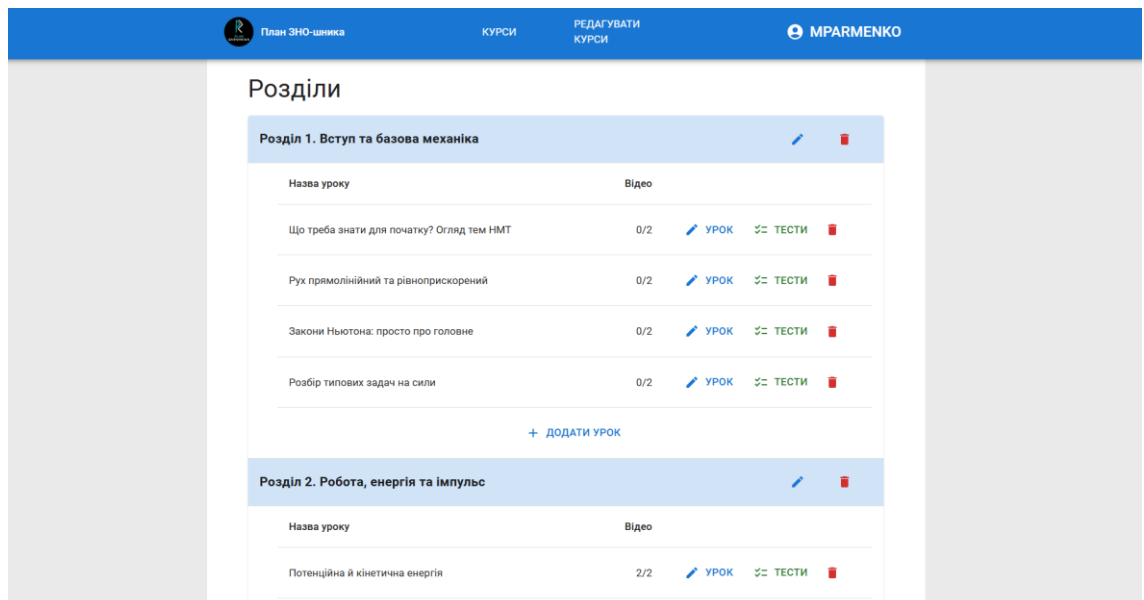


Рисунок 4.8 – Приклад структури створеного курсу

Функціональність також дозволяє редагувати або видаляти будь-який курс, розділ чи урок. Для цього біля кожного елементу є відповідні кнопки.

4.2.2 Завантаження відео, перевірка інтеграції з хмарним сховищем

Під час тестування важливо перевірити, чи працює механізм завантаження відео та чи коректно відбувається інтеграція з хмарним сховищем, у якому ці файли зберігаються.

Форма створення або редагування уроку містить поля для завантаження відеофайлів. Користувач може або натиснути на область вибору файлу і вибрати відео з його файлової системи, або перетягнути відео на сайт. Доданий файл користувач одразу бачить. На рис. 4.9 показано вигляд форми уроку з доданими відео.

Після натиснення кнопки «Зберегти» система починає процес завантаження у хмарне сховище, про що користувача інформує індикатор прогресу. Після успішного завершення завантаження система отримує URL-

посилання на відео з хмарного сервісу, яке автоматично додається до вмісту уроку.

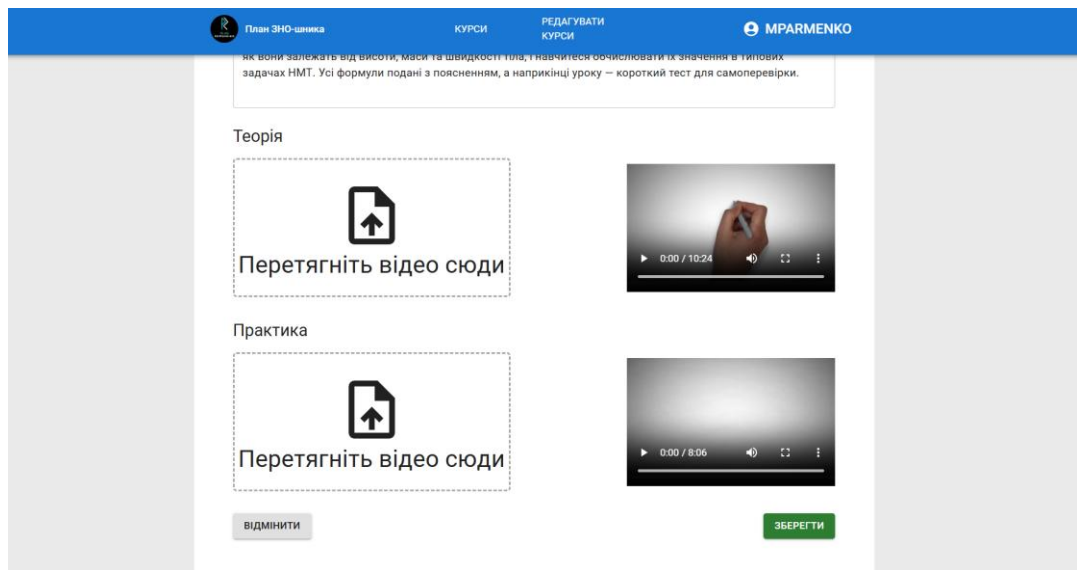


Рисунок 4.9 – Форма уроку з доданими відео

Під час тестування також перевірено:

- чи генерується посилання на відео в базі даних;
- чи відео відтворюється за посиланням;
- чи відео завантажено в сховище Stripe.

4.2.3 Додавання тестів

У редакторі курсу навпроти кожного уроку є кнопка «Тести», яка відкриває інтерфейс створення запитань. Користувач може додати нове запитання і варіанти відповідей до нього. Всі створені питання і варіанти відповідей одразу зберігаються в БД.

На рис. 4.10 показано приклад створеного питання з варіантами відповіді.

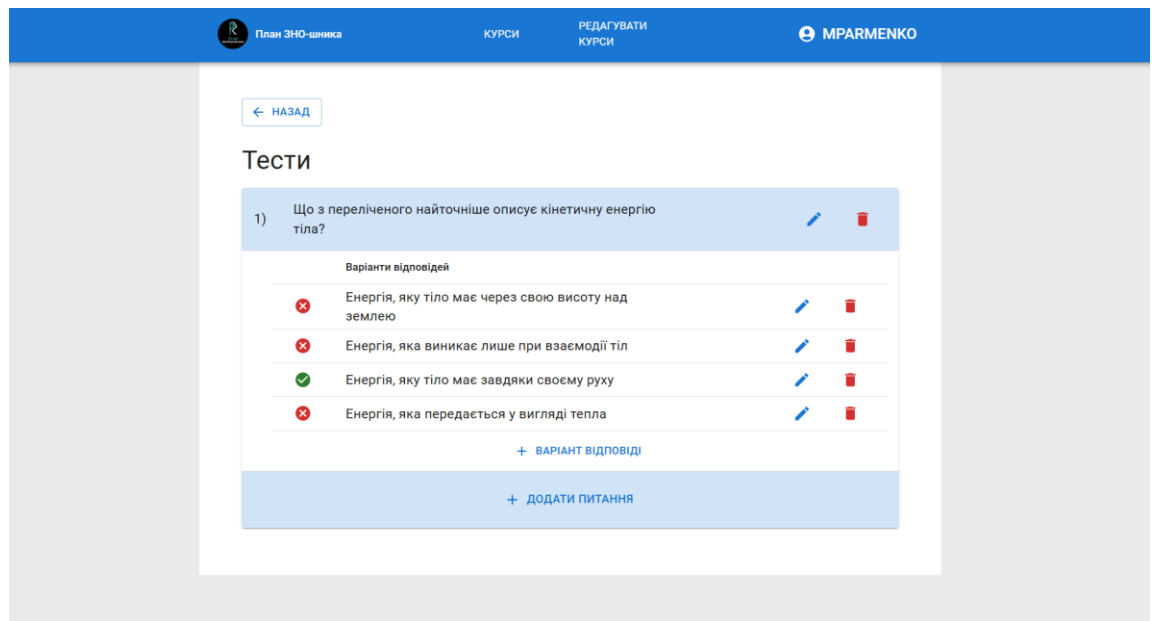


Рисунок 4.10 – Приклад створеного запитання

Під час тестування також перевірено зміну правильної відповіді і сценарії редагування, видалення питань та варіантів відповідей. Якщо користувач видалить питання, яке має варіанти відповідей, тоді вони також видаляються з БД.

4.2.4 Публікація і поширення курсу

Після створення та заповнення змісту навчального курсу викладач має можливість керувати його публікацією та поширенням.

На сторінці «Редагувати курси» відображається список усіх курсів, створених поточним викладачем. Для кожного курсу виводиться наступна інформація:

- назва курсу;
- статистика – кількість користувачів, які придбали доступ до курсу, а також сума отриманих платежів;

- вартість – відображається повна вартість курсу і ціна за окремий розділ;
- кількість розділів – дозволяє швидко оцінити масштаб курсу;
- стан курсу – активний або неактивний. Активний курс доступний для перегляду користувачами на платформі, неактивний – недоступний у публічному каталозі.

На рис. 4.11 представлено інтерфейс сторінки з прикладами курсів у різних станах публікації.

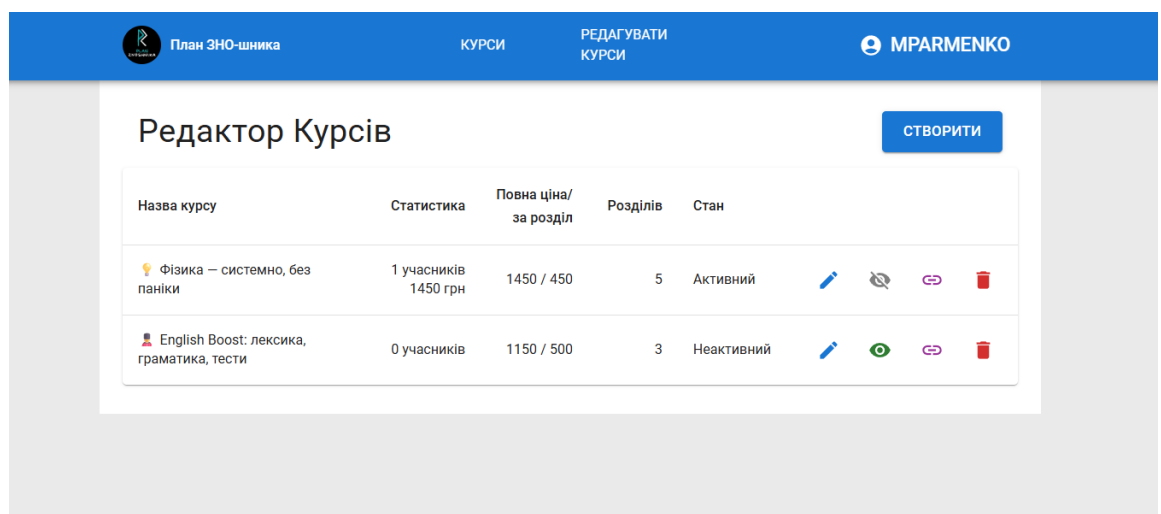


Рисунок 4.11 – Інтерфейс керування курсами

Курс за замовчуванням створюється з неактивним статусом. Таким чином, курс не є доступним для всіх одразу після створення. Коли викладач готовий публікувати курс, тоді його потрібно активувати. Перевірено коректність зміни статусу після натискання кнопки активації/деактивації.

Після опублікування курсу викладач також має змогу поширювати курс. Перевірено роботу кнопки для копіювання URL-посилання на курс. Результат показано на рис. 4.12.

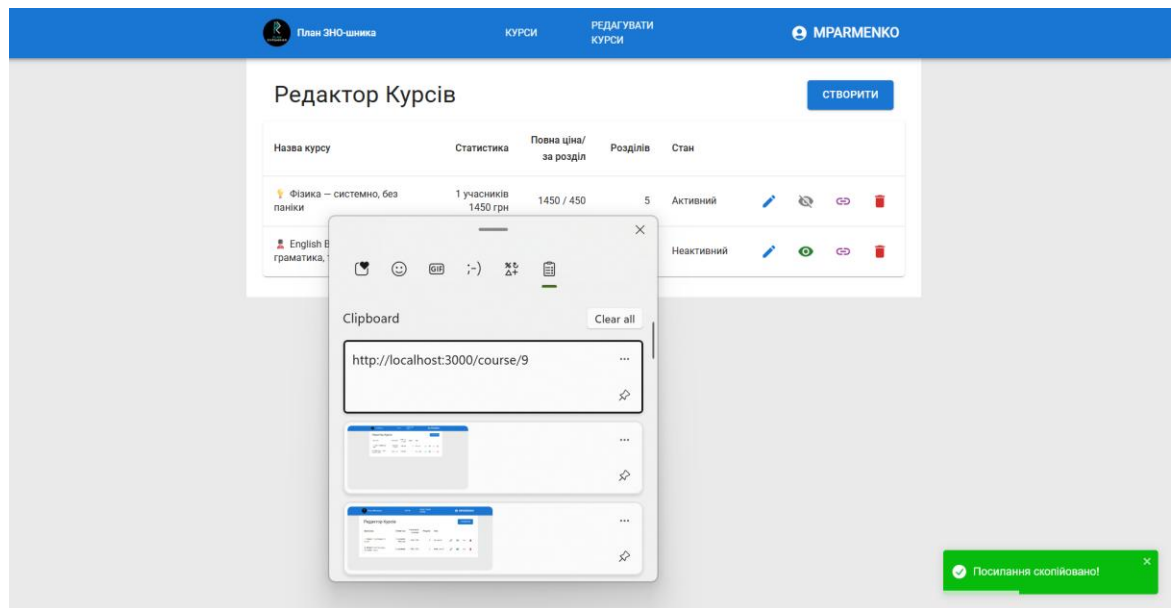


Рисунок 4.12 – Посилання на курс скопійоване в буфер обміну

Усі функції пройшли тестування і відповідають очікуваній поведінці.

4.3 Взаємодія користувача з навчальними матеріалами

4.3.1 Придбання курсу

Доступ до курсу повинен бути лише після його придбання. Але перед цим користувач може переглянути структуру курсу: які розділи і уроки входять. На рис. 4.13 зображено вигляд сторінки курсу для користувача, коли він не має доступу до курсу.

Також перевірено, що перехід до уроків недоступний не тільки з структури курсу, а й якщо ввести точне посилання на один з уроків, то показано лише повідомлення, що такої сторінки не існує.

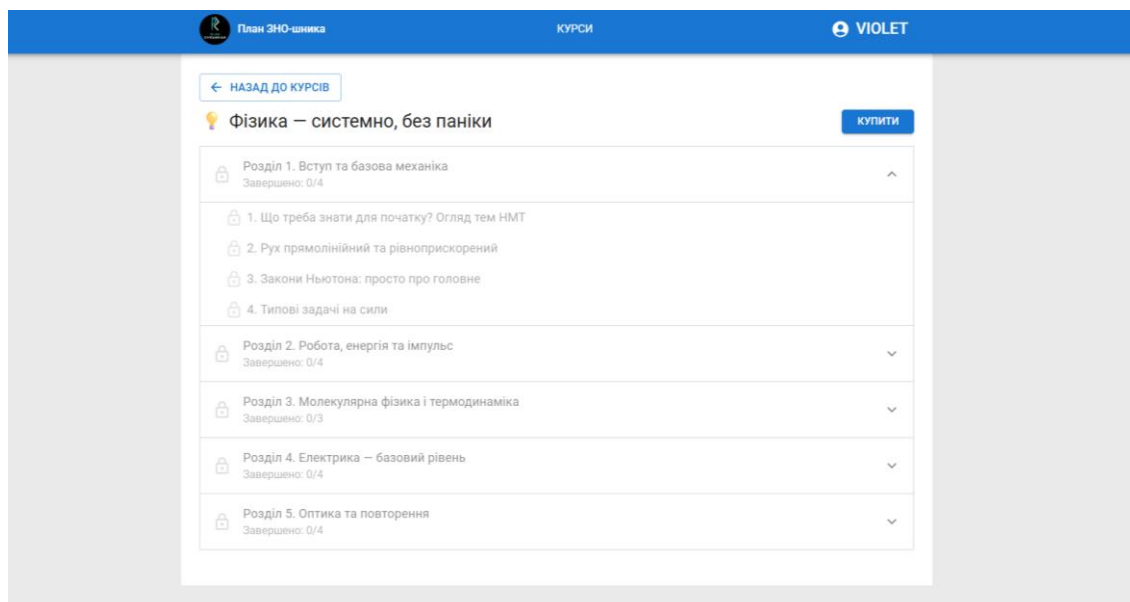


Рисунок 4.13 – Сторінка огляду недоступного курсу

Для придбання курсу на його сторінці або в загальному каталозі користувач бачить кнопку «Купити». Після натискання відкривається сторінка для оформлення оплати. Спочатку користувач обирає план: оплатити весь курс або частинами по розділу. Далі переглядає суму до оплати, переходить на форму для заповнення даних карти і натискає кнопку «Оплатити». Після успішно проведеного платежу користувач бачить повідомлення (рис. 4.14).

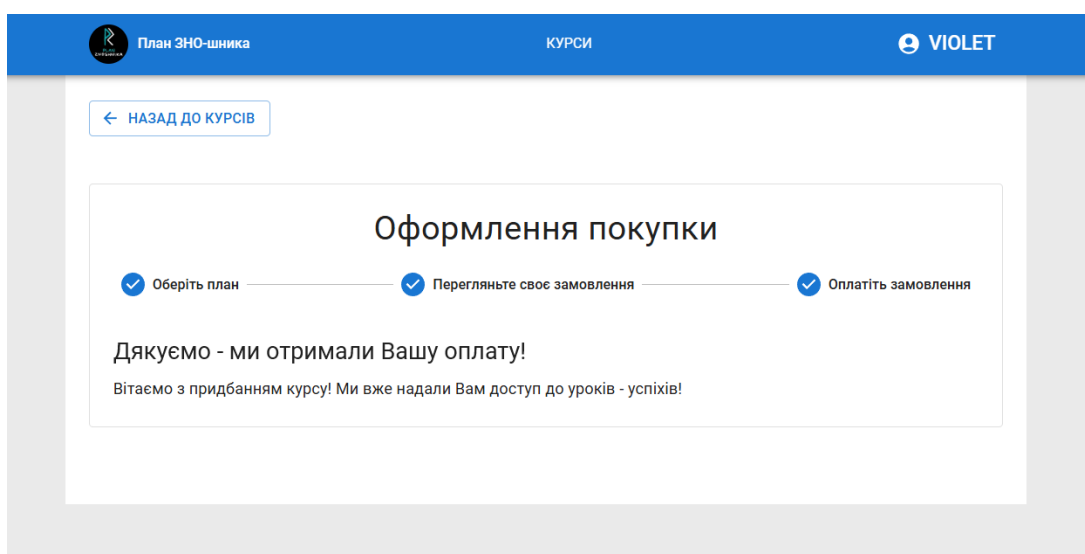


Рисунок 4.14 – Повідомлення про успішне придбання курсу

Також у ході тестування було перевірено такі аспекти:

- чи коректно відображається вартість курсу;
- чи неможливо здійснити оплату, якщо користувач не авторизований (у такому разі система переадресовує на сторінку входу);
- чи передається правильна інформація в платіжну систему (сумарна ціна, ідентифікатори користувача і курсу);
- чи обробляються відповіді від платіжного сервісу;
- у базі даних створюється запис про доступ користувача до відповідного курсу або розділу;
- інформація про платіж зберігається у відповідній таблиці БД;
- курс з'являється в особистому кабінеті користувача;
- користувач може одразу розпочати перегляд матеріалів.

4.3.2 Перегляд відеоуроків

Після переходу до сторінки уроку, користувач бачить його назву, опис, відео з теорією і практикою, якщо вони є, та результат тестування.

Однією з особливостей є можливість відстеження прогресу перегляду. Біля кожного відеоуроку в інтерфейсі відображається прапорець з підписом, який позначає, чи вважається відео переглянутим. Цей прапорець має дві логіки спрацювання:

- ручна – користувач самостійно може встановити або зняти позначку;
- автоматична – якщо під час перегляду відео до його завершення залишається менше ніж 10 секунд, система автоматично відмічає урок як переглянутий.

На рис. 4.15 зображено приклад інтерфейсу перегляду уроку з активним прапорцем «Переглянуто».

					ІАЛЦ.467200.003 ПЗ	Арк.
						60
Зм.	Арк.	№ докум.	Підпис	Дата		

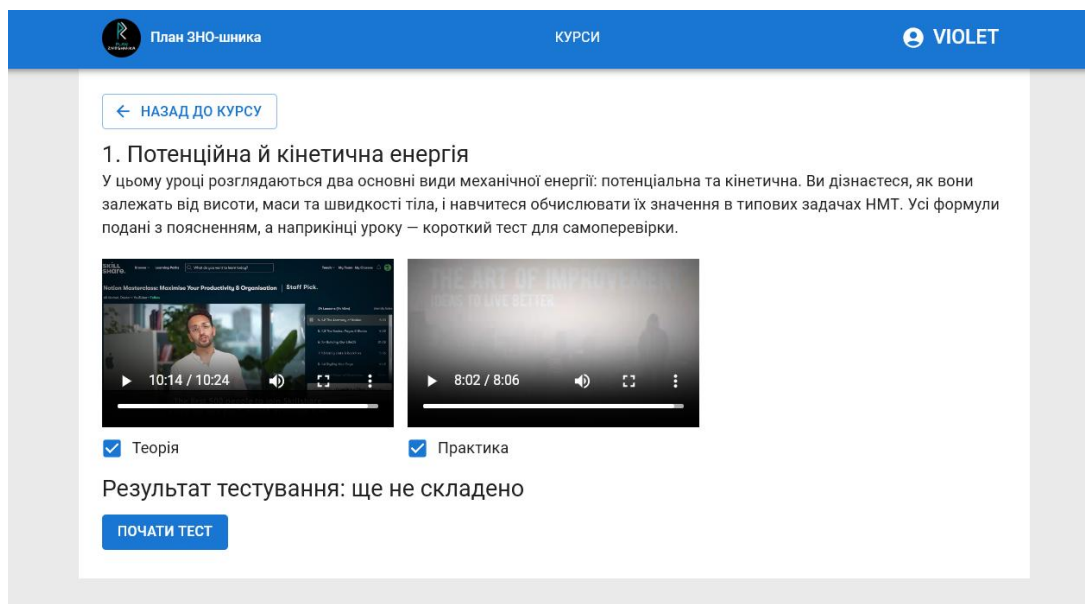


Рисунок 4.15 – Активна відмітка про перегляд відеоуроку

Під час тестування перевірено:

- чи прапорець відображається біля кожного відео;
- чи зберігається його стан при перезавантаженні сторінки (в базі або локально);
- чи автоматично спрацьовує логіка позначення після досягнення 10 секунд до кінця;
- чи не виникає конфліктів між ручною і автоматичною діями;
- чи можна скасувати позначку після перегляду (наприклад, якщо користувач хоче переглянути ще раз).

4.3.3 Проходження тестування

Після перегляду відеоуроку користувач має можливість одразу перевірити свої знання натиснувши на кнопку «Почати тест» внизу сторінки уроку.

					ІАЛЦ.467200.003 ПЗ	Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		61

Особливістю реалізації є вільна навігація між питаннями – користувач може перемикатися між ними в будь-якому порядку, не обов’язково проходити послідовно. Для кожного запитання передбачено кнопку «Відповісти», після натискання якої відбувається миттєва перевірка результату.

Після підтвердження:

- правильна відповідь підсвічується зеленим кольором;
- якщо користувач обрав неправильний варіант, він підсвічується червоним, а правильний все одно показується для порівняння.

Це дозволяє користувачеві одразу бачити результат і краще засвоїти матеріал (рис. 4.16).

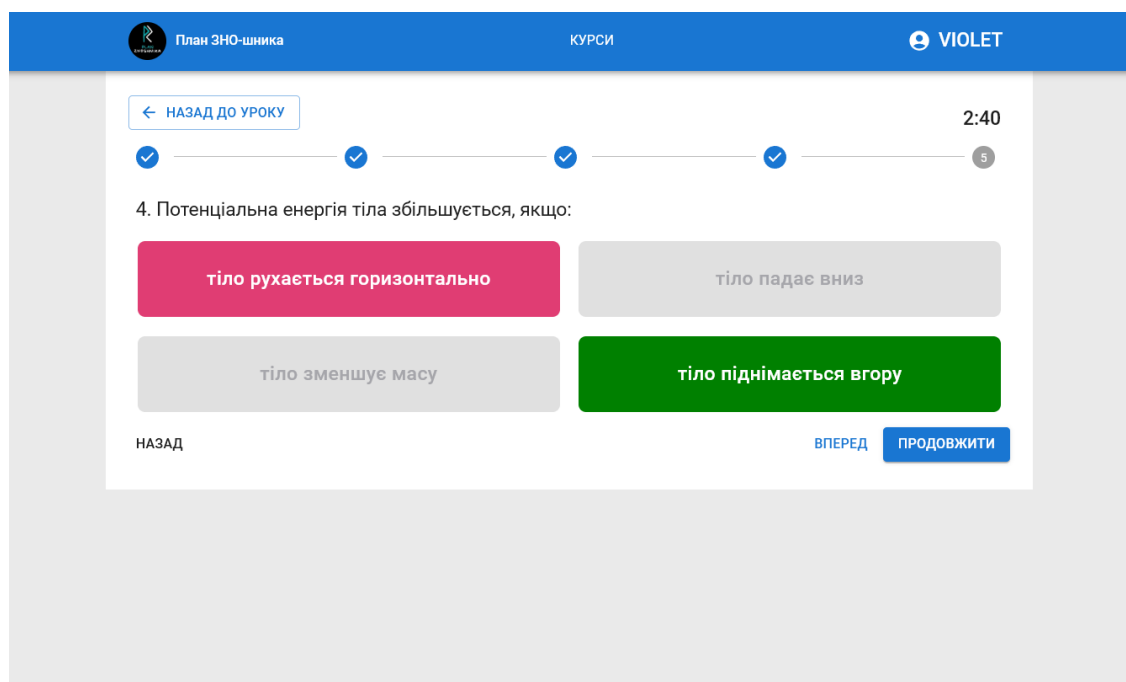


Рисунок 4.16 – Результати проходження тесту з підсвічуванням правильних і неправильних відповідей

Після завершення тесту, окрім загального відсоткового результату, користувач бачить мотиваційне повідомлення, яке формується на основі як його особистого балу, так і загальної статистики інших учасників. Для цього

система надсилає запит до API з ідентифікатором уроку та бажаним відсотком топу, за замовчуванням – 25%.

На основі отриманих статистичних даних система показує одне з таких повідомлень:

- Якщо результат менше ніж 50%: «Схоже, вам є над чим попрацювати. Спробуйте повторити тему для кращого засвоєння.»
- Якщо користувач увійшов у найкращі 25% за результатами: «ЮХУ!.. Ви серед 25% учасників з найкращими результатами. Продовжуйте в тому ж дусі!»
- Якщо результат 80% або більше (і не в найкращих 25%): «Ви отримали відмінний результат – 80% і більше! Чудова робота!»
- Якщо результат вищий за середній (але не в найкращих і менше ніж 80%): «Ваш результат вище середнього серед інших учасників! Молодець!»
- В інших випадках (при середньому або трохи нижчому результаті): «Супер! Це гарний показник, але ще є куди прямувати!» (рис. 4.17).

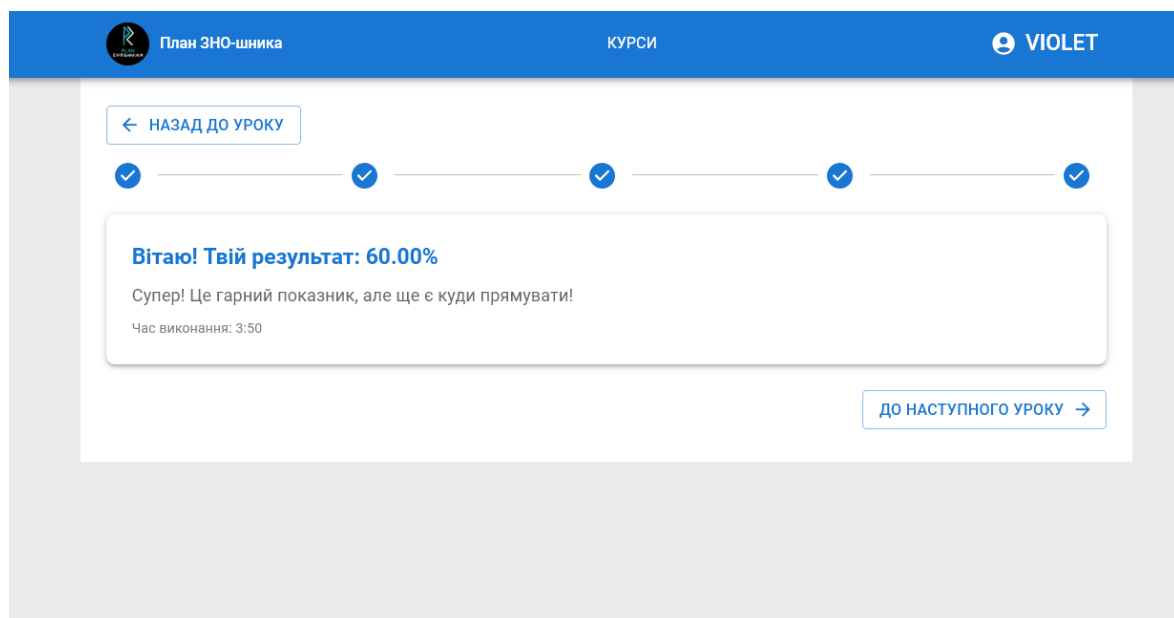


Рисунок 4.17 – Результат після проходження тесту

					ІАЛЦ.467200.003 ПЗ	Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		63

Цей підхід дозволяє не просто показати статистику, а й підтримати мотивацію користувача, незалежно від результату.

4.3.4 Перегляд прогресу навчання

Прогрес користувача відображається візуально на сторінці курсу. Це дає змогу швидко орієнтуватися, які уроки вже завершені, які перебувають у процесі проходження, а які ще не були розпочаті. Для цього перед кожним розділом і кожним уроком у структурі курсу відображається спеціальна іконка, що відповідає поточному статусу.

На рис. 4.18 показано приклад інтерфейсу структури курсу з іконками прогресу.

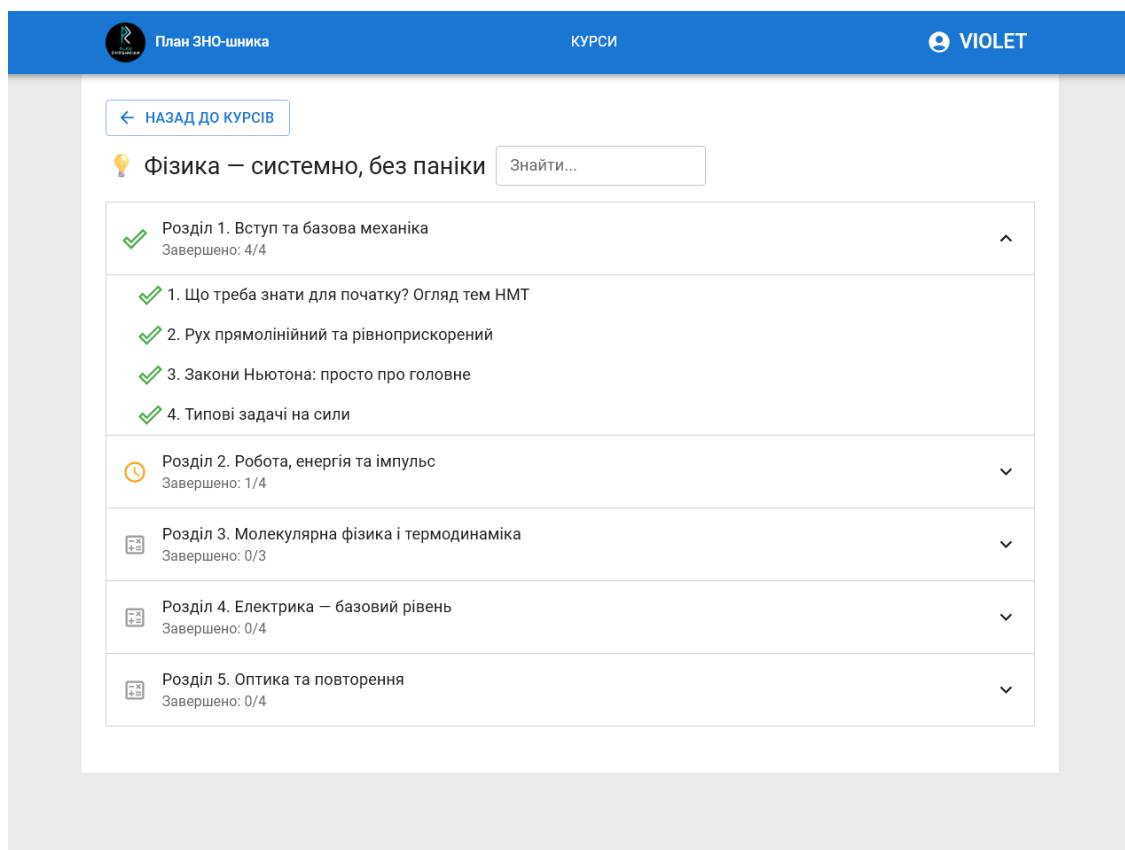


Рисунок 4.18 – Візуальне відображення прогресу навчання

Передбачено три стани:

- нерозпочатий: позначається нейтральною іконкою калькулятора, що має темно-сірий колір;
- у процесі: позначається іконкою помаранчевого годинника, який відображає, що користувач розпочав урок або пройшов частину матеріалу;
- завершено: позначається зеленою галочкою, яка з'являється після повного завершення уроку: відео переглянуто, тест пройдено. Розділ отримує статус завершеного, якщо всі уроки всередині нього мають статус «завершено».

Завдяки простій, інтуїтивно-зрозумілій кольоровій індикації користувач бачить свій прогрес і відчуває досягнення при переході до нових тем.

					ІАЛЦ.467200.003 ПЗ	Арк.
						65
Зм.	Арк.	№ докум.	Підпис	Дата		

ВИСНОВКИ ДО РОЗДІЛУ 4

У четвертому розділі дипломного проєкту проведено тестування можливостей розробленої навчальної платформи для розміщення і проходження курсів.

Протестовано ключові сценарії використання:

- механізм авторизації та реєстрації;
- створення курсів, розділів і уроків, завантаження відеоматеріалів, додавання тестів, публікація;
- оплата й отримання доступу до курсу;
- перегляд відеоуроків, проходження тестів, візуалізація прогресу навчання.

За результатами тестування можна зробити висновок, що система успішно реалізує всі заплановані функції, забезпечує логічну та зрозумілу взаємодію з користувачем, контролює доступ, а також реагує на типові помилки відповідними повідомленнями.

					ІАЛЦ.467200.003 ПЗ	Арк.
						66
Зм.	Арк.	№ докум.	Підпис	Дата		

ВИСНОВКИ

У межах дипломного проєкту успішно реалізовано повнофункціональну навчальну платформу для розміщення та проходження онлайн-курсів.

У першому розділі проведено дослідження, що дало змогу визначити загальні тенденції розвитку онлайн-освіти та обґрунтувати доцільність створення власного рішення. Визначено ключові функціональні потреби користувачів – як викладачів, так і студентів. Аналіз наявних платформ виявив обмеження, такі як висока конкуренція та суворі вимоги до викладачів (наприклад, обов’язкова співпраця з університетами), які враховано при проєктуванні власної системи.

У другому розділі розроблено архітектуру платформи на основі клієнт-серверної моделі, що забезпечує розділення відповідальностей та сприяє масштабованості. Обґрунтовано вибір технологічного стека. Обрано сервіси для інтеграції з хмарним сховищем для зберігання відео та з платіжною системою для придбання курсів.

У третьому розділі реалізовано функціонал платформи, включаючи механізми авторизації, створення курсів і уроків, збереження навчального прогресу користувачів, завантаження відео та прийом оплат. Окрему увагу приділено побудові чіткої логіки доступу до матеріалів залежно від ролі користувача та стану оплати.

У четвертому розділі проведено функціональне тестування платформи. Результати перевірки показали, що всі основні сценарії – від реєстрації до проходження курсів – працюють стабільно. Інтерфейс є інтуїтивно зрозумілим, а система адекватно реагує на типові помилки користувача. Це свідчить про готовність системи до реального використання.

Отримані результати можуть бути використані як комерційний продукт для індивідуальних викладачів.

					ІАЛЦ.467200.003 ПЗ	Арк.
						67
Зм.	Арк.	№ докум.	Підпис	Дата		

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. GitHub - yaryna-bashchak/platform-for-courses. GitHub. URL: <https://github.com/yaryna-bashchak/platform-for-courses>.
2. The COVID-19 pandemic has changed education forever. This is how. World Economic Forum. URL: <https://www.weforum.org/stories/2020/04/coronavirus-education-global-covid19-online-digital-learning> (дата звернення: 15.04.2025).
3. The 10 benefits of online learning - school of education. School of Education. URL: <https://drexel.edu/soe/resources/student-teaching/advice/benefits-of-online-and-virtual-learning/> (дата звернення: 15.04.2025).
4. Online courses, learn anything on your schedule | Udemy. Udemy. URL: <https://www.udemy.com/> (дата звернення: 20.04.2025).
5. Coursera partnerships | coursera. Coursera. URL: <https://www.coursera.org/partnerships> (дата звернення: 20.04.2025).
6. Moodle в Україні: що таке moodle | moodle.org. Moodle challenge. URL: <https://moodle.org/mod/page/view.php?id=8174> (дата звернення: 21.04.2025).
7. Moodle в Україні: власний веб-сервер чи зовнішній хостинг? | moodle.org. Moodle challenge. URL: <https://moodle.org/mod/page/view.php?id=8180> (дата звернення: 21.04.2025).
8. Client-server architecture. Online courses from QATestLab | Main page. URL: <https://en.training.qatestlab.com/blog/technical-articles/client-server-architecture/> (дата звернення: 23.04.2025).
9. Angular vs React vs Vue: core differences | browserstack. BrowserStack. URL: <https://www.browserstack.com/guide/angular-vs-react-vs-vue> (дата звернення: 27.04.2025).

- 10.Порівнюємо React, Angular і Vue — найпопулярніші бібліотеки й фреймворки у 2022 році | DOU. Dou.ua. URL: <https://dou.ua/forums/topic/39933/> (дата звернення: 27.04.2025).
- 11.MUI: The React component library you always wanted. MUI: The React component library you always wanted. URL: <https://mui.com/> (дата звернення: 27.04.2025).
- 12.Hutsulyak O. .NET vs Node.js: what to choose in 2024 – TechMagic. Blog | TechMagic. URL: <https://www.techmagic.co/blog/node-js-vs-net-what-to-choose> (дата звернення: 29.04.2025).
- 13.ASP.NET vs Spring – a detailed comparison. Hire top developers for contract or full-time roles – Flexiple. URL: <https://flexiple.com/compare/asp-dot-net-vs-spring> (дата звернення: 30.04.2025).
- 14.Choosing the best database solution for your project. Yalantis. URL: <https://yalantis.com/blog/how-to-choose-a-database/> (дата звернення: 25.04.2025).
- 15.Technology | 2024 Stack Overflow Developer Survey. Stack Overflow Insights - Developer Hiring, Marketing, and User Research. URL: <https://survey.stackoverflow.co/2024/technology> (дата звернення: 25.04.2025).
- 16.Cloudinary image & video APIs overview | Cloudinary. URL: https://cloudinary.com/documentation/programmable_media_overview#landingpage (дата звернення: 18.05.2025).
- 17.The flexible payment system | Stripe Payments. Online payment service Stripe. URL: <https://stripe.com/payments#global-payments> (дата звернення: 18.05.2025).
- 18.About us. PCI Security Standards Council. URL: https://www.pcisecuritystandards.org/about_us/ (дата звернення: 21.05.2025).

					ІАЛЦ.467200.003 ПЗ	Арк.
						69
Зм.	Арк.	№ докум.	Підпис	Дата		

19. Microsoft.AspNetCore.Identity namespace. Microsoft Learn: Build skills that open doors in your career. URL: <https://learn.microsoft.com/en-us/dotnet/api/microsoft.aspnetcore.identity?view=aspnetcore-9.0> (дата звернення: 25.04.2025).
20. Configuration. AutoMapper documentation. URL: <https://docs.automapper.org/en/latest/Configuration.html#profile-instances> (дата звернення: 19.05.2025).
21. Material Icons - Material UI. MUI: The React component library you always wanted. URL: <https://mui.com/material-ui/material-icons/> (дата звернення: 19.05.2025).
22. Cloudinary .NET SDK Quick Start | Cloudinary. Image and video upload, storage, optimization and CDN. URL: https://cloudinary.com/documentation/dotnet_quickstart#1_set_up_and_configure_the_library (дата звернення: 22.05.2025).

					ІАЛЦ.467200.003 ПЗ	Арк.
						70
Зм.	Арк.	№ докум.	Підпис	Дата		

ДОДАТОК А

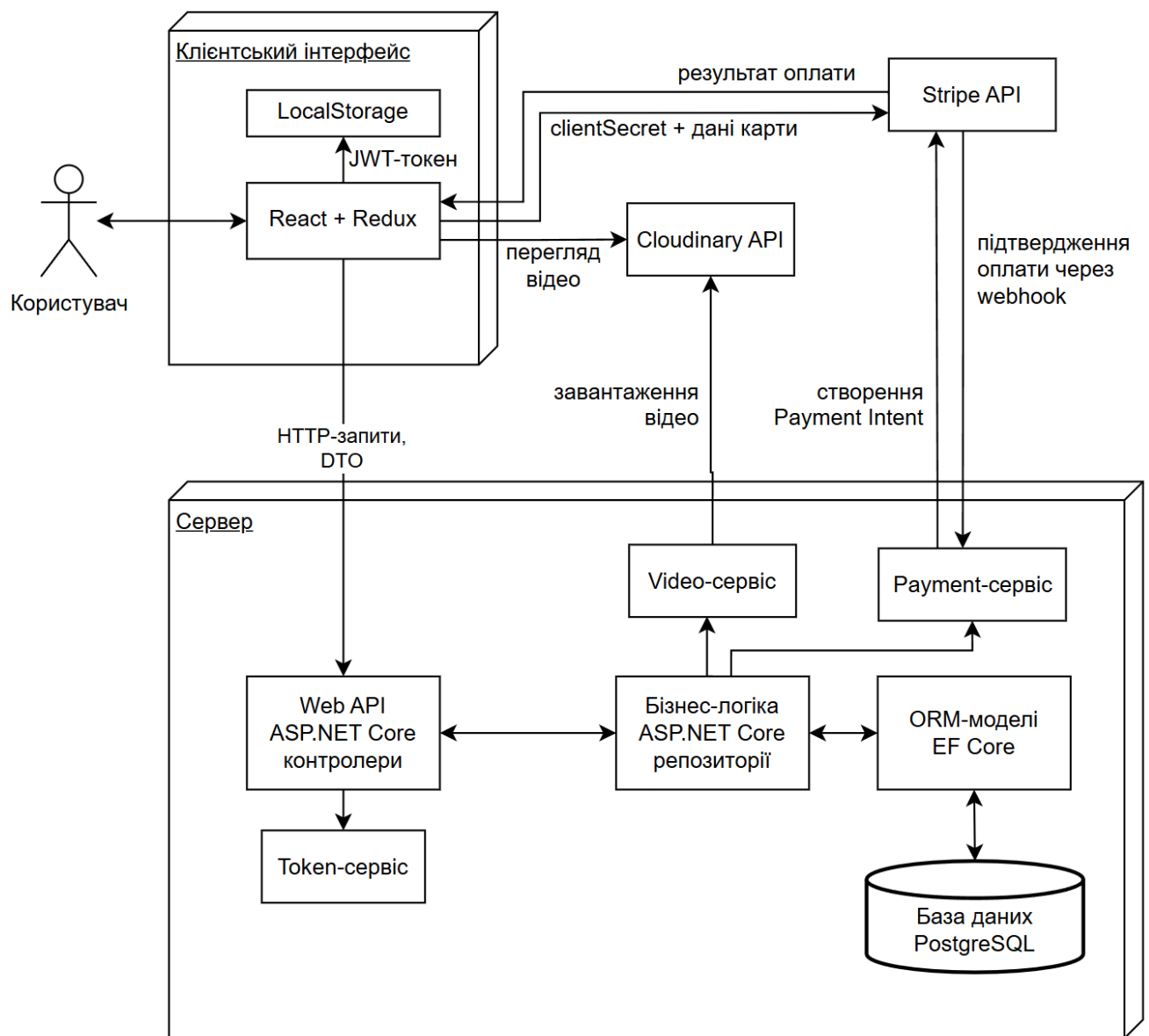
Навчальна платформа для розміщення і проходження курсів

Структурна схема системи

ІАЛЦ.467200.004 Д1

Аркушів 1

Київ 2025 р



					ІАЛЦ.467200.004 Д1			
		№ докум.	Підпис	Дата				
Розробив	Бащак Я. В.				Навчальна платформа для розміщення та проходження курсів Структурна схема системи		Літ.	Аркуш
Перевірив	Гайдай А. Р.							Аркушів
Реценз.	Шимкович В. М.						1	1
Н. Контр.	Нікольський С.С.						КПІ ім. Ігоря Сікорського, ФІОТ, ІМ-11	
Затвердив								

ДОДАТОК Б

Навчальна платформа для розміщення і проходження курсів

Функціональна схема (діаграма класів)

ІАЛЦ.467200.005 Д2

Аркушів 1

Київ 2025 р

ДОДАТОК В

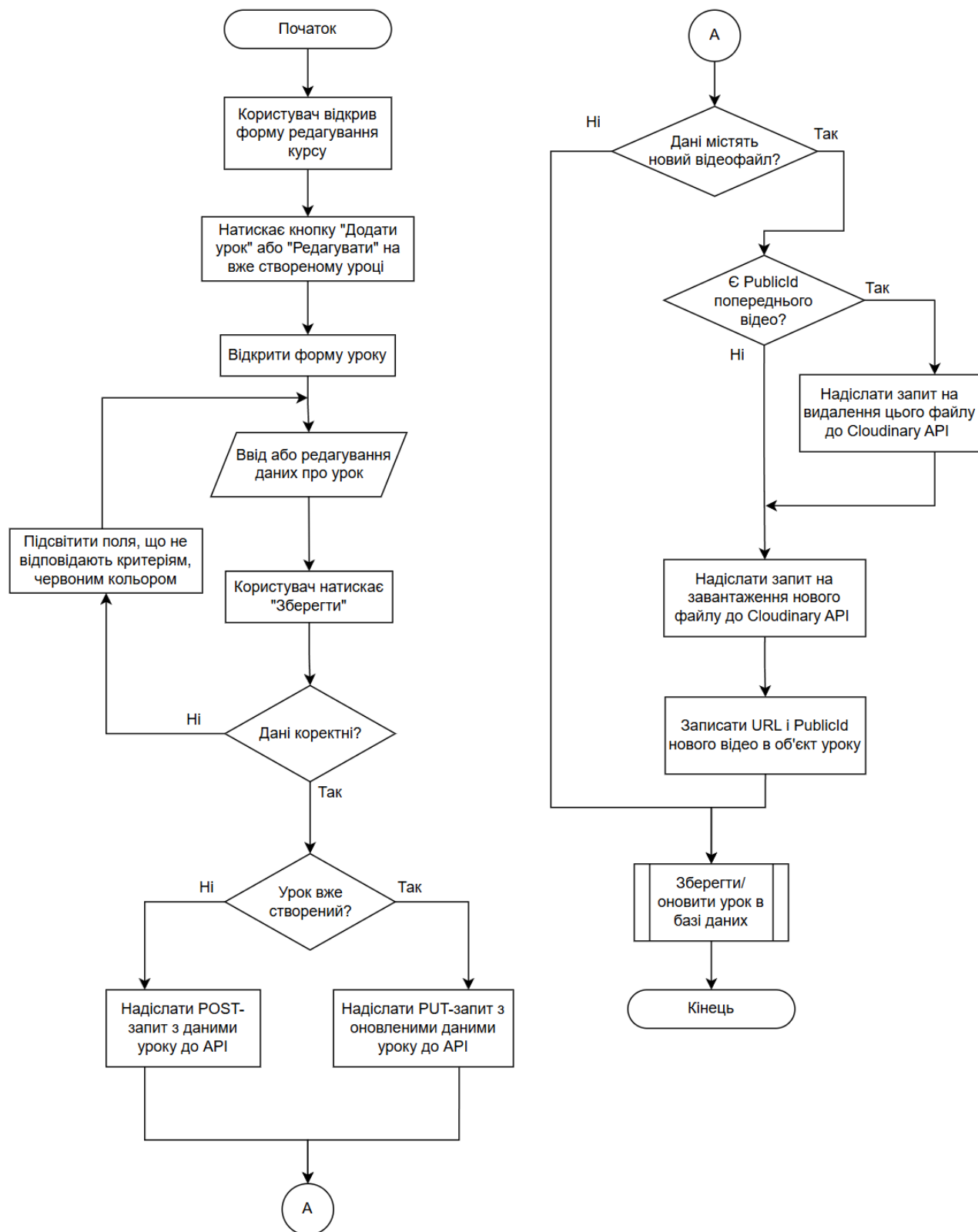
Навчальна платформа для розміщення і проходження курсів

Принципова схема (блок-схема алгоритму)

ІАЛЦ.467200.006 ДЗ

Аркушів 1

Київ 2025 р



					ІАЛЦ.467200.006 ДЗ		
		№ докум.	Підпис	Дата	Навчальна платформа для розміщення та проходження курсів Принципова схема (блок-схема алгоритму)		
Розробив	Бащак Я. В.						
Перевірив	Гайдай А. Р.						
Реценз.	Шимкович В. М.						
Н. Контр.	Нікольський С.С.						
Затвердив					Літ. Аркуш Аркушів 1 1 КПІ ім. Ігоря Сікорського, ФІОТ, ІМ-11		

ДОДАТОК Г

Навчальна платформа для розміщення і проходження курсів

Текст програмного коду

ІАЛЦ.467200.007 Д4

Аркушів 44

Київ 2025 р

```

namespace API.Entities;

public class User : IdentityUser
{
    public List<UserSection> UserSections { get; set; }
    public List<UserLesson> UserLessons { get; set; }
    public List<Payment> Payments { get; set; }
}

public class Course
{
    public int Id { get; set; }
    public string Title { get; set; }
    public string Description { get; set; }
    public int PriceFull { get; set; }
    public int PriceMonthly { get; set; }
    public List<Section> Sections { get; set; }
    public string CreatedBy { get; set; }
    public bool IsActive { get; set; }
}

public class Section
{
    public int Id { get; set; }
    public int Number { get; set; } = 1;
    public string Title { get; set; }
    public string Description { get; set; }
    public int CourseId { get; set; }
    public Course Course { get; set; }
    public List<Lesson> Lessons { get; set; }
    public List<UserSection> UserSections { get; set; }
}

public class Lesson
{
    public int Id { get; set; }
    public string Title { get; set; }
    public string Description { get; set; }
    public string UrlTheory { get; set; }
    public string UrlPractice { get; set; }
    public string TheoryTitle { get; set; }
    public string PracticeTitle { get; set; }
    public string TheoryPublicId { get; set; }
    public string PracticePublicId { get; set; }
    public int SectionId { get; set; }
    public Section Section { get; set; }
    public int Number { get; set; }
    public List<UserLesson> UserLessons { get; set; }
    public List<Test> Tests { get; set; }
}

public class Test
{
    public int Id { get; set; }
    public string Question { get; set; }
    public List<Option> Options { get; set; }
    public string Type { get; set; }
    public int LessonId { get; set; }
}

```

					ІАЛЦ.467200.007 Д4			
		№ докум.	Підпис	Дата				
Розробив	Бацак Я. В.				Навчальна платформа для розміщення та проходження курсів Текст програмного коду		Літ.	Аркуш
Перевірив	Гайдай А. Р.							Аркушів
Реценз.	Шимкович В. М.						1	44
Н. Контр.	Нікольський С.С.						КПІ ім. Ігоря Сікорського, ФІОТ, ІМ-11	
Затвердив								

```

        public Lesson Lesson { get; set; }
    }

    public class Option
    {
        public int Id { get; set; }
        public string Text { get; set; }
        public bool isAnswer { get; set; }
        public int TestId { get; set; }
        public Test Test { get; set; }
    }

    public class Payment
    {
        public int Id { get; set; }
        public string UserId { get; set; }
        public User User { get; set; }
        public decimal Amount { get; set; }
        public DateTime PaymentDate { get; set; } = DateTime.UtcNow;
        public PaymentStatus PaymentStatus { get; set; } = PaymentStatus.Pending;
        public string PaymentIntentId { get; set; }
        public string ClientSecret { get; set; }
        public string PurchaseType { get; set; }
        public int PurchaseId { get; set; }
    }

    public enum PaymentStatus
    {
        Pending,
        PaymentReceived,
        PaymentFailed,
    }

    public class UserSection
    {
        public string UserId { get; set; }
        public User User { get; set; }
        public int SectionId { get; set; }
        public Section Section { get; set; }
        public bool isAvailable { get; set; } = false;
    }

    public class UserLesson
    {
        public string UserId { get; set; }
        public User User { get; set; }
        public int LessonId { get; set; }
        public Lesson Lesson { get; set; }
        public bool IsTheoryCompleted { get; set; } = false;
        public bool IsPracticeCompleted { get; set; } = false;
        public float? TestScore { get; set; }
    }

    namespace API.Controllers
    {
        public class AccountController : BaseApiController
        {
            private readonly UserManager<User> _userManager;
            private readonly RoleManager<IdentityRole> _roleManager;
            private readonly TokenService _tokenService;
            public AccountController(UserManager<User> userManager, TokenService
tokenService, RoleManager<IdentityRole> roleManager)
            {
                _tokenService = tokenService;
            }
        }
    }

```

```

        _userManager = userManager;
        _roleManager = roleManager;
    }

    [HttpPost("login")]
    public async Task<ActionResult<UserDto>> Login(LoginDto loginDto)
    {
        var user = await _userManager.FindByNameAsync(loginDto.Username);
        if (user == null || !await _userManager.CheckPasswordAsync(user,
loginDto.Password))
            return Unauthorized();

        var roles = await _userManager.GetRolesAsync(user);
        var userClaims = await _userManager.GetClaimsAsync(user);

        return new UserDto
        {
            Id = user.Id,
            Username = user.UserName,
            Token = await _tokenService.GenerateToken(user),
            Roles = roles.ToList(),
            Claims = userClaims.Select(c => $"{c.Type}: {c.Value}").ToList()
        };
    }

    [HttpPost("register")]
    public async Task<ActionResult> Register(RegisterDto registerDto)
    {
        var user = new User { UserName = registerDto.Username };

        var result = await _userManager.CreateAsync(user, registerDto.Password);

        if (!result.Succeeded)
        {
            foreach (var error in result.Errors)
            {
                ModelState.AddModelError(error.Code, error.Description);
            }

            return ValidationProblem();
        }

        if (registerDto.IsTeacher)
        {
            await DbInitializer.AssignRoleWithClaimsAsync(_userManager, _roleManager,
user, "Teacher");
        }
        else
        {
            await DbInitializer.AssignRoleWithClaimsAsync(_userManager, _roleManager,
user, "Member");
        }

        return StatusCode(201);
    }

    [Authorize]
    [HttpGet("currentUser")]
    public async Task<ActionResult<UserDto>> GetCurrentUser()
    {
        var user = await _userManager.FindByNameAsync(User.Identity.Name);

        var roles = await _userManager.GetRolesAsync(user);
        var userClaims = await _userManager.GetClaimsAsync(user);

```

					ІАЛЦ.467200.007 Д4	Арк.
						3
Зм.	Арк.	№ докум.	Підпис	Дата		

```

        return new UserDto
        {
            Id = user.Id,
            Username = user.UserName,
            Token = await _tokenService.GenerateToken(user),
            Roles = roles.ToList(),
            Claims = userClaims.Select(c => $"{c.Type}: {c.Value}").ToList()
        };
    }
}

namespace API.Services
{
    public class TokenService
    {
        private readonly UserManager<User> _userManager;
        private readonly IConfiguration _config;
        public TokenService(UserManager<User> userManager, IConfiguration config)
        {
            _userManager = userManager;
            _config = config;
        }

        public async Task<string> GenerateToken(User user)
        {
            var claims = new List<Claim>
            {
                new Claim(ClaimTypes.Name, user.UserName),
            };

            var roles = await _userManager.GetRolesAsync(user);
            foreach (var role in roles)
            {
                claims.Add(new Claim(ClaimTypes.Role, role));
            }

            var userClaims = await _userManager.GetClaimsAsync(user);
            foreach (var claim in userClaims)
            {
                claims.Add(new Claim(claim.Type, claim.Value));
            }

            var key = new
SymmetricSecurityKey(Encoding.UTF8.GetBytes(_config["JWTSettings:TokenKey"]));
            var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha512);

            var tokenOptions = new JwtSecurityToken(
                issuer: null,
                audience: null,
                claims: claims,
                expires: DateTime.Now.AddDays(7),
                signingCredentials: creds
            );

            return new JwtSecurityTokenHandler().WriteToken(tokenOptions);
        }
    }
}

namespace API.Controllers
{
    public class CoursesController : BaseApiController

```

					ІАЛЦ.467200.007 Д4	Арк.
						4
Зм.	Арк.	№ докум.	Підпис	Дата		

```

{
    private ICoursesRepository _coursesRepository;
    public CoursesController(ICoursesRepository coursesRepository)
    {
        _coursesRepository = coursesRepository;
    }

    [HttpGet]
    public async Task<ActionResult<List<GetCoursePreviewDto>>> GetCourses()
    {
        var username = User.Identity.Name ?? "";
        var result = await _coursesRepository.GetCourses(username);

        if (!result.IsSuccess)
        {
            return NotFound(result.ErrorMessage);
        }

        return result.Data;
    }

    [HttpGet("preview/{id}")]
    public async Task<ActionResult<GetCoursePreviewDto>> GetCoursePreview(int id)
    {
        var result = await _coursesRepository.GetCoursePreview(id);

        if (!result.IsSuccess)
        {
            return NotFound(result.ErrorMessage);
        }

        return result.Data;
    }

    [HttpGet("{id}")]
    public async Task<ActionResult<GetCourseDto>> GetCourse(int id, [FromQuery]
LessonParams lessonParams)
    {
        var username = User.Identity.Name ?? "";
        var result = await _coursesRepository.GetCourse(id,
lessonParams.MaxImportance, lessonParams.OnlyUncompleted, lessonParams.SearchTerm,
username);

        if (!result.IsSuccess)
        {
            return NotFound(result.ErrorMessage);
        }

        return result.Data;
    }

    [Authorize(Policy = "ManageCourses")]
    [HttpPost]
    public async Task<ActionResult<GetCourseDto>> AddCourse(AddCourseDto newCourse)
    {
        var username = User.Identity.Name ?? "";

        if (newCourse.PriceFull <= 0 || newCourse.PriceMonthly <= 0)
            return BadRequest("Full price and section price must be greater than
zero.");

        var result = await _coursesRepository.AddCourse(newCourse, username);

        if (!result.IsSuccess)

```

					ІАЛЦ.467200.007 Д4	Арк.
						5
Зм.	Арк.	№ докум.	Підпис	Дата		

```

        {
            return BadRequest(result.ErrorMessage);
        }

        return result.Data;
    }

    [Authorize(Policy = "ManageCourses")]
    [HttpPut("{id}")]
    public async Task<ActionResult<GetCourseDto>> UpdateCourse(int id,
UpdateCourseDto updatedCourse)
    {
        var username = User.Identity.Name ?? "";

        if (updatedCourse.PriceFull <= 0 || updatedCourse.PriceMonthly <= 0)
            return BadRequest("Full price and section price must be greater than
zero.");

        var result = await _coursesRepository.UpdateCourse(id, updatedCourse,
username);

        if (!result.IsSuccess)
        {
            if (result.ErrorMessage.Contains("Unauthorized")) return Unauthorized();
            return NotFound(result.ErrorMessage);
        }

        return result.Data;
    }

    [Authorize(Policy = "ManageCourses")]
    [HttpDelete("{id}")]
    public async Task<ActionResult> DeleteCourse(int id)
    {
        var result = await _coursesRepository.DeleteCourse(id);

        if (!result.IsSuccess)
        {
            return NotFound(result.ErrorMessage);
        }

        return Ok();
    }
}

namespace API.Controllers
{
    [Authorize(Policy = "ManageCourses")]
    public class SectionsController : BaseApiController
    {
        private ISectionsRepository _sectionsRepository;
        public SectionsController(ISectionsRepository sectionsRepository)
        {
            _sectionsRepository = sectionsRepository;
        }

        [HttpPut("{id}")]
        public async Task<ActionResult<GetSectionDto>> UpdateSection(int id,
UpdateSectionDto updatedSection)
        {
            var username = User.Identity.Name ?? "";
            var result = await _sectionsRepository.UpdateSection(id, updatedSection,
username);

```

					ІАЛЦ.467200.007 Д4	Арк.
						6
Зм.	Арк.	№ докум.	Підпис	Дата		

```

        if (!result.IsSuccess)
        {
            return NotFound(result.ErrorMessage);
        }

        return result.Data;
    }

    [HttpPost]
    public async Task<ActionResult<GetSectionDto>> AddSection(AddSectionDto
newSection)
    {
        var username = User.Identity.Name ?? "";
        var result = await _sectionsRepository.AddSection(newSection, username);

        if (!result.IsSuccess)
        {
            return BadRequest(result.ErrorMessage);
        }

        return result.Data;
    }

    [HttpDelete("{id}")]
    public async Task<ActionResult> DeleteSection(int id)
    {
        var result = await _sectionsRepository.DeleteSection(id);

        if (!result.IsSuccess)
        {
            return NotFound(result.ErrorMessage);
        }

        return Ok();
    }
}

namespace API.Controllers
{
    [Authorize(Policy = "ManageCourses")]
    public class LessonsController : BaseApiController
    {
        private ILessonsRepository _lessonsRepository;
        public LessonsController(ILessonsRepository lessonsRepository)
        {
            _lessonsRepository = lessonsRepository;
        }

        [HttpPost]
        public async Task<ActionResult<GetLessonDto>> AddLesson([FromForm] AddLessonDto
newLesson)
        {
            var username = User.Identity.Name ?? "";
            var result = await _lessonsRepository.AddLesson(newLesson, username);

            if (!result.IsSuccess)
            {
                return BadRequest(result.ErrorMessage);
            }

            return result.Data;
        }
    }
}

```

					ІАЛЦ.467200.007 Д4	Арк.
						7
Зм.	Арк.	№ докум.	Підпис	Дата		

```

[HttpGet]
public async Task<ActionResult<List<GetLessonDto>>> GetLessons()
{
    var result = await _lessonsRepository.GetLessons();

    if (!result.IsSuccess)
    {
        return NotFound(result.ErrorMessage);
    }

    return result.Data;
}

[HttpGet("{id}")]
public async Task<ActionResult<GetLessonDto>> GetLesson(int id)
{
    var username = User.Identity.Name ?? "";
    var result = await _lessonsRepository.GetLesson(id, username);

    if (!result.IsSuccess)
    {
        return NotFound(result.ErrorMessage);
    }

    return result.Data;
}

[HttpPut("{id}")]
public async Task<ActionResult<GetLessonDto>> UpdateLesson(int id, [FromForm]
UpdateLessonDto updatedLesson)
{
    var username = User.Identity.Name ?? "";
    var result = await _lessonsRepository.UpdateLesson(id, updatedLesson,
username);

    if (!result.IsSuccess)
    {
        if (result.ErrorMessage.Contains("Unauthorized")) return Unauthorized();
        return NotFound(result.ErrorMessage);
    }

    return result.Data;
}

[HttpDelete("{id}")]
public async Task<ActionResult> DeleteLesson(int id)
{
    var result = await _lessonsRepository.DeleteLesson(id);

    if (!result.IsSuccess)
    {
        return NotFound(result.ErrorMessage);
    }

    return Ok();
}
}

namespace API.Controllers
{
    public class TestsController : BaseApiController
    {

```

					ІАЛЦ.467200.007 Д4	Арк.
						8
Зм.	Арк.	№ докум.	Підпис	Дата		

```

private readonly ITestsRepository _testsRepository;
public TestsController(ITestsRepository testsRepository)
{
    _testsRepository = testsRepository;
}

[HttpGet("lessonId/{lessonId}")]
public async Task<ActionResult<List<GetTestDto>>> GetTestsOfLesson(int lessonId)
{
    var username = User.Identity.Name ?? "";
    var result = await _testsRepository.GetTestsOfLesson(lessonId, username);

    if (!result.IsSuccess)
    {
        return NotFound(result.ErrorMessage);
    }

    return result.Data;
}

[HttpPost("statistics")]
public async Task<ActionResult<TestsStatisticDto>>
GetStatisticsOfTests(TestsStatisticParametersDto parameters)
{
    if (parameters.TopPercent < 0 || parameters.TopPercent > 100)
    {
        return BadRequest("Top Percent must be a number between 0 and 100.");
    }

    var username = User.Identity.Name ?? "";
    var result = await _testsRepository.GetStatisticsOfLessonTests(parameters,
username);

    if (!result.IsSuccess)
    {
        return NotFound(result.ErrorMessage);
    }

    return result.Data;
}

[Authorize(Policy = "ManageCourses")]
[HttpPost]
public async Task<ActionResult<GetTestDto>> AddTest(AddTestDto newTest)
{
    var username = User.Identity.Name ?? "";
    var result = await _testsRepository.AddTest(newTest);

    if (!result.IsSuccess)
    {
        return NotFound(result.ErrorMessage);
    }

    return result.Data;
}

[Authorize(Policy = "ManageCourses")]
[HttpPut("{id}")]
public async Task<ActionResult<GetTestDto>> UpdateTest(int id, UpdateTestDto
updatedTest)
{
    var result = await _testsRepository.UpdateTest(id, updatedTest);

    if (!result.IsSuccess)

```

					ІАЛЦ.467200.007 Д4	Арк.
						9
Зм.	Арк.	№ докум.	Підпис	Дата		

```

        {
            return NotFound(result.ErrorMessage);
        }

        return result.Data;
    }

    [Authorize(Policy = "ManageCourses")]
    [HttpDelete("{id}")]
    public async Task<IActionResult> DeleteTest(int id)
    {
        var username = User.Identity.Name ?? "";
        var result = await _testsRepository.DeleteTest(id);

        if (!result.IsSuccess)
        {
            return NotFound(result.ErrorMessage);
        }

        return Ok();
    }
}

public class PaymentsController : BaseApiController
{
    private readonly PaymentService _paymentService;
    private readonly CourseContext _context;
    private readonly IPaymentsRepository _paymentsRepository;
    private readonly IConfiguration _config;
    private readonly ISectionsRepository _sectionsRepository;

    public PaymentsController(PaymentService paymentService, CourseContext context,
        IPaymentsRepository paymentsRepository, IConfiguration config, ISectionsRepository
        sectionsRepository)
    {
        _context = context;
        _paymentService = paymentService;
        _paymentsRepository = paymentsRepository;
        _config = config;
        _sectionsRepository = sectionsRepository;
    }

    [Authorize]
    [HttpPost]
    public async Task<ActionResult<GetPaymentDto>>
    CreateOrUpdatePaymentIntent(CreatePaymentIntentDto paymentIntent)
    {
        var username = User.Identity.Name;
        var existingResult = await _paymentsRepository.GetPaymentAsync(paymentIntent,
        username);
        GetPaymentDto payment;

        if (existingResult.IsSuccess)
        {
            payment = existingResult.Data;
            if (payment.PaymentStatus == PaymentStatus.PaymentReceived) return
            BadRequest(new ProblemDetails { Title = "This payment is already received" });
        }
        else
        {
            var addingResult = await _paymentsRepository.AddPaymentAsync(paymentIntent,
            username);

```

```

        if (!addingResult.IsSuccess)
        {
            return BadRequest(addingResult.ErrorMessage);
        }

        payment = addingResult.Data;
    }

    var intent = await _paymentService.CreateOrUpdatePaymentIntent(payment);

    if (intent == null) return BadRequest(new ProblemDetails { Title = "Problem
creating payment intent" });

    payment.Amount = payment.Amount != 0 ? payment.Amount : intent.Amount / 100;
    payment.PaymentIntentId ??= intent.Id;
    payment.ClientSecret ??= intent.ClientSecret;

    var result = await _paymentsRepository.UpdatePaymentAsync(payment);

    if (!result.IsSuccess)
    {
        return BadRequest(new ProblemDetails { Title = "Problem updating payment with
intent" });
    }

    return result.Data;
}

[Authorize]
[HttpGet]
public async Task<ActionResult<List<GetPaymentDto>>> GetUserPayments()
{
    var username = User.Identity.Name;
    var result = await _paymentsRepository.GetUserPayments(username);

    if (!result.IsSuccess)
    {
        return NotFound(result.ErrorMessage);
    }

    return result.Data;
}

[HttpPost("webhook")]
public async Task<ActionResult> StripeWebhook()
{
    var json = await new StreamReader(HttpContext.Request.Body).ReadToEndAsync();

    var stripeEvent = EventUtility.ConstructEvent(json, Request.Headers["Stripe-
Signature"],
        _config["StripeSettings:WhSecret"]);

    var charge = (Charge)stripeEvent.Data.Object;

    var payment = await _context.Payments.FirstOrDefaultAsync(x =>
        x.PaymentIntentId == charge.PaymentIntentId);

    if (charge.Status == "succeeded")
    {
        payment.PaymentStatus = PaymentStatus.PaymentReceived;

        if (payment.PurchaseType == "Course")
        {

```

					ІАЛЦ.467200.007 Д4	Арк.
						11
Зм.	Арк.	№ докум.	Підпис	Дата		

```

        await _sectionsRepository.MakeCourseAvailable(payment.PurchaseId,
payment.UserId);
    }
    else if (payment.PurchaseType == "Section")
    {
        await _sectionsRepository.MakeSectionAvailable(payment.PurchaseId,
payment.UserId);
    }
}

await _context.SaveChangesAsync();

return new EmptyResult();
}
}

namespace API.Repositories.Implementation
{
    public class CoursesRepository : ICoursesRepository
    {
        private readonly CourseContext _context;
        private readonly IMapper _mapper;
        private readonly UserManager<User> _userManager;
        private readonly ISectionsRepository _sectionsRepository;
        public CoursesRepository(
            CourseContext context,
            UserManager<User> userManager,
            IMapper mapper,
            ISectionsRepository sectionsRepository)
        {
            _userManager = userManager;
            _context = context;
            _mapper = mapper;
            _sectionsRepository = sectionsRepository;
        }

        public async Task<Result<GetCourseDto>> AddCourse(AddCourseDto newCourse, string
username)
        {
            var course = _mapper.Map<Course>(newCourse);
            var user = await _userManager.FindByNameAsync(username);

            course.CreatedBy = user.Id;
            course.Id = _context.Courses.Any() ? _context.Courses.Max(c => c.Id) + 1 : 1;
            await _context.Courses.AddAsync(course);

            return await SaveChangesAndReturnResult(course.Id, username);
        }

        public async Task<Result<GetCourseDto>> UpdateCourse(int id, UpdateCourseDto
updatedCourse, string username)
        {
            var dbCourse = await GetCourseById(id);

            if (dbCourse == null) return new Result<GetCourseDto> { IsSuccess = false,
ErrorMessage = "Lesson with the provided ID not found." };

            dbCourse.Title = updatedCourse.Title ?? "";
            dbCourse.Description = updatedCourse.Description ?? "";
            dbCourse.PriceFull = updatedCourse.PriceFull != -1 ? updatedCourse.PriceFull
: dbCourse.PriceFull;
            dbCourse.PriceMonthly = updatedCourse.PriceMonthly != -1 ?
updatedCourse.PriceMonthly : dbCourse.PriceMonthly;
            dbCourse.IsActive = updatedCourse.IsActive;

```

					ІАЛЦ.467200.007 Д4	Арк.
						12
Зм.	Арк.	№ докум.	Підпис	Дата		

```

        return await SaveChangesAndReturnResult(id, username);
    }

    public async Task<Result<GetCourseDto>> GetCourse(int id, int? maxImportance,
        bool? onlyUncompleted, string searchTerm, string username)
    {
        var user = await _userManager.FindByNameAsync(username);

        try
        {
            var dbCourse = await _context.Courses
                .Include(c => c.Sections).ThenInclude(s => s.UserSections)
                .Include(c => c.Sections).ThenInclude(s =>
                    s.SectionLessons).ThenInclude(sl => sl.Lesson)
                .ThenInclude(l => l.LessonKeywords).ThenInclude(lk => lk.Keyword)
                .Include(c => c.Sections).ThenInclude(s =>
                    s.SectionLessons).ThenInclude(sl => sl.Lesson)
                .ThenInclude(l => l.PreviousLessons).ThenInclude(lp1 =>
                    lp1.PreviousLesson)
                .Include(c => c.Sections).ThenInclude(s =>
                    s.SectionLessons).ThenInclude(sl => sl.Lesson)
                .ThenInclude(l => l.UserLessons)
                .FirstOrDefault(l => l.Id == id);

            var course = _mapper.Map<GetCourseDto>(dbCourse, opts =>
                opts.Items["UserId"] = user?.Id)
                .SortAndRenumberLessons()
                .Filter(maxImportance, onlyUncompleted)
                .Search(searchTerm);

            foreach (var section in course.Sections)
            {
                if (!section.IsAvailable)
                {
                    var lessonPreviews =
                        _mapper.Map<List<GetLessonPreviewDto>>(section.Lessons);
                    var lessons = _mapper.Map<List<GetLessonDto>>(lessonPreviews);
                    section.Lessons = lessons;
                }
            }

            return new Result<GetCourseDto> { IsSuccess = true, Data = course };
        }
        catch (System.Exception)
        {
            return new Result<GetCourseDto> { IsSuccess = false, ErrorMessage =
                "Course with the provided ID not found." };
        }
    }

    public async Task<Result<GetCoursePreviewDto>> GetCoursePreview(int id)
    {
        try
        {
            var dbCourse = await _context.Courses
                .Include(c => c.Sections).ThenInclude(s =>
                    s.SectionLessons).ThenInclude(sl => sl.Lesson)
                .FirstOrDefault(l => l.Id == id);

            var course = _mapper.Map<GetCoursePreviewDto>(dbCourse)
                .SortAndRenumberLessons();
        }
    }

```

```

        return new Result<GetCoursePreviewDto> { IsSuccess = true, Data = course
    };
    }
    catch (System.Exception)
    {
        return new Result<GetCoursePreviewDto> { IsSuccess = false, ErrorMessage
= "Course with the provided ID not found." };
    }
}

public async Task<Result<List<GetCoursePreviewDto>>> GetCourses(string username)
{
    var user = await _userManager.FindByNameAsync(username);

    try
    {
        var dbCourses = await _context.Courses
            .Include(c => c.Sections).ThenInclude(s => s.UserSections)
            .Include(c => c.Sections).ThenInclude(s =>
s.SectionLessons).ThenInclude(sl => sl.Lesson)
            .ToListAsync();

        var courses = dbCourses.Select(c => _mapper.Map<GetCoursePreviewDto>(c,
opts => opts.Items["UserId"] = user?.Id)).ToList();
        return new Result<List<GetCoursePreviewDto>> { IsSuccess = true, Data =
courses };
    }
    catch (System.Exception)
    {
        return new Result<List<GetCoursePreviewDto>> { IsSuccess = false,
ErrorMessage = "Courses not found." };
    }
}
}

namespace API.Controllers
{
    [Authorize(Policy = "ManageCourses")]
    public class OptionsController : BaseApiController
    {
        private readonly IOptionsRepository _optionsRepository;
        public OptionsController(IOptionsRepository optionsRepository)
        {
            _optionsRepository = optionsRepository;
        }

        [HttpPost]
        public async Task<ActionResult<GetOptionDto>> AddOption(AddOptionDto newOption)
        {
            var result = await _optionsRepository.AddOption(newOption);

            if (!result.IsSuccess)
            {
                return NotFound(result.ErrorMessage);
            }

            return result.Data;
        }

        [HttpPut("{id}")]
        public async Task<ActionResult<GetOptionDto>> UpdateOption(int id,
UpdateOptionDto updatedOption)
        {

```

```

        var result = await _optionsRepository.UpdateOption(id, updatedOption);

        if (!result.IsSuccess)
        {
            return NotFound(result.ErrorMessage);
        }

        return result.Data;
    }

    [HttpDelete("{id}")]
    public async Task<IActionResult> DeleteOption(int id)
    {
        var result = await _optionsRepository.DeleteOption(id);

        if (!result.IsSuccess)
        {
            return NotFound(result.ErrorMessage);
        }

        return Ok();
    }
}

namespace API.Repositories.Implementation
{
    public class LessonsRepository : ILessonsRepository
    {
        private readonly CourseContext _context;
        private readonly IMapper _mapper;
        private readonly VideoService _videoService;
        private readonly UserManager<User> _userManager;
        public LessonsRepository(
            CourseContext context,
            UserManager<User> userManager,
            IMapper mapper,
            VideoService videoService)
        {
            _userManager = userManager;
            _context = context;
            _mapper = mapper;
            _videoService = videoService;
        }

        public async Task<Result<List<GetLessonDto>>> GetLessons()
        {
            var dbLessons = _context.Lessons
                .Include(l => l.LessonKeywords).ThenInclude(lk => lk.Keyword)
                .Include(l => l.PreviousLessons).ThenInclude(lpl => lpl.PreviousLesson);

            var lessons = await dbLessons.Select(l =>
                _mapper.Map<GetLessonDto>(l)).ToListAsync();
            return new Result<List<GetLessonDto>> { IsSuccess = true, Data = lessons };
        }

        public async Task<Result<GetLessonDto>> GetLesson(int id, string username)
        {
            var user = await _userManager.FindByNameAsync(username);

            var dbLesson = await _context.Lessons
                .Include(l => l.LessonKeywords).ThenInclude(lk => lk.Keyword)
                .Include(l => l.PreviousLessons).ThenInclude(lpl => lpl.PreviousLesson)
                .Include(l => l.UserLessons)

```

```

        .FirstOrDefaultAsync(l => l.Id == id);

        if (dbLesson == null) return NotFoundResult("Lesson with the provided ID not
found.");

        var lesson = _mapper.Map<GetLessonDto>(dbLesson, opts => opts.Items["UserId"]
= user?.Id);
        return new Result<GetLessonDto> { IsSuccess = true, Data = lesson };
    }

    public async Task<Result<GetLessonDto>> AddLesson(AddLessonDto newLesson, string
username)
    {
        var lesson = _mapper.Map<Lesson>(newLesson);

        var theoryResult = await ProcessVideoForLesson(newLesson.TheoryFile, null,
(url, id) => { lesson.UrlTheory = url; lesson.TheoryPublicId = id; });
        if (theoryResult != null) return theoryResult;

        var practiceResult = await ProcessVideoForLesson(newLesson.PracticeFile,
null, (url, id) => { lesson.UrlPractice = url; lesson.PracticePublicId = id; });
        if (practiceResult != null) return practiceResult;

        lesson.Id = _context.Lessons.Any() ? _context.Lessons.Max(c => c.Id) + 1 : 1;
        await _context.Lessons.AddAsync(lesson);

        return await SaveChangesAndReturnResult(lesson.Id, username);
    }

    public async Task<Result<GetLessonDto>> UpdateLesson(int id, UpdateLessonDto
updatedLesson, string username)
    {
        var dbLesson = await GetLessonById(id);

        if (dbLesson == null) return NotFoundResult("Lesson with the provided ID not
found.");

        UpdateLessonDetails(dbLesson, updatedLesson);

        var theoryResult = await ProcessVideoForLesson(updatedLesson.TheoryFile,
dbLesson.TheoryPublicId, (url, id) => { dbLesson.UrlTheory = url; dbLesson.TheoryPublicId
= id; });
        if (theoryResult != null) return theoryResult;

        var practiceResult = await ProcessVideoForLesson(updatedLesson.PracticeFile,
dbLesson.PracticePublicId, (url, id) => { dbLesson.UrlPractice = url;
dbLesson.PracticePublicId = id; });
        if (practiceResult != null) return practiceResult;

        return await SaveChangesAndReturnResult(id, username);
    }

    public async Task<Result<GetLessonDto>> UpdateLessonCompletion(int lessonId,
UpdateUserLessonDto updatedUserLesson, GetCourseDto course, string username)
    {
        var user = await GetUser(username);
        if (user == null) return UnauthorizedResult("Unauthorized user.");

        if (!await UserIsAdmin(user))
        {
            var lessonAvailabilityResult = CheckLessonAvailability(lessonId,
updatedUserLesson, course, username);
            if (lessonAvailabilityResult != null)
            {

```

```

        return lessonAvailabilityResult;
    }
}

var dbLesson = await GetLessonById(lessonId);
if (dbLesson == null) return NotFoundResult("Lesson with the provided ID not
found.");

var userLesson = await GetUserLesson(lessonId, user.Id);
if (userLesson == null)
{
    CreateUserLesson(lessonId, user.Id, updatedUserLesson);
}
else
{
    UpdateExistingUserLesson(userLesson, updatedUserLesson);
}

return await SaveChangesAndReturnResult(lessonId, username);
}

public async Task<Result<bool>> DeleteLesson(int id)
{
    var dbLesson = await GetLessonById(id);
    if (dbLesson == null)
        return new Result<bool> { IsSuccess = false, ErrorMessage = "Lesson with
the provided ID not found." };

    if (!string.IsNullOrEmpty(dbLesson.TheoryPublicId))
        await _videoService.DeleteVideoAsync(dbLesson.TheoryPublicId);

    if (!string.IsNullOrEmpty(dbLesson.PracticePublicId))
        await _videoService.DeleteVideoAsync(dbLesson.PracticePublicId);

    _context.Lessons.Remove(dbLesson);
    await _context.SaveChangesAsync();

    return new Result<bool> { IsSuccess = true, Data = true };
}

private async Task<Result<GetLessonDto>> ProcessVideoForLesson(IFormFile file,
string existingPublicId, Action<string, string> updateLesson)
{
    if (file != null)
    {
        if (!string.IsNullOrEmpty(existingPublicId))
            await _videoService.DeleteVideoAsync(existingPublicId);

        var videoResult = await _videoService.AddVideoAsync(file);

        if (videoResult.Error != null)
            return new Result<GetLessonDto> { IsSuccess = false, ErrorMessage =
videoResult.Error.Message };

        updateLesson(videoResult.SecureUrl.ToString(), videoResult.PublicId);
    }
    return null;
}

}

public class TestsRepository : ITestsRepository
{
    private readonly CourseContext _context;

```

```

private readonly IMapper _mapper;
private readonly ImageService _imageService;
private readonly UserManager<User> _userManager;
private readonly IOptionsRepository _optionsRepository;
public TestsRepository(
    CourseContext context,
    UserManager<User> userManager,
    IMapper mapper,
    ImageService imageService,
    IOptionsRepository optionsRepository)
{
    _context = context;
    _mapper = mapper;
    _userManager = userManager;
    _imageService = imageService;
    _optionsRepository = optionsRepository;
}

public async Task<Result<List<GetTestDto>>> GetTestsOfLesson(int lessonId, string
username)
{
    var user = await _userManager.FindByNameAsync(username);
    if (user == null) return new Result<List<GetTestDto>> { IsSuccess = false,
ErrorMessage = "Unauthorized user." };

    var dbLesson = await _context.Lessons
        .FirstOrDefaultAsync(l => l.Id == lessonId);

    if (dbLesson == null) return new Result<List<GetTestDto>> { IsSuccess = false,
ErrorMessage = "Lesson with the provided ID not found." };

    var tests = await _context.Tests
        .Where(t => t.Lesson.Id == lessonId)
        .Include(t => t.Options)
        .Select(t => _mapper.Map<GetTestDto>(t))
        .ToListAsync();

    return new Result<List<GetTestDto>> { IsSuccess = true, Data = tests };
}

public async Task<Result<TestsStatisticDto>>
GetStatisticsOfLessonTests(TestsStatisticParametersDto parameters, string username)
{
    var user = await _userManager.FindByNameAsync(username);
    if (user == null) return new Result<TestsStatisticDto> { IsSuccess = false,
ErrorMessage = "Unauthorized user." };

    var lessonId = parameters.LessonId;
    var topPercent = parameters.TopPercent;

    var dbLesson = await _context.Lessons
        .FirstOrDefaultAsync(l => l.Id == lessonId);

    if (dbLesson == null) return new Result<TestsStatisticDto> { IsSuccess = false,
ErrorMessage = "Lesson with the provided ID not found." };

    var userLessons = await _context.UserLessons
        .Where(ul => ul.Lesson.Id == lessonId && ul.TestScore != null)
        .ToListAsync();

    var isScoreHigherAverage = IsScoreHigherThanAverage(userLessons, user.Id);
    var isUserInTop = IsUserInTopPercent(userLessons, topPercent, user.Id);

    var testsStatistics = new TestsStatisticDto

```

```

        {
            IsScoreHigherThanAverage = isScoreHigherAverage,
            IsUserInTopPercent = isUserInTop,
        };

        return new Result<TestsStatisticDto> { IsSuccess = true, Data = testsStatistics
    };
}

public async Task<Result<GetTestDto>> AddTest(AddTestDto newTest)
{
    var test = _mapper.Map<Test>(newTest);

    if (newTest.File != null)
    {
        var result = await _imageService.ProcessImageAsync(newTest.File, null, (url,
id) => { test.ImgUrl = url; test.PublicId = id; });
        if (!result.IsSuccess) return new Result<GetTestDto> { IsSuccess = false,
ErrorMessage = result.ErrorMessage }; ;
    }

    test.Id = _context.Tests.Any() ? _context.Tests.Max(c => c.Id) + 1 : 1;
    await _context.Tests.AddAsync(test);

    return await SaveChangesAndReturnResult(test.Id);
}

public async Task<Result<GetTestDto>> UpdateTest(int id, UpdateTestDto updatedTest)
{
    var dbTest = await _context.Tests.FirstOrDefaultAsync(t => t.Id == id);

    if (dbTest == null) return new Result<GetTestDto> { IsSuccess = false,
ErrorMessage = "Test with the provided ID not found." };

    UpdateTestDetails(dbTest, updatedTest);

    if (updatedTest.File != null)
    {
        var result = await _imageService.ProcessImageAsync(updatedTest.File,
dbTest.PublicId, (url, id) => { dbTest.ImgUrl = url; dbTest.PublicId = id; });
        if (!result.IsSuccess) return new Result<GetTestDto> { IsSuccess = false,
ErrorMessage = result.ErrorMessage };
    }

    if (updatedTest.OptionIdsToAdd != null)
    {
        foreach (var optionIdToAdd in updatedTest.OptionIdsToAdd)
        {
            var option = await _context.Options.FindAsync(optionIdToAdd);
            if (option != null)
            {
                option.TestId = id;
                _context.Options.Update(option);
            }
        }
    }

    if (updatedTest.OptionIdsToDelete != null)
    {
        foreach (var optionIdToDelete in updatedTest.OptionIdsToDelete)
        {
            var option = await _context.Options.FirstOrDefaultAsync(o => o.Id ==
optionIdToDelete && o.TestId == id);
            if (option != null)

```

```

        await _optionsRepository.DeleteOption(optionIdToDelete);
    }
}

return await SaveChangesAndReturnResult(id);
}

public async Task<Result<bool>> DeleteTest(int id)
{
    try
    {
        var dbTest = await _context.Tests.FirstOrDefaultAsync(t => t.Id == id);

        if (dbTest == null) return new Result<bool> { IsSuccess = false, ErrorMessage
= "Test with the provided ID not found." };

        if (!string.IsNullOrEmpty(dbTest.PublicId))
            await _imageService.DeleteImageAsync(dbTest.PublicId);

        var resultOfDeletingOptions = await
_optionsRepository.DeleteAllOptionsOfTest(id);

        if (!resultOfDeletingOptions.IsSuccess) return resultOfDeletingOptions;

        _context.Tests.Remove(dbTest);
        await _context.SaveChangesAsync();

        return new Result<bool> { IsSuccess = true, Data = true };
    }
    catch (System.Exception ex)
    {
        return new Result<bool> { IsSuccess = false, ErrorMessage = ex.Message };
    }
}

private bool? IsScoreHigherThanAverage(List<UserLesson> userLessons, string userId)
{
    if (!userLessons.Any(ul => ul.UserId != userId))
    {
        return null;
    }

    var otherScores = userLessons
        .Where(ul => ul.UserId != userId && ul.TestScore.HasValue)
        .Select(ul => ul.TestScore.Value)
        .ToList();

    var average = CalculateAverageScore(otherScores);
    var userScore = userLessons.First(ul => ul.UserId == userId).TestScore;

    return userScore >= average;
}

private float CalculateAverageScore(List<float> scores)
{
    float total = scores.Sum();
    float average = total / scores.Count;

    return average;
}

private bool? IsUserInTopPercent(List<UserLesson> userLessons, int topPercent, string
userId)
{

```

					ІАЛЦ.467200.007 Д4	Арк.
						20
Зм.	Арк.	№ докум.	Підпис	Дата		

```

        if (!userLessons.Any(ul => ul.UserId != userId))
        {
            return null;
        }

        var scores = userLessons
            .Where(ul => ul.TestScore.HasValue)
            .Select(ul => ul.TestScore.Value)
            .ToList();

        var threshold = CalculateTopPercentThreshold(scores, topPercent);
        var userScore = userLessons.First(ul => ul.UserId == userId).TestScore;

        return userScore >= threshold;
    }

    private float CalculateTopPercentThreshold(List<float> scores, int percent)
    {
        scores.Sort();

        int totalCount = scores.Count;
        int topCount = Math.Max(1, (int)Math.Ceiling(totalCount * percent / 100.0));

        float topPercentThreshold = scores[totalCount - topCount];
        return topPercentThreshold;
    }

    private void UpdateTestDetails(Test dbTest, UpdateTestDto updatedTest)
    {
        dbTest.Question = updatedTest.Question ?? "";
        dbTest.Type = updatedTest.Type ?? dbTest.Type;
    }
}

namespace API.Repositories.Implementation
{
    public class OptionsRepository : IOptionsRepository
    {
        private CourseContext _context;
        private IMapper _mapper;
        private readonly UserManager<User> _userManager;
        private readonly ImageService _imageService;
        public OptionsRepository(
            CourseContext context,
            UserManager<User> userManager,
            IMapper mapper,
            ImageService imageService)
        {
            _context = context;
            _mapper = mapper;
            _userManager = userManager;
            _imageService = imageService;
        }

        public async Task<Result<GetOptionDto>> AddOption(AddOptionDto newOption)
        {
            var option = _mapper.Map<Option>(newOption);

            if (newOption.File != null)
            {
                var result = await _imageService.ProcessImageAsync(newOption.File, null,
                    (url, id) => { option.ImgUrl = url; option.PublicId = id; });
                if (!result.IsSuccess) return new Result<GetOptionDto> { IsSuccess =
                    false, ErrorMessage = result.ErrorMessage }; ;
            }
        }
    }
}

```

```

    }

    option.Id = _context.Options.Any() ? _context.Options.Max(c => c.Id) + 1 : 1;

    await _context.Options.AddAsync(option);

    return await SaveChangesAndReturnResult(option.Id);
}

public async Task<Result<GetOptionDto>> UpdateOption(int id, UpdateOptionDto
updatedOption)
{
    var dbOption = await _context.Options.FirstOrDefaultAsync(t => t.Id == id);

    if (dbOption == null) return new Result<GetOptionDto> { IsSuccess = false,
ErrorMessage = "Option with the provided ID not found." };

    UpdateOptionDetails(dbOption, updatedOption);

    if (updatedOption.File != null)
    {
        var result = await _imageService.ProcessImageAsync(updatedOption.File,
dbOption.PublicId, (url, id) => { dbOption.ImgUrl = url; dbOption.PublicId = id; });
        if (!result.IsSuccess) return new Result<GetOptionDto> { IsSuccess =
false, ErrorMessage = result.ErrorMessage };
    }

    return await SaveChangesAndReturnResult(id);
}

public async Task<Result<bool>> DeleteOption(int id)
{
    try
    {
        var dbOption = await _context.Options.FirstOrDefaultAsync(o => o.Id ==
id);

        if (dbOption == null) return new Result<bool> { IsSuccess = false,
ErrorMessage = "Option with the provided ID not found." };

        if (!string.IsNullOrEmpty(dbOption.PublicId))
            await _imageService.DeleteImageAsync(dbOption.PublicId);

        _context.Options.Remove(dbOption);
        await _context.SaveChangesAsync();

        return new Result<bool> { IsSuccess = true, Data = true };
    }
    catch (System.Exception ex)
    {
        return new Result<bool> { IsSuccess = false, ErrorMessage = ex.Message };
    }
}

public async Task<Result<bool>> DeleteAllOptionsOfTest(int testId)
{
    var dbOptions = _context.Options
        .Where(o => o.TestId == testId);

    foreach (var option in dbOptions)
    {
        var result = await DeleteOption(option.Id);

        if (!result.IsSuccess) return result;
    }
}

```

```

    }

    return new Result<bool> { IsSuccess = true, Data = true };
}

private void UpdateOptionDetails(Option dbOption, UpdateOptionDto updatedOption)
{
    dbOption.Text = updatedOption.Text ?? "";
    dbOption.isAnswer = updatedOption.isAnswer != -1 ? (updatedOption.isAnswer !=
0) : dbOption.isAnswer;
}
}
}

namespace API.Repositories.Implementation
{
    public class PaymentsRepository : IPaymentsRepository
    {
        private CourseContext _context;
        private readonly UserManager<User> _userManager;
        public PaymentsRepository(
            CourseContext context,
            UserManager<User> userManager
        )
        {
            _context = context;
            _userManager = userManager;
        }

        public async Task<Result<List<GetPaymentDto>>> GetUserPayments(string username)
        {
            var user = await _userManager.FindByNameAsync(username);

            var payments = await _context.Payments
                .Where(p => p.UserId == user.Id)
                .Select(p => MapToPaymentDto(p))
                .ToListAsync();

            return new Result<List<GetPaymentDto>> { IsSuccess = true, Data = payments };
        }

        public async Task<Result<GetPaymentDto>> AddPaymentAsync(CreatePaymentIntentDto
paymentIntent, string username)
        {
            var user = await _userManager.FindByNameAsync(username);

            var payment = MapToPayment(paymentIntent, user.Id);
            payment.Id = _context.Payments.Any() ? _context.Payments.Max(p => p.Id) + 1 :
1;
            await _context.Payments.AddAsync(payment);

            return await SaveChangesAndReturnResult(payment.Id);
        }

        public async Task<Result<GetPaymentDto>> GetPaymentAsync(CreatePaymentIntentDto
paymentIntent, string username)
        {
            var user = await _userManager.FindByNameAsync(username);

            var payment = await _context.Payments
                .Where(p => p.UserId == user.Id
                    && p.PurchaseType == paymentIntent.PurchaseType
                    && p.PurchaseId == paymentIntent.PurchaseId)
                .FirstOrDefaultAsync();

```

					ІАЛЦ.467200.007 Д4	Арк.
						23
Зм.	Арк.	№ докум.	Підпис	Дата		

```

        if (payment == null) return new Result<GetPaymentDto> { IsSuccess = false };

        return new Result<GetPaymentDto> { IsSuccess = true, Data =
MapToPaymentDto(payment) };
    }

    public async Task<Result<GetPaymentDto>> UpdatePaymentAsync(GetPaymentDto
paymentDto)
    {
        var dbPayment = await _context.Payments.FirstOrDefaultAsync(p => p.Id ==
paymentDto.Id);

        dbPayment.Amount = paymentDto.Amount;
        dbPayment.PaymentIntentId = paymentDto.PaymentIntentId;
        dbPayment.ClientSecret = paymentDto.ClientSecret;
        dbPayment.PaymentStatus = paymentDto.PaymentStatus;

        return await SaveChangesAndReturnResult(paymentDto.Id);
    }

    private Payment MapToPayment(CreatePaymentIntentDto paymentIntentDto, string
userId)
    {
        return new Payment
        {
            UserId = userId,
            PurchaseType = paymentIntentDto.PurchaseType,
            PurchaseId = paymentIntentDto.PurchaseId,
        };
    }

    private Payment MapToPayment(GetPaymentDto paymentDto)
    {
        return new Payment
        {
            Id = paymentDto.Id,
            UserId = paymentDto.UserId,
            Amount = paymentDto.Amount,
            PaymentDate = paymentDto.PaymentDate,
            PaymentStatus = paymentDto.PaymentStatus,
            PaymentIntentId = paymentDto.PaymentIntentId,
            ClientSecret = paymentDto.ClientSecret,
            PurchaseType = paymentDto.PurchaseType,
            PurchaseId = paymentDto.PurchaseId
        };
    }

    private async Task<Result<GetPaymentDto>> SaveChangesAndReturnResult(int id)
    {
        try
        {
            await _context.SaveChangesAsync();

            var dbPayment = await _context.Payments
                .FirstOrDefaultAsync(p => p.Id == id);

            return new Result<GetPaymentDto> { IsSuccess = true, Data =
MapToPaymentDto(dbPayment) };
        }
        catch (System.Exception ex)
        {
            return new Result<GetPaymentDto> { IsSuccess = false, ErrorMessage =
ex.Message };
        }
    }

```

					ІАЛЦ.467200.007 Д4	Арк.
						24
Зм.	Арк.	№ докум.	Підпис	Дата		

```

    }
}

private static GetPaymentDto MapToPaymentDto(Payment payment)
{
    return new GetPaymentDto
    {
        Id = payment.Id,
        UserId = payment.UserId,
        Amount = payment.Amount,
        PaymentDate = payment.PaymentDate,
        PaymentStatus = payment.PaymentStatus,
        PaymentIntentId = payment.PaymentIntentId,
        ClientSecret = payment.ClientSecret,
        PurchaseType = payment.PurchaseType,
        PurchaseId = payment.PurchaseId
    };
}
}
}

public class VideoService
{
    private readonly Cloudinary _cloudinary;
    public VideoService(IConfiguration config)
    {
        var acc = new Account
        (
            config["Cloudinary:CloudName"],
            config["Cloudinary:ApiKey"],
            config["Cloudinary:ApiSecret"]
        );

        _cloudinary = new Cloudinary(acc);
    }

    public async Task<VideoUploadResult> AddVideoAsync(IFormFile file)
    {
        var uploadResult = new VideoUploadResult();

        if (file.Length > 0)
        {
            using var stream = file.OpenReadStream();
            var uploadParams = new VideoUploadParams
            {
                File = new FileDescription(file.FileName, stream)
            };
            uploadResult = await _cloudinary.UploadAsync(uploadParams);
        }

        return uploadResult;
    }

    public async Task<DeletionResult> DeleteVideoAsync(string publicId)
    {
        var deleteParams = new DeletionParams(publicId);

        var result = await _cloudinary.DestroyAsync(deleteParams);

        return result;
    }
}

namespace API

```

					ІАЛЦ.467200.007 Д4	Арк.
						25
Зм.	Арк.	№ докум.	Підпис	Дата		

```

{
    public class AutoMapperProfile : Profile
    {
        public AutoMapperProfile()
        {
            CreateMap<Lesson, GetLessonDto>()
                .ForMember(dto => dto.Keywords, l => l.MapFrom(l =>
1.LessonKeywords.Select(lk => lk.Keyword)))
                .ForMember(dto => dto.PreviousLessons, l => l.MapFrom(l =>
1.PreviousLessons.Select(lp1 => lp1.PreviousLesson)))
                .AfterMap((src, dest, context) =>
                {
                    var userId = context.Items["UserId"] as string;
                    var userLesson = src.UserLessons.FirstOrDefault(ul => ul.UserId ==
userId);

                    if (userLesson != null)
                    {
                        dest.IsTheoryCompleted = userLesson.IsTheoryCompleted;
                        dest.IsPracticeCompleted = userLesson.IsPracticeCompleted;
                        dest.TestScore = userLesson.TestScore;
                    }
                });
            CreateMap<Lesson, GetLessonPreviewDto>();
            CreateMap<AddLessonDto, Lesson>();
            CreateMap<UpdateLessonDto, Lesson>();
            CreateMap<Lesson, GetPreviousLessonDto>();
            CreateMap<GetLessonDto, GetLessonPreviewDto>();
            CreateMap<GetLessonPreviewDto, GetLessonDto>();

            CreateMap<Keyword, GetKeywordDto>();
            CreateMap<AddKeywordDto, Keyword>();

            CreateMap<AddLessonKeywordDto, LessonKeyword>();
            CreateMap<LessonKeyword, AddLessonKeywordDto>();

            CreateMap<Test, GetTestDto>()
                .ForMember(dto => dto.Options, t => t.MapFrom(t => t.Options));
            CreateMap<Option, GetOptionDto>();

            CreateMap<AddTestDto, Test>();
            CreateMap<AddOptionDto, Option>();

            CreateMap<Section, GetSectionDto>()
                .ForMember(dto => dto.Lessons, opt => opt.MapFrom(s =>
s.SectionLessons.Select(sl => sl.Lesson)))
                .ForMember(dto => dto.IsAvailable, opt => opt.MapFrom((src, _, _,
context) =>
                {
                    var userId = context.Items["UserId"] as string;
                    var userSection = src.UserSections.FirstOrDefault(us => us.UserId ==
userId);

                    return userSection?.IsAvailable ?? false;
                }));
            CreateMap<Section, GetSectionPreviewDto>()
                .ForMember(dto => dto.Lessons, opt => opt.MapFrom(s =>
s.SectionLessons.Select(sl => sl.Lesson)))
                .ForMember(dto => dto.IsAvailable, opt => opt.MapFrom((src, _, _,
context) =>
                {
                    var userId = context.Items["UserId"] as string;
                    var userSection = src.UserSections.FirstOrDefault(us => us.UserId ==
userId);

                    return userSection?.IsAvailable ?? false;
                }));
        }
    }
}

```

```

        CreateMap<AddSectionDto, Section>();

        CreateMap<Course, GetCourseDto>()
            .ForMember(dto => dto.Sections, opt => opt.MapFrom(c => c.Sections));
        CreateMap<Course, GetCoursePreviewDto>()
            .ForMember(dto => dto.Sections, opt => opt.MapFrom(c => c.Sections));
        CreateMap<AddCourseDto, Course>();
    }
}

export interface User {
    id: string;
    email: string;
    username: string;
    token: string;
    roles?: string[];
    claims?: string[];
}

export interface Course {
    id: number
    title: string
    description: string
    priceFull: number
    priceMonthly: number
    isActive: boolean
    numberOfParticipants: number
    earned: number
    createdBy: string
    sections: Section[]
}

export interface Section {
    id: number
    courseId: number
    number: number
    title: string
    description: string
    isAvailable: boolean
    lessons: Lesson[]
}

export interface CoursePreview {
    id: number
    title: string
    description: string
    duration: number
    priceFull: number
    priceMonthly: number
    sections: SectionPreview[]
}

export interface SectionPreview {
    id: number
    courseId: number
    number: number
    title: string
    description: string
    isAvailable: number
    lessons: LessonPreview[]
}

export interface Lesson {

```

```

    id: number
    title: string
    description: string
    urlTheory: string
    urlPractice: string
    number: number
    importance: number
    testScore: number | null;
    isTheoryCompleted: boolean
    isPracticeCompleted: boolean
  }

export interface LessonPreview {
  id: number
  title: string
  description: string
}

export interface Test {
  id: number
  question: string
  type: string
  imgUrl: string
  options: Option[]
}

export interface Option {
  id: number
  text: string
  imgUrl: string
  isAnswer: boolean
}

export interface UserStatistic {
  isScoreHigherThanAverage: boolean
  isUserInTopPercent: boolean
}

export interface Payment {
  id: number
  userId: string
  amount: number
  paymentDate: string
  paymentStatus: number
  paymentIntentId: string
  clientSecret: string
  purchaseType: string
  purchaseId: number
}

export interface PaymentPayload {
  purchaseType: string
  purchaseId: number
}

export const router = createBrowserRouter([
  {
    path: '/',
    element: <App />,
    children: [
      // authenticated routes
      {
        element: <RequireAuth />, children: [

```

```

        { path: 'course/:courseId/lesson/:lessonId', element: <LessonDetails
/> },
        { path: 'course/:courseId/lesson/:lessonId/test', element:
<TestsSpace /> },
        { path: 'checkout/:courseId', element: <CheckoutWrapper /> },
      ]
    },
    // admin routes
    {
      element: <RequireAuth claims={['Permission: ManageCourses']} />,
children: [
      { path: 'editor', element: <CourseEditor /> },
    ]
  },
  { path: '', element: <Navigate to='/course' replace /> },
  { path: 'course', element: <CourseCatalog /> },
  { path: 'course/:courseId', element: <CourseDetails /> },
  { path: 'register', element: <Register /> },
  { path: 'login', element: <Login /> },
  { path: 'server-error', element: <ServerError /> },
  { path: 'not-found', element: <NotFound /> },
  { path: '*', element: <Navigate replace to='not-found' /> },
]
}
})

interface Props {
  claims?: string[];
}

export default function RequireAuth({ claims }: Props) {
  const { user } = useAppSelector(state => state.account);
  const location = useLocation();

  if (!user) {
    return <Navigate to='/login' state={{ from: location }} />
  }

  if (claims && !claims.some(claim => user.claims?.includes(claim))) {
    toast.error('Not authorised to access this area')
    return <Navigate to='/course' />
  }

  return <Outlet />
}

const requests = {
  get: (url: string, params?: URLSearchParams) =>
    axios.get(url, { params }).then(responseBody),
  post: (url: string, body: object) => axios.post(url, body).then(responseBody),
  put: (url: string, body: object) => axios.put(url, body).then(responseBody),
  delete: (url: string) => axios.delete(url).then(responseBody),
  postForm: (url: string, data: FormData) =>
    axios
      .post(url, data, {
        headers: { 'Content-type': 'multipart/form-data' }
      })
      .then(responseBody),
  putForm: (url: string, data: FormData) =>
    axios
      .put(url, data, {
        headers: { 'Content-type': 'multipart/form-data' }
      })
      .then(responseBody)
}

```

```

}

const createFormData = (item: any) => {
  const formData = new FormData()
  for (const key in item) {
    formData.append(key, item[key])
  }
  return formData
}

const Course = {
  list: () => requests.get(`courses`),
  preview: (id: number) => requests.get(`courses/preview/${id}`),
  details: (id: number, params: URLSearchParams) =>
    requests.get(`courses/${id}`, params),
  create: (body: any) => requests.post('courses', body),
  update: (id: number, body: any) => requests.put(`courses/${id}`, body),
  delete: (id: number) => requests.delete(`courses/${id}`)
}

const Section = {
  create: (body: any) => requests.post('sections', body),
  update: (id: number, body: any) => requests.put(`sections/${id}`, body),
  delete: (id: number) => requests.delete(`sections/${id}`)
}

const Lesson = {
  updateCompletion: (id: number, body: object) =>
    requests.put(`userlessons/${id}`, body),
  create: (lesson: any) => requests.postForm('lessons', createFormData(lesson)),
  update: (id: number, lesson: any) =>
    requests.putForm(`lessons/${id}`, createFormData(lesson))
  // delete: (id: number) => requests.delete(`courses/${id}`)
}

const Test = {
  details: (lessonId: number) => requests.get(`tests/lessonId/${lessonId}`),
  getUserStatistic: (body: any) => requests.post('tests/statistics', body),
  create: (body: any) => requests.post('tests', body),
  update: (id: number, body: any) => requests.put(`tests/${id}`, body),
  delete: (id: number) => requests.delete(`tests/${id}`)
}

const Account = {
  login: (values: any) => requests.post('account/login', values),
  register: (values: any) => requests.post('account/register', values),
  currentUser: () => requests.get('account/currentUser')
}

const Payments = {
  getAll: () => requests.get('payments'),
  createOrUpdatePaymentIntent: (values: any) =>
    requests.post('payments', values)
}

const agent = {
  Course,
  Lesson,
  Test,
  Account,
  Section,
  Payments
}

```

```

export default agent

interface UseCoursesParams {
  onlyActive?: boolean;
  onlyEditableByUser?: boolean;
}

export default function useCourses({ onlyActive = false, onlyEditableByUser = false}:
UseCoursesParams = {}) {
  const allCourses = useAppSelector(courseSelectors.selectAll);
  const { user } = useAppSelector(state => state.account);
  const { coursesLoaded, status } = useAppSelector(state => state.courses);
  const dispatch = useAppDispatch();

  useEffect(() => {
    if (!coursesLoaded) dispatch(fetchCoursesAsync())
  }, [coursesLoaded, dispatch])

  const filteredCourses = useMemo(() => {
    return allCourses.filter((course: Course) => {
      if (onlyActive && !course.isActive) return false;

      if (onlyEditableByUser) {
        if (!user) return false;
        const isCreator = course.createdBy === user.id;
        const isAdmin = user.claims?.includes('Permission: AdminAccess');
        return isCreator || isAdmin;
      }

      return true;
    });
  }, [allCourses, onlyActive, onlyEditableByUser, user]);

  return {
    courses: filteredCourses,
    coursesLoaded,
    status
  }
}

export const store = configureStore({
  reducer: {
    courses: coursesSlice.reducer,
    account: accountSlice.reducer,
    tests: testsSlice.reducer,
    payments: paymentSlice,
  }
})

export type RootState = ReturnType<typeof store.getState>
export type AppDispatch = typeof store.dispatch

export const useAppDispatch = () => useDispatch<AppDispatch>()
export const useAppSelector: TypedUseSelectorHook<RootState> = useSelector

export default function Login() {
  const navigate = useNavigate();
  const location = useLocation();
  const dispatch = useAppDispatch();
  const { register, handleSubmit, formState: { isSubmitting, errors, isValid } } =
useForm({
  mode: 'onTouched',
});
}

```

```

const submitForm = async (data: FieldValues) => {
  try {
    dispatch(clearCourses());
    dispatch(clearTests());
    await dispatch(singInUser(data));
    navigate(location.state?.from || '/course');
  } catch (error: any) {
    console.log(error);
  }
}

return (
  <Container component={Paper} maxWidth="xs" sx={{ display: 'flex', flexDirection:
'column', alignItems: 'center', p: 4 }}>
    <CssBaseline />
    <Avatar sx={{ m: 1, bgcolor: 'secondary.main' }}>
      <LockOutlinedIcon />
    </Avatar>
    <Typography component="h1" variant="h5">
      Вхід
    </Typography>
    <Box component="form" onSubmit={handleSubmit(submitForm)} noValidate sx={{
mt: 1 }}>
      <TextField
        margin="normal"
        fullWidth
        label="Нік"
        autoComplete="username"
        autoFocus
        {...register('username', { required: 'Будь ласка, вкажіть нік' })}
        error={!!errors.username}
        helperText={errors?.username?.message as string}
      />
      <TextField
        margin="normal"
        fullWidth
        label="Пароль"
        type="password"
        autoComplete="current-password"
        {...register('password', { required: 'Будь ласка, вкажіть пароль' })}
        error={!!errors.password}
        helperText={errors?.password?.message as string}
      />
      <LoadingButton
        loading={isSubmitting}
        disabled={!isValid}
        type="submit"
        fullWidth
        variant="contained"
        sx={{ mt: 3, mb: 2 }}
      >
        Увійти
      </LoadingButton>
      <Grid container>
        <Grid item>
          <Link to="/register">
            {"Ще не маєте акаунта? Зареєструватися"}
          </Link>
        </Grid>
      </Grid>
    </Box>
  </Container>
);
}

```

```

function splitCourses(courses: Course[], user?: User | null): { title: string, courses: Course[] }[] {

    const ownCourses: Course[] = [];
    const myCourses: Course[] = [];
    const otherCourses: Course[] = [];

    courses.forEach(course => {
        if (course.createdBy === user?.id) {
            ownCourses.push(course);
        } else
        if (stageOfCourse(course) === "notBought") {
            otherCourses.push(course);
        } else {
            myCourses.push(course);
        }
    });

    return [
        { title: "Створені мною", courses: ownCourses },
        { title: "Мої курси", courses: myCourses },
        { title: "Доступні курси", courses: otherCourses }
    ];
}

export default function CourseCatalog() {
    const { courses, status } = useCourses({ onlyActive: true });
    const { user } = useAppSelector(state => state.account);

    if (status.includes('pending')) return <LoadingComponent />

    const splitedCourses = splitCourses(courses, user);

    return (
        <div style={{ display: 'flex', flexDirection: 'column', gap: '16px' }}>
            {splitedCourses.map((courseCategory, index) =>
                courseCategory.courses.length > 0 && <CourseCategory
courseCategory={courseCategory} key={index + 1} />
            )}
        </div>
    )
}

interface Props {
    courseCategory: { title: string, courses: Course[] },
}

export default function CourseCategory({ courseCategory }: Props) {
    const {title, courses} = courseCategory;

    return (
        <div>
            <Typography variant="h5">{title}</Typography>
            {courses.length === 0
                ? <Typography>Курсів не знайдено</Typography>
                : <div style={{ display: 'flex', flexWrap: 'wrap', gap: '16px',
justifyContent: 'center', marginTop: '12px' }}>
                    {courses.sort((a, b) => a.id - b.id).map((course) =>
                        <CourseCard courseId={course.id} key={course.id + 1} />
                    )}
                </div>
            }
        </div>
    )
}

```

```

}

interface Props {
  courseId: number;
}

export default function CourseCard({ courseId }: Props) {
  const course = useAppSelector(state => courseSelectors.selectById(state, courseId!));

  return (
    <>
      <Card sx={{
        width: '90%',
        maxWidth: '300px',
        minWidth: '250px',
        boxShadow: 3,
        borderRadius: 2,
        p: 1,
        display: 'flex',
        flexDirection: 'column',
      }}>
        <CardContent sx={{ flexGrow: 1 }}>
          <Typography gutterBottom variant="h5" component="div">
            {course?.title}
          </Typography>
          <Typography variant="body2" color="text.secondary">
            {course?.description}
          </Typography>
          <Typography variant="h6" sx={{ mt: '5px' }}>
            Розділів: {course?.sections.length} <br />
            Ціна: <Box component="span"
color="secondary.main">{course?.priceFull} грн</Box>
            <Box component="span" sx={{ display: 'block', fontSize: '1rem',
lineHeight: 1.2, mt: '4px' }}>або {course?.priceMonthly} грн за розділ, якщо купувати
частинами</Box>
          </Typography>
        </CardContent>
        <CardActions sx={{ display: 'flex', justifyContent: 'flex-end' }}>
          <Button component={Link} to={` /course/${course?.id}`} size="small">
            {stageOfCourse(course) === "notBought" ? "Теми уроків" :
"Проходити курс"}
          </Button>
          {stageOfCourse(course) === "notBought" &&
            <Button
              component={Link}
              to={` /checkout/${courseId}`}
              size="small"
              variant="contained"
            >
              Купити
            </Button>
          }
        </CardActions>
      </Card >
    </>
  )
}

type Stage = "notBought" | "boughtInPart" | "boughtInFull";

export default function stageOfCourse(course: Course | undefined): Stage {
  if (course?.sections.every(value => value.isAvailable === true)) {
    return "boughtInFull";
  } else if (course?.sections.some(value => value.isAvailable === true)) {
    return "boughtInPart";
  }
}

```

```

    } else {
      return "notBought";
    }
  }

export default function CourseDetails() {
  const { course, status, courseLoaded, lessonParams } = useCourse();

  useEffect(() => {
    if (course) {
      const index = firstUncompletedSection();
      setOpenIndex(index);
    }
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [course]);

  const firstUncompletedSection = () => {
    if (course) {
      for (let i = 0; i < course.sections.length; i++) {
        if (!course.sections[i].lessons.every(l =>
          (l.urlPractice === "" || l.isPracticeCompleted)
          && (l.urlTheory === "" || l.isTheoryCompleted)
          && l.testScore
        ))
          return course.sections[i].id;
      }
    }
    return -1;
  }

  const [openIndex, setOpenIndex] = useState(firstUncompletedSection());

  const handleItemClick = (index: number) => {
    setOpenIndex(openIndex !== index ? index : -1);
  };

  if ((!course || course.sections.length === 0) && !courseLoaded) {
    if (!lessonParams || status.includes('pending')) return <LoadingComponent />;
    return <NotFound />;
  }

  const isCourseBought = stageOfCourse(course) !== "notBought";

  return (
    <>
      <Box sx={{ display: 'flex', justifyContent: 'start', mb: '10px' }}>
        <Button startIcon={<ArrowBackIcon />} variant="outlined" component={Link}
to={`~/course`} >Назад до курсів</Button>
      </Box>
      <Box sx={{ display: 'flex', flexWrap: 'wrap', justifyContent: isCourseBought
? 'start' : 'space-between', alignItems: "center", gap: '10px' }}>
        <Typography variant="h5">{course?.title}</Typography>
        {isCourseBought
          ? <Filters />
          : <Button
            component={Link}
            to={`~/checkout/${course?.id}`}
            size="small"
            variant="contained"
            sx={{ p: '4px 20px' }}
          >
            Купити
          </Button>}
      </Box>
    </>
  )

```

```

        <List className="list-border" sx={{ p: "0px", m: "16px 0px" }}>
          {course && course.sections.map((section) =>
            section.lessons.length !== 0 ?
              !courseLoaded ?
                <SectionSkeleton
                  key={section.id}
                  section={section}
                  isOpen={openIndex === section.id}
                /> :
                <SectionItem
                  key={section.id}
                  section={section}
                  isOpen={openIndex === section.id}
                  onItemClick={handleItemClick}
                /> : <></>
              )}
        </List>
      </>
    )
  }
}

export function isAvailable(course: Course | undefined, lessonId: string) {
  const section = course ? course.sections.find(s => s.lessons.some(l => l.id ===
    parseInt(lessonId))) : null;
  return section?.isAvailable;
}

export function findLessonById(course: Course, lessonId: number): Lesson | null {
  for (const section of course.sections) {
    for (const lesson of section.lessons) {
      if (lesson.id === lessonId) {
        return lesson;
      }
    }
  }
  return null;
}

function isLoading(course: Course | undefined, courseLoaded: boolean, lessonParams:
LessonParams, status: string) {
  return (!course || !course.sections?.length) && !courseLoaded && (!lessonParams ||
status.includes('pending'));
}

function isNotFound(course: Course | undefined, lesson: Lesson | null, status: string) {
  return (!course || !course.sections?.length || !lesson) &&
!status.includes('pending');
}

export default function LessonDetails() {
  const { course, status, courseLoaded, lessonParams } = useCourse();

  const { lessonId } = useParams<{ courseId: string, lessonId: string }>();
  const lesson = course ? findLessonById(course, parseInt(lessonId!)) : null;

  if (isLoading(course, courseLoaded, lessonParams, status)) return <LoadingComponent
/>;

  if (isNotFound(course, lesson, status) || !isAvailable(course, lessonId!)) return
<NotFound />;

  return (
    lesson ? (
      <>
        <Box sx={{ display: 'flex', justifyContent: 'start', mb: '10px' }}>

```

```

        <Button startIcon={<ArrowBackIcon />} variant="outlined"
component={Link} to={` /course/${course?.id}`}>Назад до курсу</Button>
    </Box>
    <Typography variant="h5">{lesson.number}. {lesson.title}</Typography>
    <Typography variant="body1">{lesson.description}</Typography>
    <Box className="box-container">
        <VideoWithCheckbox videoNumber={0} />
        <VideoWithCheckbox videoNumber={1} />
    </Box>
    <Tests />
    </>
  ) : null
)
}

export default function Tests() {
  const { courseId, lessonId } = useParams<{ courseId: string, lessonId: string }>();
  const course = useAppSelector(state => courseSelectors.selectById(state, courseId!));
  const lesson = course ? findLessonById(course, parseInt(lessonId!)) : null;

  return (
    <>
      <Typography variant="h5" sx={{ mt: "5px", mb: "10px" }}>
        Результат тестування: {(lesson?.testScore !== null) ?
`${lesson?.testScore.toFixed(2)}%` : "ще не складено"}
      </Typography>
      <Button component={Link} to={` /course/${courseId}/lesson/${lessonId}/test`}
variant="contained" disableElevation>
        Почати тест
      </Button>
    </>
  )
}

interface Props {
  videoNumber: number;
}

function isLoading(lessonParams: LessonParams, status: string, courseStatus: string) {
  return !lessonParams || status.includes('pending') ||
courseStatus.includes('pending');
}

function isNotFound(status: string, courseStatus: string, previousLessonId: number,
lessonId: number, tests: Test[]) {
  return !status.includes('pending') && !courseStatus.includes('pending') &&
(previousLessonId !== lessonId || tests.length === 0);
}

interface Props {
  formattedTime: string;
  lessonId: number;
  testScore: number | null;
}

export default function Results({ formattedTime, lessonId, testScore }: Props) {
  const topPercent = 25;
  const [statistics, setStatistics] = useState<UserStatistic>();

  useEffect(() => {
    const fetchStatistics = async () => {
      try {
        const data = await agent.Test.getUserStatistic({ lessonId, topPercent });
        setStatistics(data);
      }
    }
  });
}

```

```

        } catch (error) {
            console.error("Error fetching statistics:", error);
        }
    };

    fetchStatistics();
}, [lessonId, topPercent]);

const getMessage = () => {
    if (!testScore || !statistics) return;

    if (testScore < 50)
        return 'Схоже, вам є над чим попрацювати. Спробуйте повторити тему для кращого засвоєння.';

    if (statistics.isUserInTopPercent)
        return `ЮХУ!.. Ви серед топ-${topPercent}% учасників з найкращими результатами. Продовжуйте в тому ж дусі!`;

    if (testScore >= 80)
        return 'Ви отримали відмінний результат – 80% і більше! Чудова робота!';

    if (statistics.isScoreHigherThanAverage)
        return 'Ваш результат вище середнього серед інших учасників! Молодець!';

    return 'Супер! Це гарний показник, але ще є куди прямувати!'
}

return (<>
    <Paper elevation={3} sx={{ p: 3, m: '24px 0px 8px', borderRadius: 2 }}>
        <Box>
            <Typography variant="h6" sx={{ fontWeight: 'bold', color: 'primary.main' }}>
                Вітаю! Твій результат: {testScore && testScore.toFixed(2)}%
            </Typography>
            <Typography variant="body1" sx={{ mt: 1, color: 'text.secondary' }}>
                {getMessage()}
            </Typography>
            <Typography variant="caption" sx={{ mt: 1, display: 'block', color:
'gray' }}>
                Час виконання: {formattedTime}
            </Typography>
        </Box>
    </Paper>
</>);
}

interface ButtonCardProps {
    option: Option;
    checked: string;
    handleChange: (id: string) => void;
    isStepCompleted: boolean;
    answerId: number;
}

interface Props {
    course?: Course;
    cancelEdit: () => void;
    handleSelectLesson: (section: Section | undefined) => (lesson: Lesson | undefined) => void;
    handleSelectTests?: (lesson: Lesson | undefined) => void;
    setSelectedCourse: (course: Course | undefined) => void;
}

```

```

export default function CourseForm({ course: givenCourse, cancelEdit, handleSelectLesson,
handleSelectTests, setSelectedCourse }: Props) {
  const { course: fullCourse } = useCourse(givenCourse?.id);
  const course = fullCourse ?? givenCourse;
  const dispatch = useAppDispatch();
  const theme = useTheme();
  const isMobile = useMediaQuery(theme.breakpoints.down('sm'));

  const { control, reset, handleSubmit, formState: { isSubmitting, isDirty } } =
useForm({
  resolver: yupResolver<any>(courseValidationSchema)
});

  useEffect(() => {
    if (course) reset(course);
  }, [course, reset]);

  const handleSubmitData = async (data: FieldValues) => {
    try {
      let response: Course;
      if (course) {
        response = await agent.Course.update(course.id, data);
        cancelEdit();
      } else {
        response = await agent.Course.create(data);
        setSelectedCourse(response);
      }

      dispatch(setCourse(response));
    } catch (error) {
      console.log(error);
    }
  }

  return (
    <Box sx={{ p: isMobile ? 2 : 4 }}>
      <Box sx={{ display: 'flex', justifyContent: 'start' }}>
        <Button startIcon={<ArrowBack />} variant="outlined"
onClick={cancelEdit}>Назад</Button>
      </Box>
      <Typography variant="h4" gutterBottom sx={{ mt: '16px' }}>
        Курс
      </Typography>
      <form onSubmit={handleSubmit(handleSubmitData)} style={{marginTop: '16px'}}>
        <Grid container spacing={3}>
          <Grid item xs={12} sm={4}>
            <AppTextInput control={control} name='title' label='Назва курсу'
/>
          </Grid>
          <Grid item xs={12} sm={4}>
            <AppTextInput type="number" control={control} name='priceFull'
label='Повна ціна' />
          </Grid>
          <Grid item xs={12} sm={4}>
            <AppTextInput type="number" control={control} name='priceMonthly'
label='Ціна розділу' />
          </Grid>
          <Grid item xs={12}>
            <AppTextInput multiline={true} rows={2} control={control}
name='description' label='Опис' />
          </Grid>
        </Grid>
      <Box display='flex' justifyContent='space-between' sx={{ mt: 3 }}>

```

```

        <Button onClick={cancelEdit} variant='contained'
color='inherit'>Відмінити</Button>
        <LoadingButton loading={isSubmitting} type="submit"
variant='contained' color='success' disabled={!isDirty}>Зберегти</LoadingButton>
      </Box>
    </form>
    <Box display='flex' justifyContent='space-between'>
      <Typography sx={{ mt: '24px' }} variant='h4'>Розділи</Typography>
      { /* <Button onClick={() => setEditMode('course')} sx={{ m: 2 }}
size='large' variant='contained'>Створити</Button> */ }
    </Box>
    <TableContainer component={Paper} sx={{ mt: '16px' }}>
      <Table>
        <TableBody>
          {course?.sections.map((section, index) =>
            <SectionForm section={section} courseId={course?.id}
key={index} handleSelectLesson={handleSelectLesson(section)}
handleSelectTests={handleSelectTests} />)}
          <SectionForm
            courseId={course?.id}
            numberOfNewSection={
              course?.sections ?
                course.sections.reduce((max, section) =>
section.number > max ? section.number : max, 0) + 1
              : 1 } />
          </TableBody>
        </Table>
      </TableContainer>
    </Box>

export default function TableOfSectionLessons({ section, handleSelectLesson,
handleSelectTests }: Props) {
  const theme = useTheme();
  const dispatch = useAppDispatch();
  const isMobile = useMediaQuery(theme.breakpoints.down('sm'));
  const [loadingState, setLoadingState] = useState<LoadingState>({});

  const setLoading = (id: number, action: 'submit' | 'delete', isLoading: boolean) => {
    setLoadingState(prevState => ({
      ...prevState,
      [id]: { ...prevState[id], [action]: isLoading }
    }));
  };

  const handleDeleteLesson = async (id: number) => {
    setLoading(id, 'delete', true);

    try {
      if (section) {
        await agent.Section.update(section.id, { ...section, lessonIdsToDelete:
[id] });
        dispatch(removeLesson({ id, sectionId: section.id }));
      }
    } catch (error) {
      console.log(error);
    } finally {
      setLoading(id, 'delete', false);
    }
  };

  return (
    <>
      {section &&

```

```

        <TableRow>
          <TableCell style={{ padding: isMobile ? 0 : '0 16px 0 40px' }}
colSpan={4}>
            <Table>
              <TableHead>
                <TableRow>
                  <TableCell align="left" sx={{ width: '50%' }}>Назва
                    <TableCell align="right" sx={{ width: '15%'
}}>Відео</TableCell>
                    <TableCell align="right" sx={{ width: '35%'
}}></TableCell>
                </TableRow>
              </TableHead>
              <TableBody>
                {section?.lessons?.map((lesson) => (
                  <LessonLine key={lesson.id} lesson={lesson}
loadingState={loadingState} handleSelectLesson={handleSelectLesson}
handleDeleteLesson={handleDeleteLesson} handleSelectTests={handleSelectTests}/>
                ))}
                <LessonLine handleSelectLesson={handleSelectLesson} />
              </TableBody>
            </Table>
          </TableCell>
        </TableRow>
      </>
    }
  )
}

export const courseValidationSchema = yup.object({
  title: yup.string().required(),
  priceFull: yup.number().required().moreThan(0),
  priceMonthly: yup.number().required().moreThan(0),
  description: yup.string().required()
})

export const sectionValidationSchema = yup.object({
  title: yup.string().required(),
  description: yup.string()
})

export const questionValidationSchema = yup.object({
  question: yup.string().required()
})

const FILE_SIZE_LIMIT = 100000000 // 100MB
const SUPPORTED_FORMATS = [
  'video/mp4'
]

const videoSchema = yup
  .mixed()
  .test('fileSize', 'File too large', (file: any) => {
    return !file || (file && file.size <= FILE_SIZE_LIMIT)
  })
  .test('fileFormat', 'Unsupported Format', (file: any) => {
    if (!file) return true
    return SUPPORTED_FORMATS.includes(file.type)
  })
  .notRequired()

export const lessonValidationSchema = yup.object({

```

```

    title: yup.string().required(),
    description: yup.string(),
    theory: videoSchema,
    practice: videoSchema
  })

interface Props {
  lesson?: Lesson;
  cancelEdit: () => void;
  section: Section;
  numberOfNewLesson: number
}

export default function LessonForm({ lesson, cancelEdit, section, numberOfNewLesson:
number }: Props) {
  const dispatch = useAppDispatch();
  const [theoryPreviewUrl, setTheoryPreviewUrl] = useState<string |
undefined>(undefined);
  const [practicePreviewUrl, setPracticePreviewUrl] = useState<string |
undefined>(undefined);
  const theme = useTheme();
  const isMobile = useMediaQuery(theme.breakpoints.down('sm'));

  const { control, reset, handleSubmit, formState: { isDirty, isSubmitting } } =
useForm({
    resolver: yupResolver<any>(lessonValidationSchema)
  });

  useEffect(() => {
    if (lesson && !isDirty) reset(lesson);
    return () => {
      if (theoryPreviewUrl) URL.revokeObjectURL(theoryPreviewUrl);
      if (practicePreviewUrl) URL.revokeObjectURL(practicePreviewUrl);
    }
  }, [isDirty, lesson, practicePreviewUrl, reset, theoryPreviewUrl]);

  const handleSubmitData = async (data: FieldValues) => {
    try {
      let response: Lesson;
      if (lesson) {
        response = await agent.Lesson.update(lesson.id, data);
      } else {
        const newLesson = { ...data, number }
        response = await agent.Lesson.create(newLesson);
        await agent.Section.update(section.id, { ...section, lessonIdsToAdd:
[response.id] })
      }

      cancelEdit();
      dispatch(setLesson({ lesson: response, sectionId: section.id }));
    } catch (error) {
      console.log(error);
    }
  }

  return (
    <Box sx={{ p: isMobile ? 2 : 4 }}>
      <Box sx={{ display: 'flex', justifyContent: 'start' }}>
        <Button startIcon={<ArrowBack />} variant="outlined"
onClick={cancelEdit}>Назад</Button>
      </Box>
      <Typography variant="h4" gutterBottom sx={{ mb: 4, mt: '16px' }}>
        Урок
      </Typography>
    </Box>
  )
}

```

```

        <form onSubmit={handleSubmit(handleSubmitData)}>
            <Grid container spacing={3}>
                <Grid item xs={12} sm={12}>
                    <AppTextInput control={control} name='title' label='Назва уроку'
/>
                </Grid>
                <Grid item xs={12}>
                    <AppTextInput multiline={true} rows={4} control={control}
name='description' label='Опис' />
                </Grid>
                <Grid item xs={12}>
                    <Typography variant="h5" gutterBottom>
                        Теорія
                    </Typography>
                    <Box display='flex' flexWrap='wrap' sx={{ gap: '10px' }}
justifyContent='space-between' alignItems='center'>
                        <AppDropzone control={control} name='theoryFile'
setPreviewUrl={setTheoryPreviewUrl} currentPreviewUrl={theoryPreviewUrl} />
                        <VideoPreview newVideoUrl={theoryPreviewUrl}
initialVideoUrl={lesson?.urlTheory} />
                    </Box>
                </Grid>
                <Grid item xs={12}>
                    <Typography variant="h5" gutterBottom>
                        Практика
                    </Typography>
                    <Box display='flex' flexWrap='wrap' sx={{ gap: '10px' }}
justifyContent='space-between' alignItems='center'>
                        <AppDropzone control={control} name='practiceFile'
setPreviewUrl={setPracticePreviewUrl} currentPreviewUrl={practicePreviewUrl} />
                        <VideoPreview newVideoUrl={practicePreviewUrl}
initialVideoUrl={lesson?.urlPractice} />
                    </Box>
                </Grid>
            </Grid>
            <Box display='flex' justifyContent='space-between' sx={{ mt: 3 }}>
                <Button onClick={cancelEdit} variant='contained'
color='inherit'>Відмінити</Button>
                <LoadingButton loading={isSubmitting} type="submit"
variant='contained' color='success'>Зберегти</LoadingButton>
            </Box>
        </form>
    </Box>
)
}

const VideoPreview = ({ initialVideoUrl, newVideoUrl }: Props) => {
    const [showVideo, setShowVideo] = useState(false);

    useEffect(() => {
        if (newVideoUrl) {
            setShowVideo(true);
        }
    }, [newVideoUrl]);

    function getCloudinaryVideoThumbnail(videoUrl: string): string {
        const urlParts = videoUrl.split('/');

        urlParts.splice(6, 0, 'so_0');

        const lastPartIndex = urlParts.length - 1;
        urlParts[lastPartIndex] = urlParts[lastPartIndex].replace(/\.w+$/, '.jpg');

        return urlParts.join('/');
    }

```

```

    }

    const handleDownloadClick = () => {
      setShowVideo(true);
    };

    const videoKey = newVideoUrl || initialVideoUrl;

    return (
      <>
        {showVideo ? (
          <video key={videoKey} style={{ maxWidth: '100%', maxHeight: 200, flex: '0
1 auto' }} controls>
            <source src={newVideoUrl || initialVideoUrl} type="video/mp4" />
            Ваш браузер не підтримує відео тег.
          </video>
        ) : initialVideoUrl ? (
          <Button
            onClick={handleDownloadClick}
            style={{
              display: 'flex',
              flexDirection: 'column',
              alignItems: 'center',
              maxWidth: '100%',
              width: '300px',
              height: '200px',
              backgroundImage:
`url(${getCloudinaryVideoThumbnail(initialVideoUrl)})`,
              backgroundSize: 'cover',
              backgroundPosition: 'center',
              border: '1px solid black'
            }}>
            <PlayArrow sx={{ fontSize: '70px' }} htmlColor="#ce7556" />
          </Button>
        ) : null}
      </>
    );
  }

  export default VideoPreview;

```